

# **Cloud IAM Sentinel: A Hybrid Framework for AWS Identity Threat Detection**

A report for a Graduation Project 2 — submitted in partial fulfillment of the requirements for obtaining the Bachelor's degree in Artificial Intelligence Engineering – Intelligent Information Security Systems Engineering

## **Prepared by:**

Elyana Koryakus Koryakos  
Khalid Moataz AL Nabulsi  
Gabriella Majed Haddad

## **Supervised by:**

Dr.Christine Zenieh  
Eng. Mouhamad Amjad Al Safadi

**2026 – 2027**

## Cloud IAM Sentinel

### إطار عمل هجين للكشف عن تهديدات الهوية في خدمات AWS

تقرير مشروع تخرج 2 — مقدّم استكمالاً لمتطلبات الحصول على درجة الإجازة في هندسة الذكاء الاصطناعي – هندسة نظم أمن المعلومات الذكية

#### إعداد:

اليانا قرياقوس قوريكس  
خالد معتز النابلسي  
غابريلا ماجد حداد

#### إشراف:

د. كريستين زينية  
م. محمد أمجد الصفدي

2026 – 2027

# Table of Contents

|                                                                  |    |
|------------------------------------------------------------------|----|
| <b>Introduction</b>                                              | 16 |
| <b>Chapter One: Theoretical Framework</b>                        | 17 |
| 1-1 Introduction                                                 | 18 |
| 1-2 Foundational Security Concepts                               | 18 |
| 1-2-1 Authentication and Authorization                           | 18 |
| 1-2-2 The Principle of Least Privilege                           | 18 |
| 1-2-3 Defense in Depth                                           | 18 |
| 1-2-4 Attack Surface and Threat Model                            | 18 |
| 1-3 Cloud Computing and the AWS Platform                         | 19 |
| 1-3-1 Definition of Cloud Computing                              | 19 |
| 1-3-2 Service Models — IaaS, PaaS, and SaaS                      | 19 |
| 1-3-3 Deployment Models — Public, Private, Hybrid, and Community | 19 |
| 1-3-4 The AWS Shared Responsibility Model                        | 19 |
| 1-3-5 The AWS Account as Security Boundary                       | 20 |
| 1-3-6 AWS Services                                               | 20 |
| 1-4 AWS Identity and Access Management (IAM)                     | 21 |
| 1-4-1 The IAM Service                                            | 21 |
| 1-4-2 IAM Principal Types                                        | 21 |
| 1-4-2-1 The Root User                                            | 21 |
| 1-4-2-2 IAM Users                                                | 22 |
| 1-4-2-3 IAM Groups                                               | 22 |
| 1-4-2-4 IAM Roles                                                | 22 |
| 1-4-2-5 Federated Identities                                     | 22 |
| 1-4-2-6 Service Principals and Service-Linked Roles              | 22 |
| 1-4-3 Authentication Credentials                                 | 22 |
| 1-4-4 Multi-Factor Authentication                                | 23 |
| 1-5 IAM Policies                                                 | 23 |
| 1-5-1 Policy Documents and JSON Structure                        | 23 |
| 1-5-2 Managed Policies and Inline Policies                       | 23 |
| 1-5-3 Identity-Based, Resource-Based, and Trust Policies         | 23 |
| 1-5-4 Policy Elements                                            | 23 |
| 1-5-5 NotAction and NotResource                                  | 24 |

|        |                                                            |    |
|--------|------------------------------------------------------------|----|
| 1-5-6  | Amazon Resource Names (ARNs).....                          | 24 |
| 1-5-7  | Action Wildcards.....                                      | 24 |
| 1-5-8  | Policy Variables .....                                     | 24 |
| 1-6    | Policy Evaluation Logic .....                              | 24 |
| 1-6-1  | The Six Policy Classes .....                               | 24 |
| 1-6-2  | The Deterministic Evaluation Algorithm.....                | 24 |
| 1-6-3  | Explicit Deny, Explicit Allow, and Implicit Deny.....      | 25 |
| 1-6-4  | AWS Security Token Service and Assume Role .....           | 25 |
| 1-6-5  | Service Control Policies (SCPs).....                       | 25 |
| 1-6-6  | Permission Boundaries.....                                 | 25 |
| 1-6-7  | Session Policies .....                                     | 25 |
| 1-7    | Access Control Models.....                                 | 26 |
| 1-7-1  | DAC, MAC, and RBAC Families.....                           | 26 |
| 1-7-2  | Role-Based Access Control.....                             | 26 |
| 1-7-3  | Attribute-Based Access Control.....                        | 26 |
| 1-7-4  | Attribute Mutability.....                                  | 28 |
| 1-7-5  | Implementation of RBAC and ABAC in AWS IAM.....            | 28 |
| 1-8    | IAM Privilege Escalation .....                             | 29 |
| 1-8-1  | Definition .....                                           | 29 |
| 1-8-2  | Common Escalation Primitives.....                          | 29 |
| 1-8-3  | Graph-Based Reachability Analysis.....                     | 29 |
| 1-8-4  | Tag Mutation as State Transition .....                     | 30 |
| 1-8-5  | Bounded Breadth-First Search over Joint State .....        | 30 |
| 1-9    | Dynamic Detection through CloudTrail .....                 | 30 |
| 1-9-1  | The CloudTrail Service .....                               | 30 |
| 1-9-2  | Management Events and Data Events .....                    | 31 |
| 1-9-3  | The CloudTrail Event Schema .....                          | 31 |
| 1-10   | Adversary Tactics: MITRE ATT&CK.....                       | 31 |
| 1-10-1 | The MITRE ATT&CK Framework .....                           | 31 |
| 1-10-2 | Tactics, Techniques, Sub-Techniques, and Procedures .....  | 31 |
| 1-10-3 | The ATT&CK for Cloud (IaaS) Matrix .....                   | 31 |
| 1-11   | Posture Benchmarks .....                                   | 32 |
| 1-11-1 | The CIS AWS Foundations Benchmark .....                    | 32 |
| 1-11-2 | AWS Security Hub Foundational Security Best Practices..... | 32 |

|        |                                                                |    |
|--------|----------------------------------------------------------------|----|
| 1-11-3 | NIST SP 800-53 Revision 5 and Control Structure .....          | 32 |
| 1-12   | Large Language Models and Retrieval-Augmented Generation ..... | 32 |
| 1-12-1 | Large Language Models .....                                    | 32 |
| 1-12-2 | The Hallucination Problem .....                                | 32 |
| 1-12-3 | Retrieval-Augmented Generation .....                           | 33 |
| 1-12-4 | Dense Vector Embeddings and FAISS .....                        | 33 |
| 1-12-5 | Citation Grounding and Hallucination Prevention .....          | 33 |
| 1-13   | Statistical Foundations for Evaluation .....                   | 33 |
| 1-13-1 | Counts, Proportions, and Recall .....                          | 33 |
| 1-13-2 | Wilson Score Confidence Interval .....                         | 34 |
| 1-13-3 | McNemar's Exact Test .....                                     | 34 |
| 1-13-4 | Bonferroni Adjustment .....                                    | 34 |
| 1-14   | Conclusion .....                                               | 34 |
|        | <b>Chapter Two: Previous Studies</b> .....                     | 36 |
| 2-1    | Introduction .....                                             | 37 |
| 2-2    | Foundational IAM Surveys .....                                 | 37 |
| 2-3    | IAM Audit and Posture Frameworks .....                         | 37 |
| 2-4    | Graph-Based IAM Privilege-Escalation Analysis .....            | 38 |
| 2-5    | Large-Language-Model-Based IAM Threat Detection .....          | 38 |
| 2-6    | CSPM Quality and False-Positive Reduction .....                | 38 |
| 2-7    | Comparative Analysis of Research Papers .....                  | 39 |
| 2-8    | Comparative Analysis of Practical Tools .....                  | 40 |
| 2-9    | Discussion and Justification of the Proposed Framework .....   | 41 |
| 2-10   | Conclusion .....                                               | 41 |
|        | <b>Chapter Three: Proposed Solution</b> .....                  | 42 |
| 3-1    | Introduction .....                                             | 43 |
| 3-2    | Framework Architecture Overview .....                          | 43 |
| 3-3    | Data Collection Layer .....                                    | 45 |
| 3-3-1  | IAM Configuration Collection .....                             | 45 |
| 3-3-2  | CloudTrail Event Collection .....                              | 45 |
| 3-3-3  | Unified Inventory .....                                        | 45 |
| 3-4    | Static IAM Analysis .....                                      | 46 |
| 3-4-1  | Rule Catalog Architecture .....                                | 46 |
| 3-4-2  | RBAC Posture Rules .....                                       | 46 |

|       |                                                                             |    |
|-------|-----------------------------------------------------------------------------|----|
| 3-4-3 | ABAC Posture Rules.....                                                     | 46 |
| 3-4-4 | Output Schema .....                                                         | 46 |
| 3-5   | Privilege-Escalation Path Analysis.....                                     | 47 |
| 3-5-1 | Graph Construction .....                                                    | 47 |
| 3-5-2 | IAM Condition Evaluation .....                                              | 47 |
| 3-5-3 | Tag-Mutation Edges and Joint-State Reachability .....                       | 47 |
| 3-5-4 | Bounded Breadth-First Search .....                                          | 47 |
| 3-5-5 | Service Control Policy and Permission Boundary Filtering .....              | 48 |
| 3-5-6 | Output Schema .....                                                         | 48 |
| 3-6   | Phase Three — Deterministic Correlation and Grounded LLM Reasoning .....    | 48 |
| 3-6-1 | CloudTrail Event Ingestion .....                                            | 48 |
| 3-6-2 | Deterministic MITRE ATT&CK Mapping.....                                     | 48 |
| 3-6-3 | Correlation with Static Outputs.....                                        | 48 |
| 3-6-4 | Retrieval-Augmented Generation Layer .....                                  | 49 |
| 3-6-5 | External Knowledge Base .....                                               | 49 |
| 3-6-6 | LLM Adapter and Backend Selection.....                                      | 49 |
| 3-6-7 | Hallucination Guard .....                                                   | 49 |
| 3-6-8 | Final Correlated Alerts .....                                               | 49 |
| 3-7   | Inter-Phase Data Flow and Output Formats .....                              | 50 |
| 3-8   | Conclusion .....                                                            | 50 |
|       | <b>Chapter Four: Practical Implementation</b> .....                         | 51 |
| 4-1   | Introduction .....                                                          | 52 |
| 4-2   | Implementation Environment .....                                            | 52 |
| 4-2-1 | Programming Language and Libraries .....                                    | 52 |
| 4-2-2 | Read-Only AWS Permission Policy .....                                       | 52 |
| 4-3   | Phase One — Static IAM Rule Catalog .....                                   | 52 |
| 4-3-1 | Rule Organization into Detection Families .....                             | 52 |
| 4-3-2 | Family A — Identity Policy Structure .....                                  | 55 |
| 4-3-3 | Family B — Attribute-Based Access Control.....                              | 55 |
| 4-3-4 | Family C — Credential and Authentication State.....                         | 56 |
| 4-3-5 | Family D — Credential Lifecycle.....                                        | 56 |
| 4-3-6 | Family E — Account-Level Password Policy.....                               | 56 |
| 4-3-7 | Family F — Provisioning Hygiene .....                                       | 57 |
| 4-3-8 | Limitations of Network-Layer Controls in Substituting Phase One Rules ..... | 57 |

|       |                                                                                                    |    |
|-------|----------------------------------------------------------------------------------------------------|----|
| 4-3-9 | Evaluation of AWS Native IAM Security Services .....                                               | 58 |
| 4-4   | Phase Two — Privilege-Graph Implementation .....                                                   | 58 |
| 4-4-1 | Graph Construction .....                                                                           | 58 |
| 4-4-2 | IAM Condition Evaluator.....                                                                       | 59 |
| 4-4-3 | Tag-Mutation Edges and Joint-State Reachability .....                                              | 59 |
| 4-4-4 | Bounded Breadth-First Search .....                                                                 | 60 |
| 4-4-5 | Service Control Policy and Permission Boundary Filtering .....                                     | 61 |
| 4-5   | Phase Three — Correlation and Grounded Reasoning Implementation .....                              | 61 |
| 4-5-1 | CloudTrail Event Ingestion and Normalization .....                                                 | 61 |
| 4-5-2 | Deterministic MITRE ATT&CK Mapping.....                                                            | 61 |
| 4-5-3 | Correlation with Phase One and Phase Two Outputs .....                                             | 62 |
| 4-5-4 | Knowledge Base and Vector Index.....                                                               | 63 |
| 4-5-5 | Large Language Model Adapter and Two Backends.....                                                 | 63 |
| 4-5-6 | Hallucination Guard .....                                                                          | 64 |
| 4-6   | Evaluation Datasets and Ground-Truth Catalogs .....                                                | 65 |
| 4-6-1 | IAM Vulnerable Benchmark.....                                                                      | 65 |
| 4-6-2 | Pathfinding.cloud Catalog.....                                                                     | 65 |
| 4-6-3 | CloudGoat Scenarios.....                                                                           | 65 |
| 4-6-4 | Stratus Red Team Attack Catalog .....                                                              | 65 |
| 4-6-5 | flaws.cloud CloudTrail Dataset.....                                                                | 65 |
| 4-7   | Conclusion .....                                                                                   | 66 |
|       | <b>Chapter Five: Evaluation</b> .....                                                              | 67 |
| 5-1   | Introduction .....                                                                                 | 68 |
| 5-2   | Methodology.....                                                                                   | 68 |
| 5-3   | Phase One - Static IAM Posture Evaluation.....                                                     | 69 |
| 5-3-1 | Evaluating Phase One Accuracy on IAM Vulnerable False-Positive and False-Negative Principals ..... | 69 |
| 5-4   | Phase Two - Privilege-Escalation Graph Evaluation .....                                            | 70 |
| 5-4-1 | Pathfinding.cloud Catalog as Primary Corpus .....                                                  | 70 |
| 5-4-2 | IAM Vulnerable Benchmark as Secondary Corpus .....                                                 | 71 |
| 5-5   | Stratus Red Team Phase Three Recall Evaluation .....                                               | 72 |
| 5-5-1 | Technique One: iam-create-admin-user (Persistence) .....                                           | 73 |
| 5-5-2 | Technique Two: iam-backdoor-user (Credential Access).....                                          | 74 |
| 5-5-3 | Technique Three: iam-backdoor-role (Privilege Escalation) .....                                    | 76 |
| 5-5-4 | Technique Four: iam-create-backdoor-role (Persistence) .....                                       | 77 |

|       |                                                                                            |    |
|-------|--------------------------------------------------------------------------------------------|----|
| 5-5-5 | Technique Five: iam-create-user-login-profile (Persistence).....                           | 78 |
| 5-5-6 | Technique Six: sts-federation-token (Credential Access).....                               | 79 |
| 5-5-7 | Technique Seven: iam-update-user-login-profile (Privilege Escalation).....                 | 81 |
| 5-6   | Integrated End-to-End Validation on CloudGoat Scenarios .....                              | 82 |
| 5-6-1 | Scenario One: iam_enum_basics (Negative Control) .....                                     | 82 |
| 5-6-2 | Scenario Two: iam_privesc_by_key_rotation (Attribute-Based Access-Control Re-Tagging)..... | 84 |
| 5-6-3 | Scenario Three: iam_privesc_by_rollback (Policy-Version Rollback) .....                    | 86 |
| 5-6-4 | Scenario Four: lambda_privesc (PassRole-via-Lambda).....                                   | 89 |
| 5-6-5 | Cross-Technique Aggregate Scoring .....                                                    | 91 |
| 5-7   | The Large-Language-Model Augmentation.....                                                 | 92 |
| 5-8   | Threats to Validity .....                                                                  | 93 |
| 5-9   | Conclusion .....                                                                           | 93 |
|       | <b>Conclusion</b> .....                                                                    | 95 |
| 6-1   | Summary of Contributions .....                                                             | 96 |
| 6-2   | Summary of Results.....                                                                    | 96 |
| 6-3   | Limitations.....                                                                           | 97 |
| 6-4   | Future Work.....                                                                           | 98 |



# List of Tables

Table 2-1: Comparison of Research Papers Against the Proposed Framework

Table 2-2: Comparison of Practical IAM Analysis Tools Against the Proposed Framework

Table 4-1: Role-Based Access Control Rules

Table 4-2: Attribute-Based Access Control Rules (Family B)

Table 5-1: Phase One adversarial-accuracy results (n = 14 pairs).

Table 5-2: Phase Two detection on the Pathfinding.cloud catalog (n = 66 paths).

Table 5-3: Paired McNemar's exact tests on the Pathfinding.cloud detection vector.

Table 5-4: Phase Two detection on IAM Vulnerable (n = 31 paths).

Table 5-5: Aggregate detection and LLM grounding outcomes across CloudGoat and Stratus scenarios.

# List of Figures

- Figure 1: The AWS Shared Responsibility Model [3]
- Figure 2: Basic ABAC Scenario — NIST SP 800-162 [8]
- Figure 3: Core ABAC Mechanisms — Policy Decision Point and Policy Enforcement Point — NIST SP 800-162 [8]
- Figure 4: Enterprise ABAC Scenario — NIST SP 800-162 [8]
- Figure 5: Attribute-Based Access Management — Logical Components and Authentication / Authorization Decision [4]
- Figure 6: Permission Flow Graph (PFG) examples for the 2019 IAM privilege-escalation incident [6]
- Figure 7: Cloud IAM Sentinel as an Assessment Intermediary in the AWS IAM Security-Assessment Workflow
- Figure 8: Cloud IAM Sentinel — Framework Architecture
- Figure 9: Illustrative example of tag-mutation-aware privilege-escalation path discovery
- Figure 10: Phase Three deterministic alert for the iam-create-admin-user headline event.
- Figure 11: LLM-augmented Phase Three alert for the iam-create-admin-user headline event.
- Figure 12: Deterministic Phase Three alert for the CreateAccessKey event in iam-backdoor-user.
- Figure 13: LLM-augmented Phase Three alert for the same CreateAccessKey event.
- Figure 14: Deterministic Phase Three alert for the UpdateAssumeRolePolicy event in iam-backdoor-role.
- Figure 15: LLM-augmented Phase Three alert for the same UpdateAssumeRolePolicy event.
- Figure 16: Deterministic Phase Three alert for the CreateRole event in iam-create-backdoor-role.
- Figure 17: LLM-augmented Phase Three alert for the same CreateRole event.
- Figure 18: Deterministic Phase Three alert for the CreateLoginProfile event in iam-create-user-login-profile.
- Figure 19: LLM-augmented Phase Three alert for the same CreateLoginProfile event.
- Figure 20: Deterministic Phase Three alert for the GetFederationToken event in sts-federation-token.
- Figure 21: LLM-augmented Phase Three alert for the same GetFederationToken event.
- Figure 22: Deterministic Phase Three alert for the UpdateLoginProfile event in iam-update-user-login-profile.
- Figure 23: LLM-augmented Phase Three alert for the same UpdateLoginProfile event.
- Figure 24: Phase One finding on the iam\_enum\_basics scenario actor.
- Figure 25: Phase Three LLM-augmented alert for the iam\_enum\_basics scenario.
- Figure 26: Phase One ABAC re-tagging finding for iam\_privesc\_by\_key\_rotation.
- Figure 27: Phase Three LLM-augmented alert for the iam\_privesc\_by\_key\_rotation headline event.

Figure 28: Phase One privilege-escalation finding for the iam\_privesc\_by\_rollback policy.

Figure 29: Phase Two self-escalation attack path via iam:SetDefaultPolicyVersion.

Figure 30: Phase Three STRONGLY\_CORRELATED LLM-augmented alert for iam\_privesc\_by\_rollback.

Figure 31: Phase One PassRole detector firing on the lambda manager policy.

Figure 32: Phase Two two-hop attack path via sts:AssumeRole and iam:PassRole\_via\_Lambda.

Figure 33: Phase Three LLM-augmented alert on the lambda\_privesc headline event.

## List of Abbreviations and Scientific Terms

| Abbreviation      | Term                                                           |
|-------------------|----------------------------------------------------------------|
| <b>ABAC</b>       | Attribute-Based Access Control                                 |
| <b>ANN</b>        | Approximate Nearest Neighbor                                   |
| <b>API</b>        | Application Programming Interface                              |
| <b>ARN</b>        | Amazon Resource Name                                           |
| <b>ATT&amp;CK</b> | Adversarial Tactics, Techniques, and Common Knowledge          |
| <b>AWS</b>        | Amazon Web Services                                            |
| <b>BFS</b>        | Breadth-First Search                                           |
| <b>CIS</b>        | Center for Internet Security                                   |
| <b>CSPM</b>       | Cloud Security Posture Management                              |
| <b>DAC</b>        | Discretionary Access Control                                   |
| <b>EC2</b>        | Elastic Compute Cloud                                          |
| <b>FAISS</b>      | Facebook AI Similarity Search                                  |
| <b>FSBP</b>       | Foundational Security Best Practices                           |
| <b>GNN</b>        | Graph Neural Network                                           |
| <b>IaaS</b>       | Infrastructure as a Service                                    |
| <b>IAM</b>        | Identity and Access Management                                 |
| <b>JSON</b>       | JavaScript Object Notation                                     |
| <b>KMS</b>        | Key Management Service                                         |
| <b>LLM</b>        | Large Language Model                                           |
| <b>MAC</b>        | Mandatory Access Control                                       |
| <b>MFA</b>        | Multi-Factor Authentication                                    |
| <b>MITRE</b>      | Massachusetts Institute of Technology Research and Engineering |

|             |                                                |
|-------------|------------------------------------------------|
| <b>NIST</b> | National Institute of Standards and Technology |
| <b>PaaS</b> | Platform as a Service                          |
| <b>PDP</b>  | Policy Decision Point                          |
| <b>PEP</b>  | Policy Enforcement Point                       |
| <b>PFG</b>  | Permission Flow Graph                          |
| <b>RAG</b>  | Retrieval-Augmented Generation                 |
| <b>RBAC</b> | Role-Based Access Control                      |
| <b>S3</b>   | Simple Storage Service                         |
| <b>SaaS</b> | Software as a Service                          |
| <b>SCP</b>  | Service Control Policy                         |
| <b>SP</b>   | Special Publication                            |
| <b>STS</b>  | Security Token Service                         |
| <b>TTP</b>  | Tactics, Techniques, and Procedures            |

# Abstract

This project presents Cloud IAM Sentinel, a hybrid three-phase framework for AWS Identity and Access Management (IAM) security analysis. The framework integrates static rule-based posture checking, graph-based privilege-escalation discovery, and dynamic CloudTrail correlation augmented by grounded Large Language Model (LLM) reasoning. It addresses a documented gap in existing cloud security tooling: rule-based Cloud Security Posture Management tools examine policy statements in isolation and may miss multi-step escalation chains, while graph-based privilege-escalation analyzers often treat AWS IAM permissions as either allowed or denied and cannot represent Attribute-Based Access Control (ABAC) paths in which a principal first mutates a tag attribute to satisfy a downstream condition.

Phase One of the framework evaluates a catalogue of Role-Based Access Control posture rules (RBAC) and Attribute-Based Access Control (ABAC), with every rule mapped to the Center for Internet Security AWS Foundations Benchmark (CIS), the AWS Security Hub Foundational Security Best Practices, and the NIST SP 800-53 Rev. 5. Phase Two constructs a privilege-escalation graph that evaluates IAM Condition operators, applies Service Control Policy (SCP) filtering, models tag-mutation actions as state-transition edges, and discovers paths through bounded breadth-first search whose visited set is keyed on the joint state of node and tag values. Phase Three ingests CloudTrail management events, maps them deterministically to MITRE ATT&CK Cloud techniques, retrieves authoritative passages from a curated vector index, and produces grounded alerts through a hallucination-guarded LLM adapter.

The framework was evaluated using externally authored benchmarks and staged AWS attack scenarios. Phase One achieved zero incorrect verdicts on fourteen pre-committed IAM Vulnerable adversarial test pairs: eleven verdicts were strictly correct, and three were acceptable over-approximations reflecting documented limits of static analysis. Phase Two detected 48 of 66 Pathfinding.cloud privilege-escalation paths under the loose criterion and 45 of 66 under the strict criterion, and achieved 100% recall on the nineteen testable IAM Vulnerable paths present in the deployed environment. In end-to-end validation, the framework was tested on four CloudGoat scenarios and seven Stratus Red Team techniques. The Stratus evaluation achieved a Phase Three detection rate of 7/7, while the LLM augmentation achieved a grounding rate of 10/11 headline alerts. These results show that Cloud IAM Sentinel can combine static posture analysis, ABAC-aware privilege-graph reasoning, CloudTrail correlation, and grounded explanation to detect and explain AWS IAM identity-threat patterns across multiple evaluation settings.

**Keywords:** AWS Identity and Access Management; Attribute-Based Access Control; Privilege Escalation; Graph Reachability; MITRE ATT&CK Cloud; Retrieval-Augmented Generation; Hybrid Detection.

## الملخص

يُقدّم هذا المشروع Cloud IAM Sentinel بوصفه إطاراً هجيناً ثلاثي المراحل لتحليل أمن إدارة الهوية والوصول (IAM) في خدمات Amazon Web Services (AWS). يجمع الإطار بين فحص الوضع الأمني الثابت القائم على القواعد، واكتشاف مسارات تصعيد الامتيازات المعتمد على الرسوم البيانية، والترابط الديناميكي مع أحداث CloudTrail مدعّم باستدلال نماذج اللغة الكبيرة (LLM) اعتماداً على مصادر موثوقة. ويُعالج الإطار فجوةً موثقةً في الأدوات الحالية لأمن السحابة، إذ تفحص أدوات إدارة الوضع الأمني للبيئات السحابية القائمة على القواعد كل سياسة بمعزلٍ عن غيرها فتُغفل سلاسل التصعيد متعدد الخطوات، بينما تعامل أدوات تحليل تصعيد الامتيازات القائمة على الرسوم البيانية صلاحيات IAM على أنها إما مسموحة أو مرفوضة، ومن ثَمَّ تعجز عن تمثيل مسارات التحكم بالوصول المعتمد على السمات (ABAC) التي يقوم فيها الكيان الرئيسي أولاً بتعديل قيمة وسمٍ بهدف استيفاء شرطٍ لاحق.

تُقيّم المرحلة الأولى من الإطار مجموعةً من قواعد التحكم بالوصول القائم على الأدوار (RBAC) والتحكم في الوصول القائم على السمات (ABAC)، مع ربط القواعد بمعيار CIS AWS Foundations Benchmark، وممارسات AWS Security Hub، و NIST SP 800-53 Rev. 5. تبني المرحلة الثانية رسماً بيانياً لتصعيد الامتيازات بهدف تحليل معاملات الشروط في IAM، وتطبيق فلتر سياسة التحكم في الخدمات (SCP)، وتمثيل إجراءات تعديل الوسوم كخطوات أساسية لانتقال الحالة، ثم اكتشاف المسارات عبر بحثٍ متّسع محدود العمق مع تسجيل الحالات التي تم الوصول إليها بناءً على الحالة المشتركة للعقدة وقيم الوسوم. أما المرحلة الثالثة، فتستقبل أحداث الإدارة المسجلة في CloudTrail، وتربطها بطريقة حتمية بتقنيات مصفوفة MITRE ATT&CK السحابية، وتسترجع المقاطع الموثوقة من فهرسٍ متّجهي مُعدّ مسبقاً، ثم تُنتج تنبيهاتٍ مُسنّدةً عبر مُحوّل LLM مزوّدٍ بآلية للحد من الهلوسة.

تم تقييم الإطار باستخدام معايير خارجية وسيناريوهات هجوم منظّمة في بيئة AWS. لم تُسجّل المرحلة الأولى أي أحكام خاطئة على أربع عشرة حالة اختبار مصممة لاختبار دقة التحليل من IAM Vulnerable، إذ كانت إحدى عشرة نتيجة صحيحة تماماً، وثلاث نتائج لم تكن خاطئة، لكنها صُنّفت كمخاطر محتملة تعكس حدوداً موثقةً للتحليل الثابت، واكتشفت المرحلة الثانية 48 من أصل 66 مساراً لتصعيد الامتيازات من Pathfinding.cloud عندما كانت شروط المطابقة أقل صرامة، و45 من أصل 66 عندما كانت شروط المطابقة أكثر صرامة، كما حققت استرجاعاً بنسبة 100% على المسارات التسعة عشر القابلة للاختبار من IAM Vulnerable والموجودة في البيئة المنشورة. عند تقييم الإطار بشكل متكامل، طُبّق على أربعة سيناريوهات من CloudGoat وسبع تقنيات من Stratus Red Team. وحقق تقييم Stratus معدل كشف في المرحلة الثالثة قدره 7/7، بينما حققت طبقة LLM معدل grounding قدره 11/10 من التنبيهات الرئيسية. وتبيّن هذه النتائج أن Cloud IAM Sentinel قادر على الجمع بين تحليل الوضع الأمني الثابت، وتحليل مسارات التصعيد مع مراعاة ABAC، وترابط أحداث CloudTrail، والتفسير المسند إلى الأدلة، بهدف كشف وشرح أنماط تهديدات الهوية في AWS IAM في أكثر من بيئة تقييم.

الكلمات المفتاحية: إدارة الهوية والوصول في AWS؛ التحكم بالوصول المعتمد على السمات؛ تصعيد الامتيازات؛ قابلية الوصول في الرسوم البيانية؛ MITRE ATT&CK السحابية؛ التوليد المعزّز بالاسترجاع؛ الكشف الهجين.

# Introduction

Cloud computing dominates modern enterprise workloads, with Amazon Web Services (AWS) leading the market. The security of an AWS account is fundamentally determined by the configuration of its Identity and Access Management (IAM) policies, as IAM serves as the definitive authorization gate between principals and resources. Despite its importance, the rapid growth in cloud adoption has been accompanied by a significant rise in security incidents, the majority of which result from misconfigured identities and overly permissive policies, rather than vulnerabilities in the cloud platform itself.

Three primary tool families address these risks, each with critical limitations. Cloud Security Posture Management (CSPM) tools, such as Prowler and Cloudsplaining, scan configurations against static rules but analyze policy statements in isolation, failing to detect multi-step privilege escalation. Graph-based tools, such as PMapper, model reachability but often treat permissions as either allowed or denied, ignoring IAM Conditions and Attribute-Based Access Control (ABAC). Detection-as-code platforms monitor AWS CloudTrail events reactively but lack the ability to statically predict possible attack paths. These approaches are not integrated, leading to fragmented security visibility and inconsistent risk-severity assessments that are not aligned with standard threat classifications.

This project proposes Cloud IAM Sentinel, a hybrid framework that integrates these domains into a single deterministic pipeline. Operating over a unified AWS inventory, the framework consists of three phases: a static posture phase that evaluates RBAC and ABAC policies against industry benchmarks, a graph phase that constructs a privilege-escalation model incorporating Deny semantics, Service Control Policies (SCPs), and IAM Conditions, with tag mutations explicitly modeled as state transitions, and a dynamic phase that maps CloudTrail events to MITRE ATT&CK techniques. To bridge the gap between raw technical data and actionable insight, the framework employs a Retrieval-Augmented Generation (RAG) pipeline that relies strictly on cited sources, ensuring that all findings are verifiable and free from hallucination.

The core research problem is the absence of an open-source framework capable of correlating multi-step ABAC privilege escalation with real-time dynamic events in a unified analysis model. This project aims to design and implement this framework, show that tag-based ABAC escalation paths can be detected statically through state modeling, and provide an open and reviewable implementation for cybersecurity researchers and practitioners.

This work is important for two main reasons. First, it presents an approach that treats IAM attributes as mutable state, revealing a type of privilege-escalation path that existing tools cannot represent. Second, by attaching authoritative citations to every finding, it ensures that the framework's output is suitable for regulatory audit and forensic analysis.

Cloud IAM misconfiguration is a problem that no existing analysis approach fully solves. This work argues that the problem does not need to be solved by one approach alone, as long as the three main approaches, static posture analysis, graph reachability, and dynamic correlation, are combined using a common set of terms based on trusted sources.



# **Chapter One: Theoretical Framework**

## **1-1 Introduction**

This chapter establishes the theoretical foundations of Cloud IAM Sentinel. It introduces the core security concepts behind every access-control decision, the AWS platform and its IAM service, including principal types, credentials, policies, and evaluation logic, the RBAC and ABAC access-control models and the idea that attributes can be changed, privilege escalation viewed as a limited graph reachability problem, the AWS CloudTrail along with threat classifications and security benchmarks used to label findings, and the foundations of Large Language Models and Retrieval-Augmented Generation with a citation-based grounding method that keeps every output verifiable.

## **1-2 Foundational Security Concepts**

### **1-2-1 Authentication and Authorization**

Authentication is the process of verifying that an entity is who it claims to be, whereas authorization is the process of deciding whether that authenticated entity is permitted to perform a requested action on a requested resource [1]. These two processes are distinct concerns and are typically implemented by separate mechanisms: in AWS, authentication is handled by each service verifying credentials, while authorization is performed by the IAM evaluator, which determines whether a request is allowed or denied by evaluating all applicable policies and rules [2]. The framework reasons exclusively about the authorization surface and assumes that credentials produced by the authentication layer are correctly attributed to the principal whose request they accompany.

### **1-2-2 The Principle of Least Privilege**

The Principle of Least Privilege requires that every principal and every process operate with the minimum set of permissions necessary to perform its function [1]. Least privilege is the central design directive of the AWS IAM service [2]. Any violation of this principle indicates the presence of excessive permissions that can be identified through analysis of access configurations. Accordingly, findings produced by the framework represent instances in which a principal holds more permissions than required within the analyzed environment.

### **1-2-3 Defense in Depth**

Defense in depth is the practice of layering independent security controls so that the failure of any single control does not compromise the system as a whole [1]. AWS realizes defense in depth at the IAM layer through the concurrent application of identity-based policies, resource-based policies, Service Control Policies, Permission Boundaries, and session policies, all of which are included in the framework's reachability modeling [2].

### **1-2-4 Attack Surface and Threat Model**

The attack surface of a system is the set of points at which an unauthorized principal could attempt to exercise a privilege [1]. In AWS, this mainly refers to the IAM authorization layer, where every allowed combination of principal, action, and resource forms part of the surface [2]. The framework aims to identify cases where permissions exceed what is necessary. The threat model assumes an internal authenticated attacker with possibly limited access, operating only through the

AWS API. Attacks that bypass IAM, such as infrastructure compromise, side-channel attacks, or physical access, are not considered.

## **1-3 Cloud Computing and the AWS Platform**

### **1-3-1 Definition of Cloud Computing**

The National Institute of Standards and Technology defines cloud computing as a model for providing easy, on-demand network access, to a shared pool of configurable computing resources that can be rapidly allocated and released with minimal management effort. The five essential characteristics listed in NIST SP 800-145 are on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [1].

### **1-3-2 Service Models — IaaS, PaaS, and SaaS**

NIST identifies three service models for cloud computing. Infrastructure-as-a-Service (IaaS) provides processing, storage, and networks on which the consumer deploys arbitrary software, including operating systems. Platform-as-a-Service (PaaS) provides a managed runtime on which the consumer deploys applications without managing the underlying infrastructure. Software-as-a-Service (SaaS) provides a complete application accessed by the consumer through a thin client. IAM itself is not one of these models, it is a cross-cutting authorization service that governs access to resources across all three [1]. The framework targets the IaaS and PaaS surfaces of AWS, where IAM is the principal authorization control.

### **1-3-3 Deployment Models — Public, Private, Hybrid, and Community**

The four deployment models defined by NIST are public cloud (provisioned for open use by the general public), private cloud (provisioned for exclusive use by a single organization), community cloud (shared by a specific community of consumers), and hybrid cloud (a composition of two or more of the above) [1]. The framework targets public-cloud and hybrid-cloud deployments of AWS, which are the dominant operating models for AWS workloads. Both configurations expose the same AWS API surface, the IAM service, the Security Token Service, and the CloudTrail event stream, that the three phases of the framework consume, whereas private-cloud and community-cloud deployments rely on non-AWS identity systems and are out of scope.

### **1-3-4 The AWS Shared Responsibility Model**

AWS operates under a shared responsibility model in which the provider is responsible for the security of the cloud infrastructure and the customer is responsible for the security of the workloads, configurations, and data deployed on that infrastructure [3]. Within this division of responsibility, the configuration of identities and permissions is exclusively the responsibility of the customer; this is what makes IAM the principal control surface available to the defender, and it is the surface on which the framework operates. This division of responsibility is shown in Figure 1, which shows the layers of the cloud stack governed by AWS and those governed by the customer.

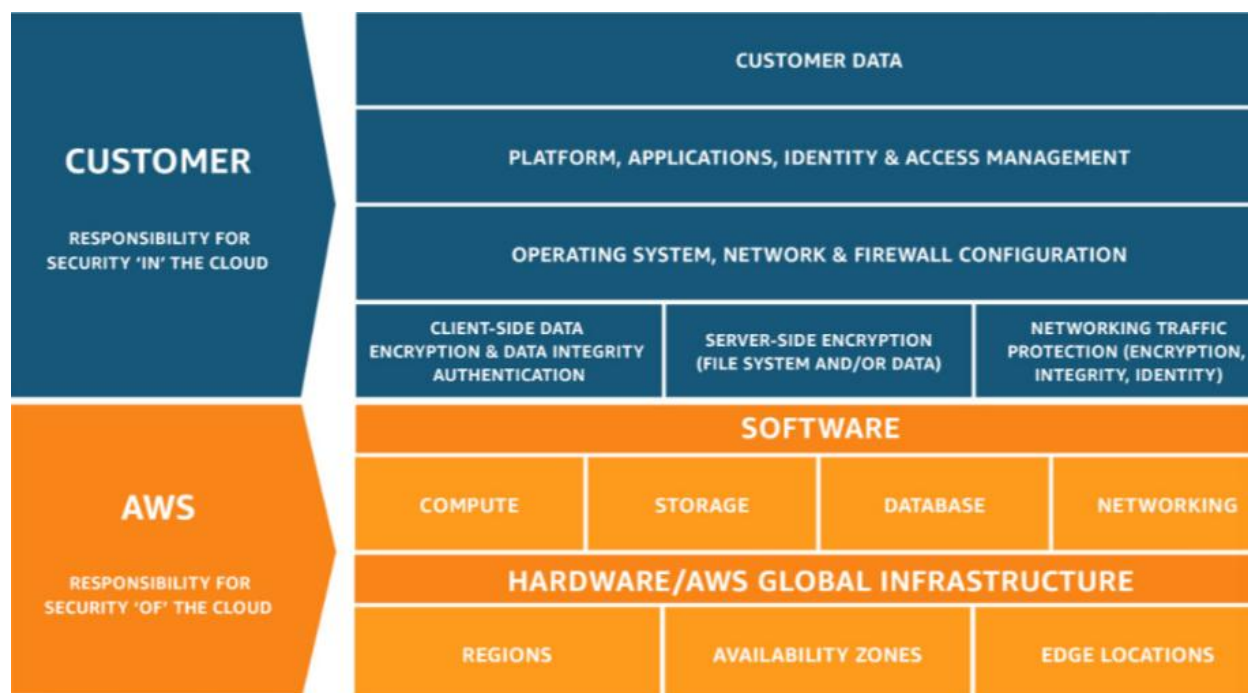


Figure 1: The AWS Shared Responsibility Model [3]

### 1-3-5 The AWS Account as Security Boundary

The AWS account is the fundamental security boundary of the AWS platform, every resource in AWS belongs to exactly one account, and every authorization decision is evaluated within the context of that account. Multiple accounts can be grouped under an AWS Organization, and that grouping is the scope at which Service Control Policies operate by defining the maximum permissions allowed across AWS accounts, restricting actions regardless of what identity-based policies grant [2].

### 1-3-6 AWS Services

AWS services are modular cloud-based components that provide specific functionalities, such as compute, storage, networking, and identity management, within the Amazon Web Services platform. Each service exposes a defined set of operations through APIs and operates within a shared infrastructure model governed by AWS security and authorization mechanisms [2].

- **AWS Identity and Access Management (IAM):** the service that governs authentication and authorization for every principal operating within an AWS account.
- **AWS Security Token Service (STS):** the service that issues short-lived credentials in response to AssumeRole requests, enabling cross-account access and the temporary delegation of permissions, STS is the central primitive of dynamic privilege transfer in AWS.
- **Amazon Simple Storage Service (S3):** the object-storage service of AWS, resources are organized into buckets, and bucket-level resource-based policies determine which principals may operate on the objects they contain.

- **Amazon Elastic Compute Cloud (EC2):** the virtual-machine service of AWS; instances may be launched with attached IAM roles, which makes EC2 a frequent participant in privilege-escalation paths through the ability to pass IAM roles to AWS services.
- **AWS Key Management Service (KMS):** the managed service for cryptographic-key creation and access, KMS key resource policies control which principals may use a key for cryptographic operations.
- **AWS Lambda:** the serverless compute service of AWS, Lambda functions execute under attached IAM roles and may be invoked through resource-based policies that grant the right to invoke to specific principals.
- **AWS CloudTrail:** the audit service that records every AWS API call as an immutable JSON event stored in an S3 bucket, CloudTrail is the source of dynamic evidence consumed by Phase Three of the framework.
- **AWS Organizations:** the service that groups AWS accounts into an organizational tree and enforces account-level governance through Service Control Policies (SCPs).
- **AWS Security Hub:** the aggregation service that consolidates security findings into the Amazon Security Finding Format (ASFF) and publishes the Foundational Security Best Practices baseline used for severity calibration in Phase One.

Together, these services form the foundational components of the AWS platform, within which identity, resource management, and service interactions are defined and controlled through a unified authorization model.

## 1-4 AWS Identity and Access Management (IAM)

### 1-4-1 The IAM Service

AWS Identity and Access Management (IAM) is the service that controls authentication and authorization for principals operating within an AWS account [2]. IAM exposes its policy and identity objects through the AWS Application Programming Interface (API), and every action performed against this API is recorded by the AWS CloudTrail service. IAM is the only authorization gate between any principal and the resources of the account, which is why IAM misconfiguration is the dominant root cause of AWS security incidents [4].

### 1-4-2 IAM Principal Types

A principal in IAM is an entity that can request an action on a resource. AWS recognizes six classes of principal that the framework models: the root user, IAM users, IAM groups, IAM roles, federated identities, and service principals (including service-linked roles) [2].

#### 1-4-2-1 The Root User

The root user is the identity created when an AWS account is first opened. It has unrestricted access to every resource in the account, cannot have its permissions reduced by any IAM policy, and cannot be deleted. AWS recommends that the root user be used only for the small number of account-management tasks that require it, and that all routine work be performed under IAM users or IAM roles [2]. The framework reports any unusual use of the root user under its Phase One posture rules and flags it as a high-severity finding.

#### **1-4-2-2 IAM Users**

An IAM user is a long-lived identity within an AWS account, intended for a specific human or application. Each IAM user has a unique name and can be associated with different types of credentials. It can also belong to one or more IAM groups and can have identity-based policies attached, in addition to inline policies embedded directly within the user object [2].

#### **1-4-2-3 IAM Groups**

An IAM group is a logical container of IAM users that simplifies permission administration, and it is not a principal, meaning it cannot make requests directly; however, any policies attached to it apply to all member users. IAM groups cannot be nested, and a user can belong to more than one group at the same time [2].

#### **1-4-2-4 IAM Roles**

An IAM role is an identity within an AWS account that does not have long-lived credentials of its own. Instead, it is assumed by a trusted principal through a call to the AWS Security Token Service. Each role has a trust policy that defines which principals are allowed to assume it, along with one or more identity-based policies that specify the permissions available during the assumed-role session. Roles are the standard mechanism for granting temporary credentials to compute instances, AWS services, and cross-account users [2].

#### **1-4-2-5 Federated Identities**

A federated identity is a principal authenticated by an external identity provider, such as an enterprise SAML provider, an OpenID Connect provider, or AWS IAM Identity Center, and mapped to an IAM role through a trust policy [2]. For graph analysis, the framework treats a federated session as a role assumption. The session inherits the permissions of the assumed role and is governed by all relevant policy types.

#### **1-4-2-6 Service Principals and Service-Linked Roles**

A service principal is an AWS service that may itself act as a principal in calls to other AWS services. For example, when an Amazon Elastic Compute Cloud (EC2) instance is launched with an attached IAM role, AWS provides temporary credentials from that role to applications running on the instance. A service-linked role is a special type of role created and managed by an AWS service, which uses it to perform actions on behalf of the customer. These roles cannot be modified directly by the customer [2]. The framework models both service principals and service-linked roles in the privilege graph, since the ability to pass IAM roles to AWS services can participate in privilege-escalation paths.

### **1-4-3 Authentication Credentials**

AWS recognizes two broad classes of credential. Long-lived credentials are bound to an IAM user and consist of a console password (used by humans) and access keys (used by programs), each access key is the pair of an Access Key Identifier and a Secret Access Key. Temporary session credentials are issued by the AWS Security Token Service (STS) after a successful AssumeRole request and consist of an Access Key Identifier, a Secret Access Key, and a Session Token, all of which expire after a configurable duration [2]. The framework treats the creation of any new

credential, whether long-lived through `iam:CreateAccessKey` or temporary through `sts:AssumeRole`, as a graph edge that transfers the resulting permissions to the target identity.

#### **1-4-4 Multi-Factor Authentication**

Multi-Factor Authentication (MFA) is the requirement that a principal present an additional factor, typically a time-based one-time password generated by a virtual or hardware device, in addition to its primary credential [2]. Several Phase One posture rules in the framework verify the configuration of MFA on the root user and on IAM users with console access, because the absence of MFA is a high-severity violation under the CIS AWS Foundations Benchmark [5].

### **1-5 IAM Policies**

#### **1-5-1 Policy Documents and JSON Structure**

An IAM policy is a JavaScript Object Notation (JSON) document that specifies a set of permitted or denied actions on a set of resources under a set of conditions. A policy document is a top-level JSON object that includes an optional Version element and a required Statement element, which may contain either a single statement or a list of statements. Each statement includes the relevant elements that define permissions and conditions [2].

#### **1-5-2 Managed Policies and Inline Policies**

AWS distinguishes managed policies from inline policies. A managed policy is a stand-alone, versioned policy that can be attached to many identities, managed policies are subdivided into AWS-managed (defined by AWS and read-only) and customer-managed (defined by the customer). An inline policy is a policy embedded directly within a single identity or resource and shares its lifecycle [2]. The framework parses both classes of policy and keeps them separate in the inventory because inline policies require different traversal rules during graph construction.

#### **1-5-3 Identity-Based, Resource-Based, and Trust Policies**

Three sub-classes of policy participate in IAM authorization. An identity-based policy is attached to an IAM user, group, or role and grants permissions to that identity. A resource-based policy is attached to a resource, such as an Amazon Simple Storage Service (S3) bucket, an AWS Key Management Service (KMS) key, or an AWS Lambda function, and specifies which principals may operate on it. A trust policy is a special form of resource-based policy linked to an IAM role, it specifies which principals are permitted to assume the role through the Security Token Service [2]. The framework parses all three sub-classes and treats them as supporting inputs to graph construction.

#### **1-5-4 Policy Elements**

Each statement within an IAM policy carries the following elements. The Effect element takes one of two values, Allow or Deny, and determines whether the statement contributes a positive or a negative authorization. The Action element enumerates the AWS API operations to which the statement applies, expressed as service-prefixed identifiers such as `iam:PassRole` or `s3:GetObject`. The Resource element lists the Amazon Resource Names (ARNs) of the resources to which the statement applies, with wildcards permitted. The Principal element appears only in resource-based

policies and identifies the principals to which the policy grants access. The Condition element is an optional Boolean expression evaluated at request time over a set of pre-defined and policy-defined keys, and it is the element that introduces ABAC semantics into IAM [2].

### **1-5-5 NotAction and NotResource**

The negative forms NotAction and NotResource are alternative elements that match all actions or resources that are not explicitly listed in the policy [2]. The framework converts these forms during policy parsing into the equivalent positive forms when feasible, because analyzing "all actions except listed ones" is more reliable when compared against the full AWS action list.

### **1-5-6 Amazon Resource Names (ARNs)**

An Amazon Resource Name (ARN) uniquely identifies an AWS resource. The general syntax is `arn:partition:service:region:account-id:resource-id`, where partition is `aws` for the standard regions, service is the service name (for example `s3` or `iam`), region is the region in which the resource resides (empty for global resources such as IAM), account-id is the twelve-digit AWS account identifier, and resource-id is a service-specific identifier [2]. The framework matches ARNs in policy statements against the ARNs of inventory resources during graph construction.

### **1-5-7 Action Wildcards**

Both Action and Resource elements support the star wildcard (\*), which matches any sequence of characters [2]. A statement that grants Action: "\*" over Resource: "\*" grants administrative access and is treated by the framework as a critical-severity finding, a statement that grants `iam:*` over a specific role is similarly treated as a high-severity violation because it permits the holder to attach arbitrary policies to that role.

### **1-5-8 Policy Variables**

Policy variables are placeholders such as `${aws:username}` and `${aws:user-id}` that are replaced at request time with the corresponding attribute of the requesting principal. Policy variables allow patterns like limiting a user to their own S3 folder [2], the framework evaluates policy variables symbolically during graph construction.

## **1-6 Policy Evaluation Logic**

### **1-6-1 The Six Policy Classes**

AWS IAM defines six classes of policy that participate in any single authorization decision, identity-based policies, resource-based policies, Service Control Policies (SCPs), Permission Boundaries, session policies, and access control lists (ACLs) [2]. The framework supports the first five classes, ACLs, supported by a small set of legacy services, are out of scope because the project targets modern IAM-based authorization only.

### **1-6-2 The Deterministic Evaluation Algorithm**

When a principal issues a request, AWS IAM evaluates all applicable policies according to a deterministic algorithm based on two main rules. First, an explicit Deny always overrides any Allow, this deny-overrides property is replicated by the framework when determining whether a



graph edge is valid. Second, if there is no Allow, access is denied by default, this is known as the closed-world assumption in IAM and forms the basis of least-privilege enforcement [2].

### **1-6-3 Explicit Deny, Explicit Allow, and Implicit Deny**

An explicit Deny is any statement whose Effect is Deny and whose Action / Resource / Principal / Condition all match the request, an explicit Allow is a statement with Effect set to Allow, an implicit Deny is the absence of any matching statement [2]. The framework records the source (policy class, policy name, statement identifier) of every Deny and Allow that affects the final decision, so that any rejected escalation path can be explained by reference to the specific statement that closed it.

### **1-6-4 AWS Security Token Service and Assume Role**

The AWS Security Token Service (STS) issues temporary credentials in response to a successful AssumeRole request. Each IAM role has a trust policy that defines which principals are allowed to assume it, an AssumeRole request succeeds only if the caller is permitted by both the trust policy and the caller's identity-based policy. The temporary credentials provided by STS inherit the permissions of the assumed role and expire after a configurable period. AssumeRole is the primary mechanism for cross-account access in AWS and is frequently involved in privilege escalation, since a successful role assumption grants the full set of permissions associated with the target role [2].

### **1-6-5 Service Control Policies (SCPs)**

A Service Control Policy (SCP) is an organization-level control that defines the maximum permissions available to accounts within an AWS Organization. SCPs act as filters and do not grant permissions, if a request is denied by any SCP in the hierarchy from the organization root to the target account, it is denied regardless of any Allow in identity-based or resource-based policies. The framework treats SCPs as filters applied to permissions during graph building [2].

### **1-6-6 Permission Boundaries**

A Permission Boundary is an attached managed policy that defines the maximum permissions an identity-based policy can grant to a principal. Permission Boundaries act as the identity-level equivalent of SCPs, they restrict the effective permissions of a single principal regardless of what its attached policies allow [2]. The framework models Permission Boundaries as a per-principal constraint applied to permissions when constructing each outgoing edge from that principal in the graph.

### **1-6-7 Session Policies**

A Session Policy is an inline policy provided during an AssumeRole call to further restrict the permissions of the resulting session beyond those granted by the role. Session policies are temporary and apply only to a specific request [2], the framework models them as per-edge filters that apply only to AssumeRole edges in the graph.

## 1-7 Access Control Models

### 1-7-1 DAC, MAC, and RBAC Families

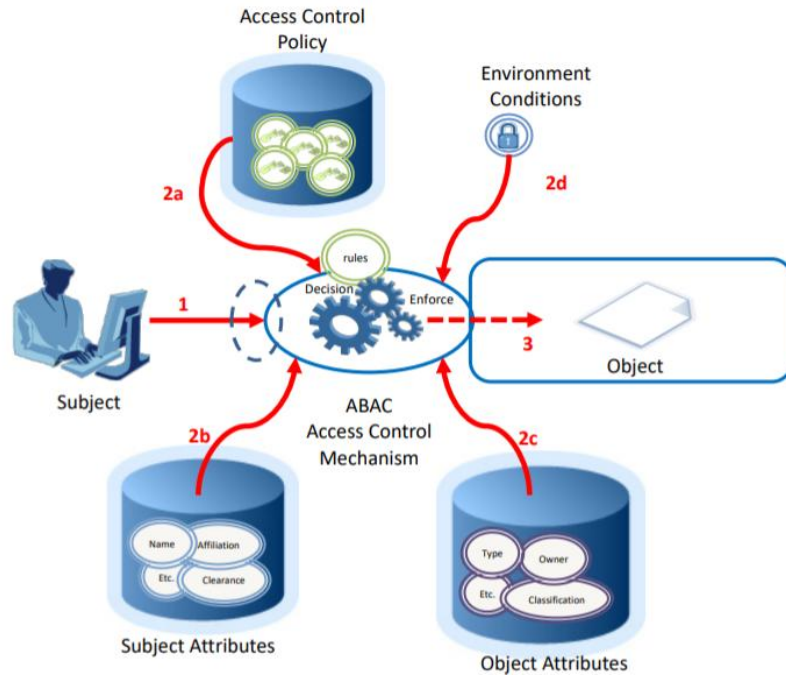
Access-control research recognizes three classical families of policy: Discretionary Access Control (DAC), in which the owner of an object decides who may access it, Mandatory Access Control (MAC), in which a system-wide policy based on labels prevents owners from granting access in a way that breaks the labeling rules, and Role-Based Access Control (RBAC), in which permissions are associated with roles rather than with individual users [7]. The framework is concerned with RBAC and ABAC, because these are the two access-control families realized in AWS IAM.

### 1-7-2 Role-Based Access Control

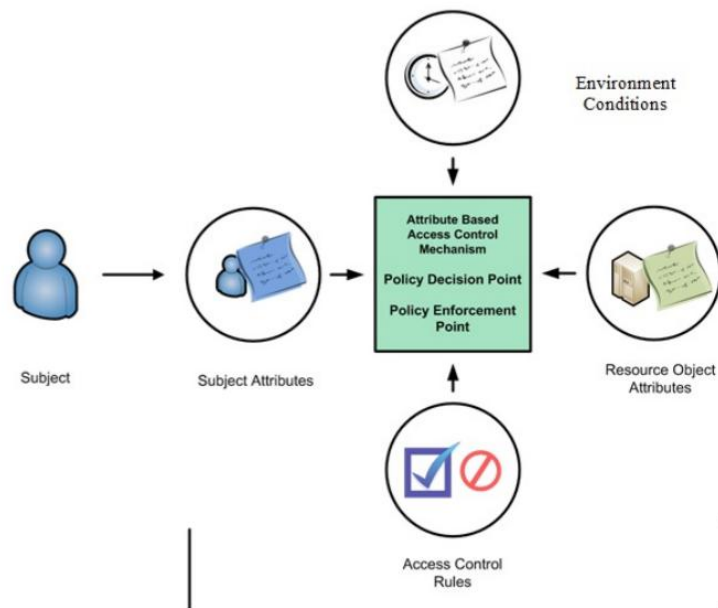
Role-Based Access Control (RBAC) is the model in which permissions are assigned to roles, and roles are assigned to users, so that authorization decisions are based on the role membership of the requester rather than on individual identity [7]. The four core components of the RBAC reference model are users (U), roles (R), permissions (P), and sessions (S), together with two assignment relations: user-assignment  $UA \subseteq U \times R$  and permission-assignment  $PA \subseteq P \times R$  [7]. Optional extensions include role hierarchies, Static Separation of Duty (SSD) constraints that forbid simultaneous membership in conflicting roles, and Dynamic Separation of Duty (DSD) constraints that prevent simultaneous activation of conflicting roles within a session [7]. AWS IAM realizes RBAC through the assume-role primitive of the Security Token Service and through the attachment of managed and inline policies to user, group, and role identities [2].

### 1-7-3 Attribute-Based Access Control

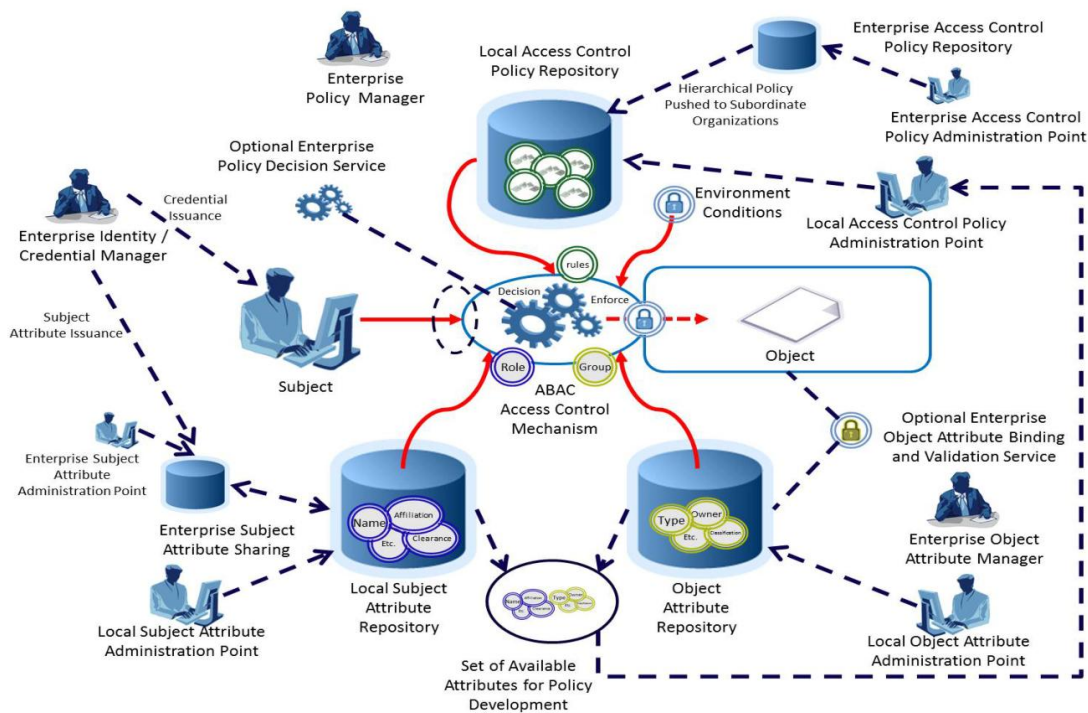
Attribute-Based Access Control (ABAC) is an access control model in which authorization decisions are determined by evaluating attributes associated with the subject, object, action, and environment against policies expressed as Boolean functions over those attributes. The basic ABAC scenario is illustrated in Figure 2: a subject requests access to an object, and the access-control system evaluates the policy rules against the subject attributes, the object attributes, and environmental conditions, the subject is granted access if and only if the policy evaluates to permit. ABAC generalizes Access Control Lists (ACLs) and RBAC as special cases based on identity and role attributes, respectively [8]. The functional architecture of an ABAC mechanism includes a Policy Decision Point (PDP), which evaluates policies, and a Policy Enforcement Point (PEP), which intercepts requests and enforces the decision returned by the PDP, as shown in Figure 3, an enterprise-scale deployment of these components is illustrated in Figure 4.



**Figure 2: Basic ABAC Scenario — NIST SP 800-162 [8]**



**Figure 3: Core ABAC Mechanisms — Policy Decision Point and Policy Enforcement Point — NIST SP 800-162 [8]**



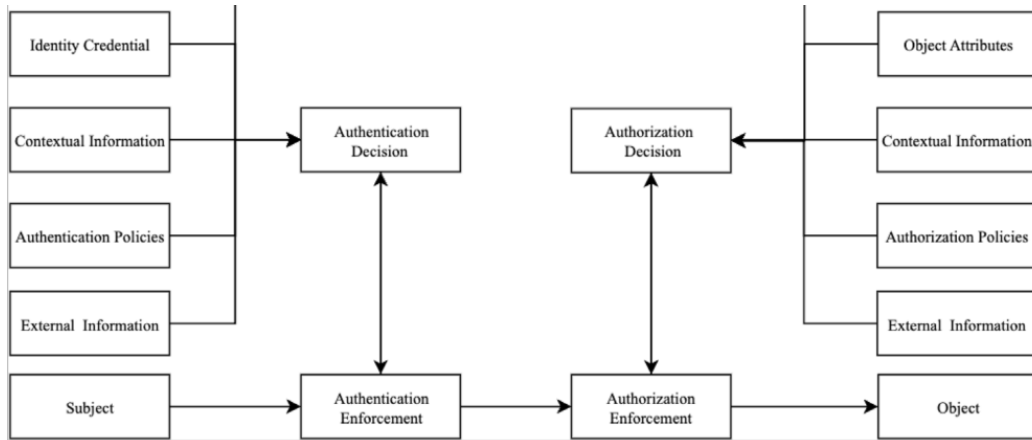
**Figure 4: Enterprise ABAC Scenario — NIST SP 800-162 [8]**

#### 1-7-4 Attribute Mutability

Because attributes can change over time, ABAC introduces the formal notion of attribute mutability, defined as the property of an attribute whose value may change as a side-effect of permitted actions. NIST SP 800-205 distinguishes between immutable and mutable attributes and notes that the correctness of static analysis of an attribute-based system depends on when the attribute is evaluated [9]. This is the property that the Phase Two graph engine treats explicitly, a tag-mutation edge changes the value of an attribute, and any condition that later uses that attribute must be re-evaluated against the new state.

#### 1-7-5 Implementation of RBAC and ABAC in AWS IAM

AWS IAM implements RBAC through the use of roles and policy attachments, and supports ABAC through Condition keys and Condition operators. The Condition keys most relevant to the framework are `aws:PrincipalTag/<key>`, `aws:ResourceTag/<key>`, `aws:RequestTag/<key>`, and `aws:TagKeys`, all of which reference tag attributes attached to identities or resources [10]. The logical components involved in an attribute-based authorization decision, including subject attributes, object attributes, contextual information, authentication policies, authorization policies, and the corresponding decision and enforcement points, are illustrated in Figure 5.



**Figure 5: Attribute-Based Access Management — Logical Components and Authentication / Authorization Decision [4]**

## 1-8 IAM Privilege Escalation

### 1-8-1 Definition

A privilege-escalation path in IAM is a sequence of permitted actions that allows a principal to obtain a set of permissions strictly greater than its initial set. Such paths arise either from misconfigured policies that grant transitive privilege-granting permissions or from logical compositions of individually benign permissions, in both cases, the resulting effective permissions available to the attacker exceed those granted by any individual policy in the inventory [11], [6].

### 1-8-2 Common Escalation Primitives

The escalation primitives most frequently encountered in published IAM vulnerability catalogues include `iam:PassRole` (which permits the caller to associate an IAM role with a service such as Amazon Elastic Compute Cloud, inheriting the role's permissions through the service), `sts:AssumeRole` (which permits the caller to obtain temporary credentials of a target role), `iam:CreateAccessKey` (which allows the caller to create long-term credentials for a target user), `iam:UpdateAssumeRolePolicy` (which permits the caller to modify a role's trust policy and add itself as a permitted assumer), `iam:AttachUserPolicy` and `iam:AttachRolePolicy` (which permit the caller to extend a target identity's permission set with managed policies), and `iam:PutUserPolicy` and `iam:PutRolePolicy` (the inline analogues of the previous two) [11], [6].

### 1-8-3 Graph-Based Reachability Analysis

The privilege-escalation discovery problem can be formulated as a reachability problem on a directed graph where nodes are principals, roles, and resources, and whose edges represent permitted privilege-granting actions. Hu et al. formalize this structure as a Permission Flow Graph (PFG), illustrated in Figure 6 for a well-known 2019 IAM privilege-escalation incident, where an attacker held permissions over a service principal that could assume a role, which in turn could assume an administrator-equivalent role, the figure further illustrates the transformation from a raw permission graph to an analyzed model, revealing hidden privilege-escalation paths that emerge from the composition of individually benign permissions [6].

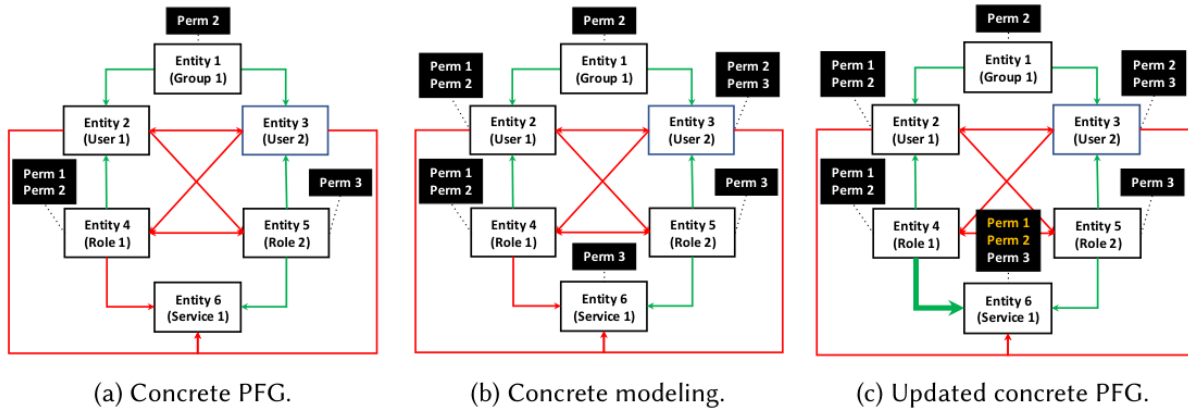


Figure 6: Permission Flow Graph (PFG) examples for the 2019 IAM privilege-escalation incident [6]

#### 1-8-4 Tag Mutation as State Transition

A class of privilege-escalation paths cannot be represented in a permission-only graph, because the satisfaction of an edge may depend on the value of a tag that a principal can modify prior to traversing that edge. NIST SP 800-205 formalizes this property as attribute mutability and emphasizes that any analysis of an attribute-based system must account for the time at which an attribute is evaluated [9]. Tag-mutation actions—including `iam:TagUser`, `iam:UntagUser`, `iam:TagRole`, `iam:UntagRole`, `ec2:CreateTags`, `ec2>DeleteTags`, `s3:PutBucketTagging`, and `s3>DeleteBucketTagging`—can be modeled as first-class graph edges representing state transitions in the attributes of principals or resources [12].

#### 1-8-5 Bounded Breadth-First Search over Joint State

Reachability over the augmented graph is computed using a bounded breadth-first search (BFS) starting from each principal, with two adjustments. First, the visited set is indexed not only by the node, but by the joint state (node, tag-state-hash), allowing the same node to be revisited under different attribute configurations. Second, the search depth is bounded by a configurable parameter  $k$ , in order to maintain tractability on large inventories while preserving coverage of previously documented escalation paths, which are typically short in practice and often require no more than a small number of steps (e.g.,  $\leq 3$ ) in publicly available datasets [11], [13].

### 1-9 Dynamic Detection through CloudTrail

#### 1-9-1 The CloudTrail Service

AWS CloudTrail is the service that records AWS API activity as immutable JSON events stored in an Amazon S3 bucket and optionally delivered to CloudWatch Logs and EventBridge. Each CloudTrail event contains information about the caller identity (`userIdentity`), the invoked action (`eventName`, `eventSource`), the affected resources (`resources`), the request context (`sourceIPAddress`, `userAgent`), and the associated request and response parameters. CloudTrail provides an authoritative record of API activity and authorization outcomes within an AWS account [14].

### **1-9-2 Management Events and Data Events**

CloudTrail distinguishes between two classes of event. Management events record control-plane operations such as CreateUser, AssumeRole, AttachRolePolicy, and PutBucketPolicy, and are recorded by default. Data events record data-plane operations such as s3:GetObject and lambda:Invoke, and require an explicit configuration. Privilege-escalation activity in AWS IAM occurs primarily in the control plane, making management events the relevant class for its analysis [14].

### **1-9-3 The CloudTrail Event Schema**

Each CloudTrail management event is a JSON document that includes the fields such as eventVersion, userIdentity (with subfields type, principalId, arn, accountId, and sessionContext), eventTime, eventSource, eventName, awsRegion, sourceIPAddress, userAgent, requestParameters, responseElements, requestID, eventID, and resources [14]. These fields provide a structured representation of API activity that can be systematically mapped to higher-level security models, such as threat taxonomies.

## **1-10 Adversary Tactics: MITRE ATT&CK**

### **1-10-1 The MITRE ATT&CK Framework**

MITRE ATT&CK is a publicly maintained knowledge base of adversary behavior observed in real-world intrusions, organized into matrices for distinct technology domains. It provides a standardized vocabulary for describing adversary tactics and techniques, enabling consistent classification, analysis, and communication of security events across the cybersecurity community [15].

### **1-10-2 Tactics, Techniques, Sub-Techniques, and Procedures**

ATT&CK organizes adversary behavior into a four-level taxonomy. A tactic represents a high-level adversary objective (e.g., Privilege Escalation, Credential Access, or Defense Evasion). A technique describes a method by which an adversary achieves a tactic (e.g., T1078 Valid Accounts). A sub-technique provides a more specific instance of a technique (e.g., T1078.004 Valid Accounts: Cloud Accounts). A procedure refers to a real-world implementation of a technique by a specific threat actor [15].

### **1-10-3 The ATT&CK for Cloud (IaaS) Matrix**

The ATT&CK for Cloud matrix and its IaaS sub-matrix enumerate techniques relevant to cloud platforms such as AWS, Azure, and GCP. Among these, several techniques are particularly relevant to cloud identity and access management scenarios, including T1078.004 Valid Accounts: Cloud Accounts, T1098.001 Account Manipulation: Additional Cloud Credentials, T1098.003 Account Manipulation: Additional Cloud Roles, T1136.003 Create Account: Cloud Account, T1530 Data from Cloud Storage, T1552.005 Unsecured Credentials: Cloud Instance Metadata API, and T1562.008 Impair Defenses: Disable or Modify Cloud Logs [15].

## **1-11 Posture Benchmarks**

### **1-11-1 The CIS AWS Foundations Benchmark**

The Center for Internet Security (CIS) AWS Foundations Benchmark is a community-developed configuration baseline that prescribes recommended security control settings for AWS accounts. Each control is mapped to a CIS Critical Security Control identifier, an applicable Profile Level (Level 1 for general production environments, Level 2 for environments requiring enhanced security), as well as corresponding audit and remediation procedures. The benchmark provides a standardized reference for evaluating the security posture of AWS environments and enables findings to be traced to an authoritative baseline [5].

### **1-11-2 AWS Security Hub Foundational Security Best Practices**

AWS Security Hub provides the Foundational Security Best Practices (FSBP) standard, a vendor-defined catalogue of security controls that complements the CIS AWS Foundations Benchmark and assigns a default severity level using the Amazon Security Finding Format (ASFF). The FSBP standard serves as a practical reference for continuous security monitoring and aligns findings with a consistent severity model [2].

### **1-11-3 NIST SP 800-53 Revision 5 and Control Structure**

NIST Special Publication 800-53 Revision 5 is the United States Federal catalogue of security and privacy controls for information systems and organizations. Each control in the catalogue follows a standardized structure consisting of a base control section, a discussion section, related-controls section, control-enhancements, and references [16]. The framework maps every rule to the most specific NIST 800-53 control family identifier (e.g., AC-6 Least Privilege) so that findings can be consumed by compliance pipelines that require NIST traceability.

## **1-12 Large Language Models and Retrieval-Augmented Generation**

### **1-12-1 Large Language Models**

A Large Language Model (LLM) is a transformer neural network trained on large textual corpora that produces natural-language output conditioned on an input prompt. LLMs can be specialized for particular domains or trained for general-purpose use, and they are commonly deployed through local or remote inference systems. Their ability to process structured and unstructured inputs makes them suitable for tasks such as reasoning, summarization, and decision support [18].

### **1-12-2 The Hallucination Problem**

Free-form generation by a Large Language Model (LLM) can produce statements that are not supported by any source, a phenomenon known as hallucination. In a security-sensitive contexts, hallucinated content is problematic because it can mislead the analysts, fabricate a non-existent identifier, or incorrectly attribute actions to entities that did not perform them [17]. Approaches to mitigating hallucination commonly include techniques such as retrieval-augmented generation and grounding outputs in verifiable sources.



### **1-12-3 Retrieval-Augmented Generation**

Retrieval-Augmented Generation (RAG) is an inference-time architecture in which a retriever first selects a set of passages from an external knowledge corpus that are most relevant to the input query, and a generator then conditions its output on both the retrieved passages in addition to the original input [18]. This architecture enables the incorporation of external, up-to-date, and domain-specific knowledge at generation time, improving the factual accuracy of the generated output and reducing reliance on the model’s internal parametric knowledge.

### **1-12-4 Dense Vector Embeddings and FAISS**

The retrieval step in retrieval-augmented systems typically relies on dense vector similarity, where textual passages are encoded as high-dimensional embeddings and indexed in an approximate nearest neighbor (ANN) structure such as the Facebook AI Similarity Search (FAISS) library. FAISS provides multiple indexing strategies, including flat, inverted-file, and product-quantized, each offering different trade-offs between retrieval accuracy and query latency. Such indexing structures enable efficient retrieval of the top- $k$  most relevant passages for a given query from large-scale knowledge corpora [19].

### **1-12-5 Citation Grounding and Hallucination Prevention**

Hallucination prevention in retrieval-augmented systems is commonly achieved by constraining the generator to ground its outputs in retrieved evidence and to associate factual claims with verifiable sources. This approach, often referred to as citation grounding, ensures that generated content can be traced back to supporting passages, thereby improving reliability and interpretability. By limiting generation to information present in the retrieved context, such mechanisms reduce the likelihood of unsupported or fabricated outputs [18], [17].

## **1-13 Statistical Foundations for Evaluation**

Statistical evaluation is used to summarize detection performance, measure uncertainty, and compare whether different tools detect or miss the same benchmark items. This section introduces the main statistical methods used in the evaluation: counts and recall, the Wilson score confidence interval for detection rates, McNemar’s exact test for comparing two tools on the same detected-or-missed benchmark items, and the Bonferroni adjustment for reducing false significance when multiple comparisons are performed. These methods are suitable for small benchmark datasets in which each test item is classified as either detected or not detected.

### **1-13-1 Counts, Proportions, and Recall**

In detection evaluation, results are often measured over a finite set of test cases. A count is the number of cases that fall into a specific category, such as the number of detected cases or the number of missed cases. A proportion is the count divided by the total number of evaluated cases, and it is usually expressed as a percentage.

Detection outcomes are commonly described using four terms. A true positive (TP) occurs when a real positive case is correctly detected. A false negative (FN) occurs when a real positive case is missed. A false positive (FP) occurs when a negative case is incorrectly reported as positive. A true negative (TN) occurs when a negative case is correctly ignored [15].

Recall measures how many real positive cases were detected. It is computed by dividing the number of true positives by the total number of real positive cases:  $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$ . A high recall means that the detector successfully identifies most real positive cases and misses fewer of them. In security evaluation, this is important because failing to detect a real attack can be more serious than producing an extra alert that can later be reviewed [15].

### **1-13-2 Wilson Score Confidence Interval**

A confidence interval is a range that shows how much uncertainty exists around a reported percentage or detection rate. It is calculated from the number of successful cases, the total number of cases, and the selected confidence level. For example, if a detector succeeds in 7 out of 7 test cases, the observed result is 100%, but this does not prove that the detector will always achieve 100% on larger datasets. The Wilson score confidence interval gives a more realistic range for this kind of result, especially when the number of test cases is small or when the percentage is close to 0% or 100%. Unlike simpler methods, it keeps the interval within the valid range from 0% to 100% and avoids giving misleading confidence ranges [20].

### **1-13-3 McNemar's Exact Test**

McNemar's exact test is used to compare two methods on the same set of test cases. Each test case has a binary result, such as detected or missed. The test focuses only on the cases where the two methods disagree: cases detected by the first method but missed by the second, and cases detected by the second method but missed by the first. If one method detects many more of these disagreement cases than the other, McNemar's exact test helps determine whether the difference is likely to be meaningful rather than due to chance. The exact version is useful for small datasets because it does not require a large number of test cases [21].

### **1-13-4 Bonferroni Adjustment**

The Bonferroni adjustment is used when several statistical tests are performed at the same time. When only one test is performed, the chance of a false significant result is limited by the chosen significance level. However, when many tests are performed, the chance that at least one of them appears significant by chance becomes higher. The Bonferroni adjustment reduces this risk by making each individual test harder to pass. It divides the original significance level by the number of tests. For example, if the significance level is 0.05 and three tests are performed, each test must meet a stricter threshold of  $0.05 / 3 = 0.0167$ . This adjustment is conservative, meaning that it may sometimes make it harder to find a significant difference. However, it is simple and useful when multiple comparisons are reported together [22].

## **1-14 Conclusion**

This chapter established the theoretical primitives underlying Cloud IAM Sentinel by introducing core security concepts including authentication, authorization, least privilege, and defense in depth, along with cloud computing as defined by NIST and the AWS shared responsibility model that defines the customer's authorization boundary. It further detailed the IAM service, covering its principal types, credential mechanisms, policy structure, and deterministic evaluation model, as well as identity and organization-level controls that limit effective permissions. Building on this, the chapter formalized RBAC/ABAC and defined attribute mutability as a core requirement

for modeling state changes based on tags. It then defined privilege escalation as a bounded reachability problem on a state-augmented permission graph, identified AWS CloudTrail as the authoritative source of dynamic evidence, and introduced the MITRE ATT&CK Cloud taxonomy, the CIS AWS Foundations Benchmark, AWS Security Hub FSBP, and NIST SP 800-53 as the primary frameworks for labeling and compliance mapping. Finally, it outlined the foundations of LLM and RAG architecture with a citation-grounding mechanism that ensures verifiable and non-hallucinated outputs.

## **Chapter Two: Previous Studies**

## 2-1 Introduction

This chapter surveys recent work on AWS and cloud Identity and Access Management security analysis published between 2024 and 2026. The relevant literature is organized by approach category, including foundational IAM surveys, IAM audit and posture frameworks, graph-based privilege-escalation analysis, Large Language Model–based threat detection, and Cloud Security Posture Management. The chapter concludes with two comparative tables. positions research approaches along a set of key design dimensions, while the second compares widely used open-source IAM analysis tools based on the consolidated evaluation of Nguyen et al. [31], and the official documentation of each tool. Industry context is provided by reports from the Cloud Security Alliance [32], Microsoft [33], and the Center for Internet Security [5], all of which identify IAM misconfiguration as a dominant root cause of cloud security incidents.

## 2-2 Foundational IAM Surveys

Recent survey literature provides a high-level perspective on the principles and design considerations underlying cloud Identity and Access Management systems. Rather than proposing concrete analysis techniques, these works synthesize existing practices and identify recurring challenges in securing IAM configurations.

Kaleru [25] presents a 2025 survey of IAM foundations, principles, and best practices, organizing the field into six pillars: strong authentication, access controls, privilege management, lifecycle processes, intelligent monitoring, and adaptive risk management. The survey is broad and prescriptive, providing practitioner-oriented motivation for the present work.

Jain [26] presents a 2025 survey of IAM in the cloud, focused on its components, authentication mechanisms, and authorization processes. The survey reviews single sign-on, multi-factor authentication, and federated identity management as the standard building blocks of modern cloud IAM, and it highlights the central role of correct authorization in overall cloud security. These surveys are included because they define the main IAM principles and challenges that the proposed framework later addresses through static analysis, graph modeling, and CloudTrail correlation.

## 2-3 IAM Audit and Posture Frameworks

Bello et al. [27] propose in 2025 a Scalable IAM Audit Framework for Distributed Cloud Security, focusing on the horizontal scalability of IAM auditing across multi-cloud infrastructures. The framework emphasizes event aggregation and cross-account correlation. It operates at the audit-log level rather than at the policy-evaluation level. Its scope is audit-log aggregation and cross-account correlation rather than identity-graph reachability or attribute-mutation analysis.

Bello et al. [28] complement the previous work in the same year with a Zero Trust IAM Framework for Unified Auditing in Hybrid Distributed Systems. This second framework integrates the same auditing pipeline with a zero-trust enforcement layer, extending the applicability of the audit pipeline to hybrid deployments. As in the previous work, the analysis is performed at the audit-log level. Therefore, it is closer to a zero-trust auditing architecture than to a privilege-escalation graph analysis framework.

## 2-4 Graph-Based IAM Privilege-Escalation Analysis

Hu, Khurshid, and Tiwari [6] present TAC, a 2025 greybox penetration-testing approach for IAM privilege escalation. TAC formalizes the IAM authorization surface as a Permission Flow Graph and combines a query-based abstraction with reinforcement learning and graph-neural-network optimization to reduce customer disclosure during third-party scanning. The Permission Flow Graph formalism defined by TAC is conceptually similar to privilege-graph approaches in the literature, with two key distinctions. First, TAC operates in a greybox setting in which the scanning service does not have full visibility into the IAM configuration, whereas whitebox approaches assume access to a complete inventory. Second, TAC does not model tag-mutation actions as state-transition edges, limiting its ability to capture attribute-dependent escalation paths.

Nguyen, Ho, and To [31] present SkyEye, a 2025 open-source framework for AWS IAM privilege-escalation discovery. SkyEye addresses the boundary-scope problem, namely escalation paths that require simultaneous reasoning across identity-based policies, Service Control Policies, permission boundaries, and trust relationships. It introduces a deep comparison model for policy documents to detect subtle differences between policy versions and maps identified escalation paths to MITRE ATT&CK techniques. SkyEye represents one of the closest approaches to comprehensive privilege-graph analysis. However, it does not treat tag-mutation actions as first-class edges, and its detection layer relies on rule-based mapping rather than grounded generation.

Madireddy [29] presents in 2025 a Graph Neural Network based Adaptive Threat Detection framework for cloud IAM logs. The framework models IAM audit trails as heterogeneous dynamic graphs with attention-based aggregation and continuously updated embeddings, enabling the detection of latent user–resource interaction patterns. This line of work applies graph reasoning to observed activity rather than to authorization structure, focusing on behavioral detection over logged events.

## 2-5 Large-Language-Model-Based IAM Threat Detection

Adediran, Awuson-David, and Ahmed [17] propose in 2026 a two-step Retrieval-Augmented Generation pipeline using Gemini 2.5 Pro for AWS CloudTrail threat detection and MITRE ATT&CK mapping. Their evaluation on a dataset of two hundred CloudTrail events reports an accuracy of 78% and an F1 score of 79%, representing a substantial improvement over the same model without retrieval augmentation. The approach demonstrates the effectiveness of combining retrieval mechanisms with language models to improve the reliability of security analysis.

This line of work highlights the potential of LLM-based detection for cloud security but remains dependent on probabilistic generation and does not incorporate deterministic grounding constraints or state-aware reasoning over IAM authorization structures.

## 2-6 CSPM Quality and False-Positive Reduction

Dikshant and Verma [30] present in 2025 a validation-driven methodology for reducing false positives in Cloud Security Posture Management (CSPM) systems. Their approach integrates active behavioral testing through lightweight automated probes that simulate adversarial actions

to assess the exploitability of policy violations in real time. This method aims to distinguish between configurations that are theoretically insecure and those that are practically exploitable.

This line of work highlights the importance of validating security findings against runtime behavior. However, it relies on active interaction with cloud systems and real-time execution of probes, introducing operational overhead and potential side effects. In contrast, static analysis approaches avoid such interaction by reasoning over configuration state alone, trading runtime validation for scalability and safety.

## 2-7 Comparative Analysis of Research Papers

Table 2-1 compares the seven practical research papers reviewed in Sections 2-3 through 2-6 across a set of six design dimensions. These dimensions include the approach category adopted by each system, whether IAM Condition operators are evaluated during policy analysis, whether attribute mutability is modeled through tag-mutation actions as state transitions, whether dynamic CloudTrail evidence is incorporated, whether Large Language Models with citation grounding are used for alert generation, and whether alerts are mapped to MITRE ATT&CK technique identifiers.

**Table 2-1: Comparison of Research Papers Against the Proposed Framework**

| System / Reference             | Approach                         | IAM Conditions | Tag Mutation | CloudTrail | Grounded LLM | MITRE ATT&CK |
|--------------------------------|----------------------------------|----------------|--------------|------------|--------------|--------------|
| Bello et al. [27]              | Audit-log analysis               | No             | No           | Indirect   | No           | No           |
| Bello et al. [28]              | Audit + zero-trust enforcement   | No             | No           | Indirect   | No           | No           |
| Hu et al. (TAC) [23]           | Symbolic graph + GNN             | Partial        | No           | No         | No           | No           |
| Nguyen et al. (SkyEye) [31]    | Symbolic graph + policy diff     | Yes            | No           | Partial    | No           | Yes          |
| Madireddy (GNN-IAM) [29]       | Graph Neural Network             | No             | No           | Yes        | No           | No           |
| Adediran et al. (RAG-AWS) [17] | RAG-LLM                          | No             | No           | Yes        | Yes          | Yes          |
| Dikshant and Verma [30]        | Active validation                | Indirect       | No           | No         | No           | No           |
| Cloud IAM Sentinel             | Hybrid (rules + graph + RAG-LLM) | Yes            | Yes          | Yes        | Yes          | Yes          |

The table highlights a residual gap in the current literature. No surveyed system simultaneously evaluates IAM Condition operators (as in symbolic graph approaches such as TAC and SkyEye), models tag-mutation actions as state transitions (absent in all surveyed systems), incorporates

dynamic CloudTrail evidence (as in audit-log and graph neural network approaches), and produces alerts in which every claim is grounded in authoritative sources (as in retrieval-augmented approaches such as Adediran et al.). The proposed approach integrates all four properties within a unified and deterministic analysis pipeline.

## 2-8 Comparative Analysis of Practical Tools

The literature surveyed in the previous sections is dominated by academic prototypes. This section shifts focus to open-source tools currently used in practice for AWS IAM analysis. The comparison draws on the consolidated evaluation presented by Nguyen et al. [31] as well as the official documentation of each tool. The tools considered include Principal Mapper (PMapper), PACU, CloudFox, ScoutSuite, Cloudsplaining, Prowler, and SkyEye, together with the proposed approach.

Table 2-2 summarizes the capabilities of each tool across seven design dimensions: static rule-based analysis of IAM configurations, construction of privilege graphs and reachability analysis, evaluation of IAM Condition operators, modeling of tag-mutation actions as state transitions, correlation with dynamic CloudTrail events, use of Large Language Models with citation grounding, and mapping of alerts to MITRE ATT&CK technique identifiers.

**Table 2-2: Comparison of Practical IAM Analysis Tools Against the Proposed Framework**

| Tool                       | Static Rules | Graph Reachability | IAM Conditions | Tag Mutation | CloudTrail Correlation | Grounded LLM | MITRE ATT&CK |
|----------------------------|--------------|--------------------|----------------|--------------|------------------------|--------------|--------------|
| PMapper (Principal Mapper) | No           | Yes                | No             | No           | No                     | No           | No           |
| PACU                       | No           | No                 | No             | No           | Indirect               | No           | No           |
| CloudFox                   | Partial      | No                 | No             | No           | No                     | No           | No           |
| ScoutSuite                 | Yes          | No                 | No             | No           | No                     | No           | No           |
| Cloudsplaining             | Yes          | No                 | No             | No           | No                     | No           | No           |
| Prowler                    | Yes          | No                 | No             | No           | No                     | No           | No           |
| SkyEye                     | Yes          | Yes                | Yes            | No           | Partial                | No           | Yes          |
| Cloud IAM Sentinel         | Yes          | Yes                | Yes            | Yes          | Yes                    | Yes          | Yes          |

The table demonstrates that no existing open-source tool simultaneously supports static posture analysis, graph-based reachability with IAM Condition evaluation, modeling of attribute mutability, dynamic CloudTrail correlation, and citation-grounded Large-Language-Model reasoning within a single pipeline. The proposed approach occupies this position in the design space, providing an integrated capability that is absent from all surveyed tools.



## 2-9 Discussion and Justification of the Proposed Framework

Three observations from the 2024-2026 literature reviewed in this chapter justify the design of Cloud IAM Sentinel. First, IAM misconfiguration and the resulting privilege escalation remain a dominant root cause of cloud-security incidents, as documented by the Cloud Security Alliance [32], by Microsoft [33], and by Bello et al. [27]. Second, prior work has converged on graph reachability as the primary formalism for static privilege-escalation analysis (Hu et al. [6]; Nguyen et al. [31]) and on graph-based representations for dynamic IAM-log analysis (Madireddy [29]), however, no surveyed system models attribute mutability, that is, the property by which modifications to tag attributes can alter subsequent authorization outcomes. Third, retrieval-augmented architectures have demonstrated improved accuracy in Large Language Model-based threat detection (Adediran et al. [17]), but existing approaches do not enforce deterministic grounding constraints on generated outputs.

The proposed approach synthesizes these observations into a unified design. The framework constructs a deterministic privilege graph that evaluates IAM Condition operators, augments the graph with tag-mutation edges representing attribute-dependent state transitions, and incorporates CloudTrail-based correlation with alert generation supported by citation-grounded retrieval mechanisms. No system surveyed in the 2024–2026 literature combines all of these properties within a single pipeline, and this integration constitutes the primary contribution of the work.

## 2-10 Conclusion

This chapter surveyed recent literature (2024–2026) on AWS and cloud IAM security analysis across five relevant approach categories, presented two comparative tables evaluating existing systems and tools, and identified a residual gap in the current state of the art. Specifically, no surveyed approach simultaneously combines IAM Condition evaluation, attribute-mutability modeling, dynamic CloudTrail correlation, and citation-grounded Large Language Model reasoning. The next chapter presents the architecture and design of the proposed framework.

## **Chapter Three: Proposed Solution**

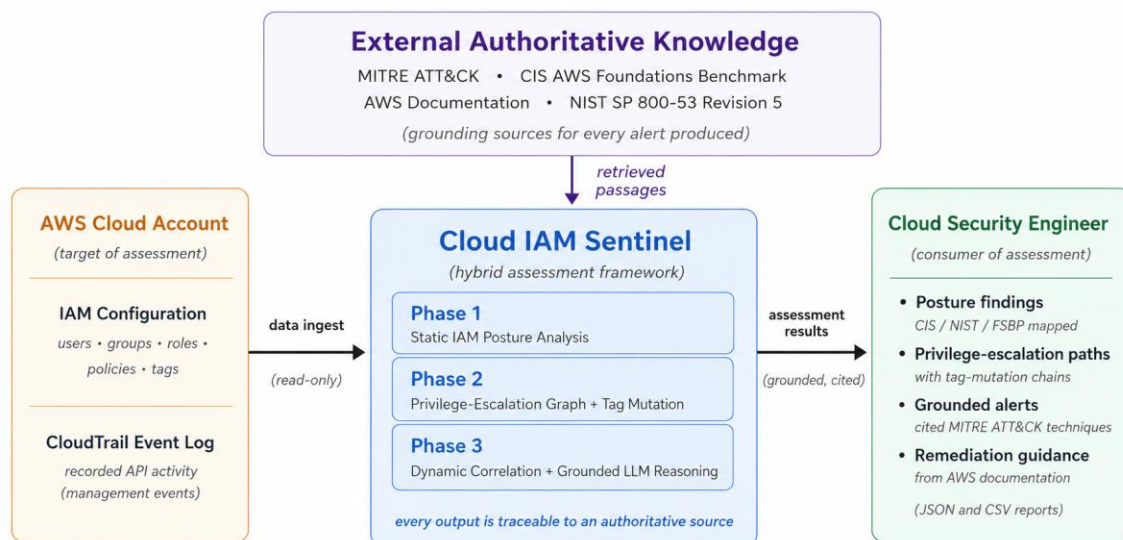
## 3-1 Introduction

This chapter presents the proposed solution, Cloud IAM Sentinel, and describes its overall architecture, the modules that comprise it, and the way these modules interact. The chapter begins with the high-level architecture diagram that summarizes the data flow from the external AWS evaluation tenant through to the final correlated alerts, then proceeds to describe each layer in turn: the data collection layer, which ingests IAM configuration and CloudTrail events into a unified inventory, the static analysis phases, which evaluate posture rules and construct a privilege-escalation graph, and the dynamic correlation phase, which maps observed activity to MITRE ATT&CK techniques and produces grounded alerts through a Retrieval-Augmented Generation pipeline. Each phase is described in terms of its inputs, processing logic, and outputs, with reference to the relevant theoretical foundations and design considerations.

## 3-2 Framework Architecture Overview

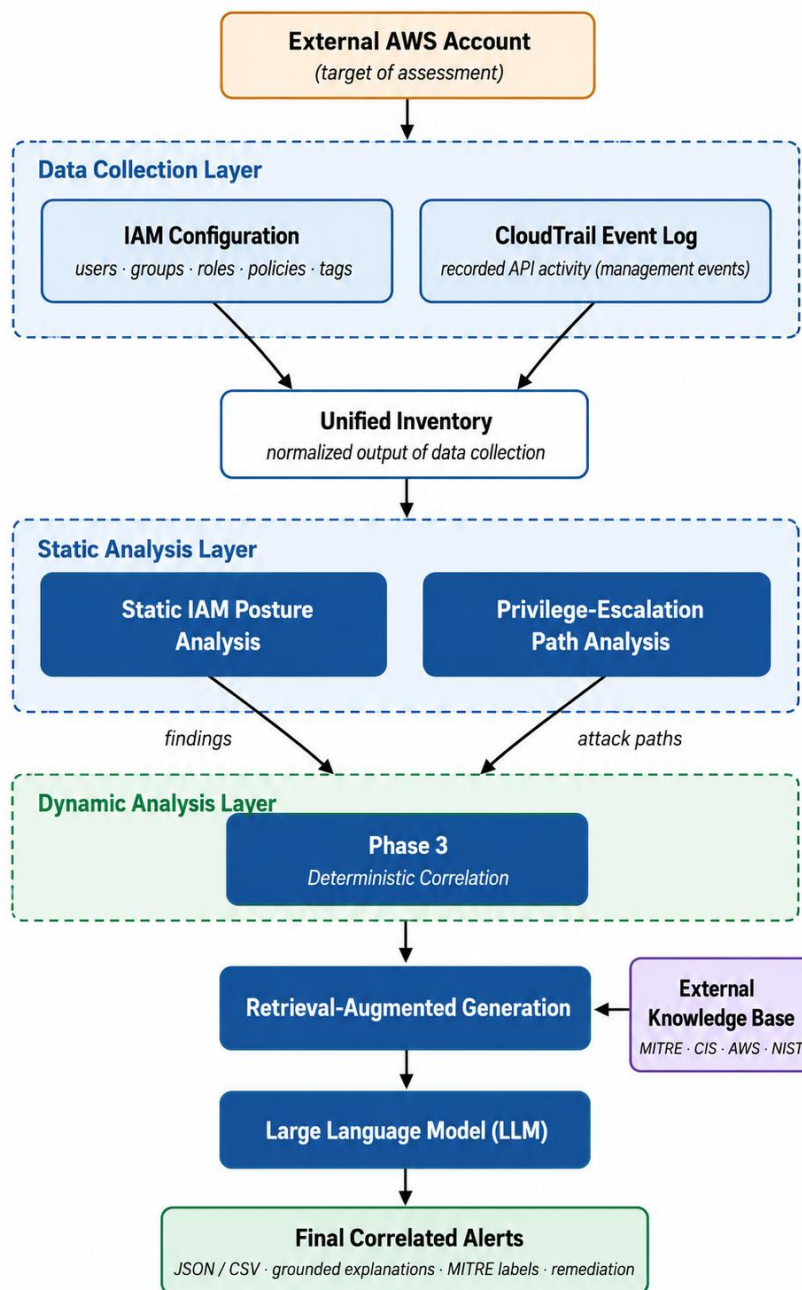
The proposed framework occupies an intermediary position within the AWS IAM security assessment workflow, operating between the AWS account under analysis, and the security engineer who consumes the assessment results. Figure 7 shows this position. The framework ingests IAM configuration data, CloudTrail event logs, and the Amazon Elastic Compute Cloud instance inventory from the cloud account, and draws on external authoritative sources including MITRE ATT&CK, CIS AWS Foundations Benchmark, AWS official documentation, and NIST SP 800-53 Revision 5, to ground its analysis. It produces posture findings, privilege-escalation paths, citation-grounded alerts, and remediation guidance, which are delivered to the security engineer in JavaScript Object Notation and Comma-Separated Value formats.

The framework performs no write operations on the cloud account. Its read-only data ingestion model ensures safe deployment as an assessment tool, without introducing any risk of unintended modifications to the target environment.



**Figure 7: Cloud IAM Sentinel as an Assessment Intermediary in the AWS IAM Security-Assessment Workflow**

The proposed framework is organized into three sequential analysis phases that operate over a shared data foundation. Figure 8 shows the end-to-end data flow from the external AWS account under analysis to the final correlated alerts.



**Figure 8: Cloud IAM Sentinel — Framework Architecture**

The data collection layer ingests two complementary sources of evidence from the external AWS account: the IAM configuration, which consists of users, groups, roles, policies, trust relationships, and tags, and the CloudTrail event log, which provides an immutable record of Application

Programming Interface activity recorded by the AWS platform [14]. These inputs are normalized into a unified inventory that is consumed by all subsequent phases.

The static portion of the pipeline consists of Phase One, which evaluates posture rules over the IAM configuration to produce findings, and Phase Two, which constructs a privilege-escalation graph over the same inventory to identify reachable attack paths. The dynamic portion comprises Phase Three, which correlates the static outputs with the CloudTrail evidence and processes the result through a Retrieval-Augmented Generation pipeline supported by an external knowledge base of MITRE ATT&CK Cloud techniques [15], AWS service authorization references [2], and CIS AWS Foundations Benchmark control descriptions [5].

A Large Language Model generates grounded reasoning for each alert, and the output of the model is filtered by a deterministic citation guard before being emitted as a set of correlated alerts in JavaScript Object Notation and Comma-Separated Value formats. Each alert includes a grounded explanation, recommended remediation, and corresponding MITRE ATT&CK technique labels.

### **3-3 Data Collection Layer**

The data collection layer is responsible for extracting and structuring data from the external AWS account into inputs required by the analysis phases. It collects two complementary classes of data: the static IAM configuration and the dynamic CloudTrail event stream.

#### **3-3-1 IAM Configuration Collection**

The IAM configuration collector queries the AWS IAM service using read-only Application Programming Interface operations defined in the project permission policy [2]. The collector retrieves the complete set of users, groups, roles, managed policies, inline policies, group memberships, role-attached policies, trust policies, attached tags, password policies, and Multi-Factor Authentication device assignments. The collected data are normalized into a JavaScript Object Notation inventory. All analysis phases operate on this pre-collected and normalized snapshot of the account state, without requiring any direct or live interaction with the AWS environment during execution. The collector additionally retrieves the Amazon Elastic Compute Cloud instance inventory, which is required by Phase Two to model PassRole-via-EC2 escalation paths.

#### **3-3-2 CloudTrail Event Collection**

The CloudTrail collector retrieves management events using the `cloudtrail:LookupEvents` Application Programming Interface operation for a configurable time window [14]. Each retrieved event is preserved in its native JavaScript Object Notation format to ensure that no fields are altered prior to deterministic processing in the dynamic analysis phase. The collector can alternatively process CloudTrail data from a pre-existing JavaScript Object Notation file, allowing analysis to be performed on previously collected event records.

#### **3-3-3 Unified Inventory**

The unified inventory is a data structure that integrates the IAM configuration, the Amazon Elastic Compute Cloud inventory, and the CloudTrail events into a single in-memory representation. The unified inventory is the only data exchange interface between the data collection layer and the

three analysis phases. The inventory is keyed by Amazon Resource Name where applicable, enabling efficient cross-references between policy statements, identities, and recorded events without requiring multiple parsing passes.

### **3-4 Static IAM Analysis**

Phase One evaluates a catalog of static posture rules over the unified inventory and produces a set of findings. It represents the static counterpart of a Cloud Security Posture Management approach, with two distinguishing properties: each rule is grounded in an authoritative compliance baseline, and each finding explicitly identifies the policy statement, principal, or resource that triggered it.

#### **3-4-1 Rule Catalog Architecture**

The rule catalog is partitioned into two sub-catalogs: a Role-Based Access Control sub-catalog that contains thirty-nine rules and an Attribute-Based Access Control sub-catalog that contains six rules. Each rule is implemented as a Python class that exposes a single evaluate method which takes the unified inventory as input and returns either a Finding object or no result when the rule does not apply. Each rule is accompanied by a metadata file with five required fields: the severity assigned to the finding, the basis for that severity, the authoritative source from which the rule is derived, the identifier of the corresponding CIS Critical Security Control [5], and the identifier of the corresponding NIST SP 800-53 Revision 5 control [16].

#### **3-4-2 RBAC Posture Rules**

The Role-Based Access Control sub-catalog addresses categories of misconfiguration that have historically contributed to major cloud-security incidents. Representative categories include the use of long-lived access keys, the absence of Multi-Factor Authentication for the root user and on console-enabled IAM users, overly permissive administrator-level managed policies, weak password policies, unused credentials, and direct policy attachments to users in violation of the recommended group-based assignment model. Each rule in this sub-catalog cites its authoritative source and corresponding CIS and NIST control identifiers, so that every finding is auditable.

#### **3-4-3 ABAC Posture Rules**

The Attribute-Based Access Control sub-catalog addresses misconfiguration patterns that arise specifically from attribute-based authorization. Representative categories include the use of fail-open Condition operators such as StringEqualsIfExists and Null without a corresponding fail-closed constraints, the absence of separation-of-duty controls when multiple principals share the same tag, and policies that grant the iam:TagUser action over a principal whose attributes are referenced by a downstream aws:PrincipalTag condition [10].

#### **3-4-4 Output Schema**

Each finding produced by Phase One is serialized into a JavaScript Object Notation document and a corresponding Comma-Separated Value record. The JavaScript Object Notation record includes the rule identifier, severity level, authoritative source, control identifier, principal or resource identifier, the policy statement that triggered the finding, and a human-readable description. A schema validator enforces the presence of all required fields before a finding is emitted, ensuring that no malformed output is propagated to subsequent phases.

## 3-5 Privilege-Escalation Path Analysis

Phase Two constructs a directed privilege-escalation graph over the unified inventory and identifies the set of reachable paths from each principal to privileged target. The phase is the principal innovation of the framework, since it explicitly evaluates IAM Condition operators and models tag-mutation actions as state transitions.

### 3-5-1 Graph Construction

The graph builder iterates over the unified inventory and emits one node per principal (user, group, role, federated identity, service principal) and one node per resource that participates in a privilege-granting relationship (Amazon Elastic Compute Cloud instance role, Amazon Simple Storage Service bucket, AWS Key Management Service key, AWS Lambda function). Edges are generated in two passes. The first pass enumerates the privilege-granting actions present in the inventory, including `iam:PassRole`, `sts:AssumeRole`, `iam:CreateAccessKey`, `iam:UpdateAssumeRolePolicy`, `iam:AttachRolePolicy`, `iam:PutRolePolicy`, and tag-mutation actions. The second pass evaluates each candidate edge according to AWS IAM policy semantics, including application of the deny-overrides evaluation model and resolution of all applicable Condition operators [2].

### 3-5-2 IAM Condition Evaluation

The Condition evaluator implements the operators most frequently encountered in production policies: `StringEquals`, `StringNotEquals`, `StringEqualsIfExists`, `StringLike`, `Bool`, `ArnLike`, `ArnEquals`, `Null`, `ForAllValues:StringEquals`, and `ForAnyValue:StringEquals` [10]. For each Condition, the evaluator returns one of three verdicts: `true`, `false`, or `unknown`. An `unknown` verdict results in the candidate edge being marked as conditional rather than admitted unconditionally, preserving soundness in the presence of operators that are not yet implemented.

### 3-5-3 Tag-Mutation Edges and Joint-State Reachability

Tag-mutation actions are modeled as a distinct edge class. When the graph engine encounters a principal that has permissions such as `iam:TagUser`, `iam:UntagUser`, `iam:TagRole`, `iam:UntagRole`, `ec2:CreateTags`, or `s3:PutBucketTagging` permission, it emits an edge that mutates the tag state of the principal or the target resource. The reachability search is performed over the joint space of node identity and tag-state hash, allowing the same node to be visited multiple times under different attribute configurations and enabling representation of escalation paths that requires modifying a tag before traversing a downstream edge. This design directly realizes the formal notion of attribute mutability defined by NIST Special Publication 800-205.

### 3-5-4 Bounded Breadth-First Search

Reachability is computed by bounded breadth-first search from each principal to each privileged target. The search depth is limited by a configurable parameter, with a default value of three, ensuring tractability on production-scale inventories while preserving coverage of known escalation paths documented in the IAM Vulnerable benchmark [11] and the Pathfinding.cloud catalog [13]. Each discovered path is annotated with the sequence of edges traversed, the authoritative-source associated with each edge type, and the Condition verdicts encountered along the path.

### **3-5-5 Service Control Policy and Permission Boundary Filtering**

After a candidate path is enumerated, it is post-filtered using Service Control Policy and Permission Boundary constraints. The Service Control Policy filter enforces organization-level guardrails, while the Permission Boundary filter applies the maximum permission set defined for each principal. If a path is denied by either filter, it is annotated as blocked by a guardrail or permission boundary rather than reported as a successful escalation. This ensures that the analysis remains consistent with AWS IAM evaluation semantics.

### **3-5-6 Output Schema**

Each discovered path is serialized into a JavaScript Object Notation document and a corresponding Comma-Separated Value record. The JavaScript Object Notation record includes the source principal, the target privilege, the sequence of traversed edge types, Service Control Policy and Permission Boundary annotations, and the citation linking each action in the path to the AWS IAM action reference [2].

## **3-6 Phase Three — Deterministic Correlation and Grounded LLM Reasoning**

Phase Three correlates the outputs of Phase One and Phase Two with the dynamic CloudTrail evidence to produce the final alerts delivered to the responder. The phase combines a deterministic mapping layer with a Retrieval-Augmented Generation layer, the latter constrained by a citation-grounding guard that prevents hallucination content.

### **3-6-1 CloudTrail Event Ingestion**

The CloudTrail ingestion module processes events collected in the data collection layer, normalizing the `userIdentity` field across the `AssumedRole`, `IAMUser`, `AWSService`, and `Root` identity types. Events are grouped into time-windowed sessions per principal, which serve as the unit of correlation in subsequent processing.

### **3-6-2 Deterministic MITRE ATT&CK Mapping**

The MITRE mapper applies a deterministic lookup table that maps each CloudTrail `eventName` to a MITRE ATT&CK technique identifier [15]. The mapping is curated from the MITRE ATT&CK Cloud (IaaS) matrix and includes techniques such as Valid Accounts: Cloud Accounts (T1078.004), Account Manipulation: Additional Cloud Credentials (T1098.001), Account Manipulation: Additional Cloud Roles (T1098.003), Create Account: Cloud Account (T1136.003), and Impair Defenses: Disable or Modify Cloud Logs (T1562.008). The deterministic nature of the mapping ensures that each alert is associated with a verifiable technique identifier.

### **3-6-3 Correlation with Static Outputs**

For each session, the correlator matches the session's principal identifier against records produced in Phase One and Phase Two. A correlated alert is generated when a session contains an event whose mapped technique corresponds to a privilege-granting action identified either in the Phase One rule catalog or in a Phase Two attack path. This correlation step transforms an isolated CloudTrail event into an observation grounded in a potential privilege-escalation path.



### **3-6-4 Retrieval-Augmented Generation Layer**

For each correlated alert, the Retrieval-Augmented Generation layer constructs a query composed of the CloudTrail evidence, the corresponding Phase One finding or Phase Two path, and the associated MITRE ATT&CK technique identifier. The query is processed by a dense retriever that searches the external knowledge base described in the next subsection and returns the top-k most relevant passages. The passages are combined with the evidence and technique label and passed to the Large Language Model adapter to generate a grounded explanation.

### **3-6-5 External Knowledge Base**

The external knowledge base is a curated collection of authoritative documents that includes the MITRE ATT&CK technique pages [15], AWS service authorization references [2], the AWS IAM Action Reference [2], the CIS AWS Foundations Benchmark control descriptions [5], and AWS official remediation playbooks. The knowledge base is indexed by the Facebook AI Similarity Search library [19] over dense vector embeddings and is rebuilt by a dedicated utility whenever any source is updated.

### **3-6-6 LLM Adapter and Backend Selection**

The Large Language Model adapter exposes a single interface that supports two backends. The first is a deterministic null backend that performs no generation and serves as the no-LLM experimental control: every Large-Language-Model field in the resulting alert is left empty, so the alert contains only the deterministic correlation tier, the MITRE ATT&CK technique mapping, the evidence identifiers, and the retrieved knowledge passages. The second is an OpenAI-compatible backend that talks to any inference endpoint conforming to the OpenAI Application Programming Interface; the default runtime path uses this backend to serve a general-purpose open-weight Large Language Model (Llama 3.1-8B-Instruct, in the Q4\_K\_M quantization) through a local Ollama inference daemon at the loopback address. Every alert records the backend that produced it through a dedicated metadata field, which permits direct experimental comparison of the no-LLM baseline against the general-purpose model under identical evidence and prompt conditions. The substitution of a cybersecurity-specialized Large Language Model in place of the general-purpose model, and the comparison of the two specializations under the same evidence and prompt conditions, is reserved for future work.

### **3-6-7 Hallucination Guard**

The hallucination guard is a deterministic post-processing module that inspects every Large Language Model output and verifies that all factual claims are supported by the retrieved passages or the input evidence. Any output that introduces a MITRE technique identifier, a CIS control number, an AWS service name, or a NIST control identifier not present in the retrieved passages or CloudTrail evidence is rejected. Rejected outputs are replaced with a deterministic alert containing only the evidence and the deterministic MITRE mapping. This mechanism ensures that no alert delivered to the responder contains unsupported claims.

### **3-6-8 Final Correlated Alerts**

Alerts are serialized into JavaScript Object Notation document and a corresponding Comma-Separated Value record. Each alert includes the session principal, the associated CloudTrail events, MITRE ATT&CK technique labels, the matched Phase One finding identifier or Phase Two path

identifier, the grounded explanation generated by the Large Language Model, the cited knowledge base passages, recommended remediation, and the backend that produced the alert.

### **3-7 Inter-Phase Data Flow and Output Formats**

The framework is implemented as a unified Python pipeline that shares a common AWS inventory between Phase One and Phase Two through a single runner entry point, and that processes the resulting findings and attack paths in Phase Three through a dedicated Phase Three runner. Both execution paths emit their outputs as JavaScript Object Notation documents validated against phase-specific schemas, alongside corresponding Comma-Separated Value files for analyst-facing review. The pipeline operates deterministically in configurations without Large Language Model generation, and remains deterministic up to the configured temperature parameter in LLM-augmented modes.

### **3-8 Conclusion**

This chapter described the proposed Cloud IAM Sentinel framework, including its three-phase architecture, the data collection layer that produces the unified inventory, the static analysis layer that evaluates posture rules and computes privilege-escalation reachability with explicit IAM Condition evaluation and tag-mutation modeling, and the dynamic correlation layer that integrates CloudTrail evidence with grounded Large Language Model reasoning. It outlined the modules within each phase, their inputs and outputs, and the schemas that govern data exchange. The next chapter presents the implementation environment and tools used to realize the proposed framework.

## **Chapter Four: Practical Implementation**

## 4-1 Introduction

This chapter presents the practical implementation of Cloud IAM Sentinel and the evaluation environment in which the framework operates. It begins by describing the programming environment and the read-only AWS permission policy governs all interactions with the cloud account under analysis. The chapter then enumerates the Phase One rule catalogue, organized by rule family, and justifies each family with respect to whether equivalent protection could be provided by network-layer controls or AWS-native services. The implementation of Phase Two and Phase Three is subsequently described. The chapter concludes with a description of the open ground-truth datasets used to evaluate the framework and the open-source comparison tools deployed alongside Cloud IAM Sentinel for the head-to-head evaluation presented in Chapter Five.

## 4-2 Implementation Environment

### 4-2-1 Programming Language and Libraries

The framework is implemented in Python (version 3.10 or 3.11). The data collection layer relies on the official AWS Software Development Kit for Python (boto3), which provides the read-only Application Programming Interface clients used to retrieve IAM configuration data, Amazon Elastic Compute Cloud inventory, and CloudTrail events. The Phase Two privilege-graph engine is built on the NetworkX library for in-memory directed-graph construction and traversal. The Phase Three retrieval layer indexes the external knowledge base using the Facebook AI Similarity Search library [19] over dense vector embeddings produced by an open-weight sentence-embedding model and the Large Language Model is served locally through a self-hosted inference daemon exposing an OpenAI-compatible Application Programming Interface on the loopback interface.

### 4-2-2 Read-Only AWS Permission Policy

The framework operates exclusively under a least-privilege, read-only Application Programming Interface policy attached to the user or role executing the assessment. The policy grants IAM read operations required to enumerate users, groups, roles, managed and inline policies, password policies, Multi-Factor Authentication device assignments, and the credential report. It also includes the CloudTrail read operation required to retrieve recent management events during execution, and an AWS Security Token Service operation used to identify the executing principal for the operator-trail-exclusion logic in Phase Three.

No write operations are permitted, and no identities are created, modified, or deleted by the framework. Cloud IAM Sentinel therefore satisfies the principle of least privilege at the deployment level while enforcing it at the analysis level.

## 4-3 Phase One — Static IAM Rule Catalog

### 4-3-1 Rule Organization into Detection Families

The Phase One catalog contains forty-five rules organized into six detection families. Thirty-nine rules address the Role-Based Access Control surface and are summarized at the family level in

Table 4-1. Six rules address the Attribute-Based Access Control surface and are presented at the rule level in Table 4-2.

**Table 4-1: Role-Based Access Control Rules**

| Family                                  | Rules | Detection focus                                                                                                                                                                                                                                                                                                                                                                                                                                                | Authoritative basis                                                                                                                                       |
|-----------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| A — Identity policy structure           | 19    | Admin-equivalent managed policy attachments to non-administrative principals; privilege-escalation primitives across identity, group, role, inline, and trust policies; full-access grants on sensitive services such as CloudTrail and AWS Key Management Service; overly permissive trust-policy principals; and missing aws:SourceArn or aws:SourceAccount conditions in cross-service trust relationships (the confused-deputy pattern documented by AWS). | CIS AWS Foundations Benchmark v3.0.0 §1.16, §1.17; NIST SP 800-53 Rev. 5 AC-6 (Least Privilege), AC-3 (Access Enforcement)                                |
| C — Credential and authentication state | 8     | Presence of access key on the AWS account root user; absence of Multi-Factor Authentication for the root user in both software and hardware forms; absence of Multi-Factor Authentication for console-enabled IAM users; centralized management of root credentials at the AWS Organizations level; multiple active access keys associated with a single user; and recent root-account activity as reported in the credential report.                          | CIS AWS Foundations Benchmark v3.0.0 §1.4–§1.10; NIST SP 800-53 Rev. 5 IA-2 (Identification and Authentication), AC-2 (Account Management)                |
| D — Credential lifecycle                | 4     | Access keys not rotated within ninety days; access keys and console access unused for forty-five days, increasing the attack surface; and expired Transport Layer Security server certificates retained in IAM that may be inadvertently reused.                                                                                                                                                                                                               | CIS AWS Foundations Benchmark v3.0.0 §1.14, §1.12, §1.13; NIST SP 800-53 Rev. 5 AC-2(3) (Disable Inactive Accounts), IA-5 (Authenticator Management)      |
| E — Account-level password policy       | 6     | Minimum password length of fourteen characters; password reuse-prevention setting of twenty-four; and enforcement of four character classes (lowercase, uppercase, numeric, and symbolic). The length and reuse rules are classified as medium severity according to CIS, while the character-class requirements are classified as low severity, as password length is the dominant factor in entropy.                                                         | CIS AWS Foundations Benchmark v3.0.0 §1.8, §1.9; NIST SP 800-53 Rev. 5 IA-5(1) (Password-Based Authentication)                                            |
| F — Provisioning hygiene                | 2     | Absence of dedicated security-audit and incident-response roles separate from administrative principals. These rules enforce governance by ensuring that audit and incident-response activities are performed using purpose-built roles rather than shared high-privilege credentials.                                                                                                                                                                         | CIS AWS Foundations Benchmark v3.0.0 §1.17; NIST SP 800-53 Rev. 5 AC-5 (Separation of Duties), AC-6(2) (Non-Privileged Access for Non-Security Functions) |

**Table 4-2: Attribute-Based Access Control Rules (Family B)**

| Rule                                           | Pattern detected                                                                                                                                                                                                                  | Required condition key               | Authoritative basis                                                                                                                     |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| PassRole without pass condition                | An iam:PassRole grant that omits the iam:PassedToService condition, allowing the holder to attach an IAM role to arbitrary AWS services rather than a specific intended service.                                                  | iam:PassedToService                  | AWS IAM User Guide — Granting a user permissions to pass a role to an AWS service [1]; NIST SP 800-162 §3.2 (subject attribute scoping) |
| Tag-write without aws:TagKeys                  | A tag-write action (e.g., iam:TagUser, iam:TagRole, ec2:CreateTags, s3:PutBucketTagging) granted without an aws:TagKeys allowlist, permitting unrestricted modification of tag keys.                                              | aws:TagKeys                          | AWS IAM Tutorial — ABAC condition operators [9]; NIST SP 800-162 §3.4 (attribute write controls)                                        |
| Sensitive IAM action without MFA condition     | A sensitive IAM action (e.g., iam:CreateAccessKey, iam:UpdateAssumeRolePolicy, iam:AttachRolePolicy) granted without an aws:MultiFactorAuthPresent condition, allowing execution without multi-factor authentication.             | aws:MultiFactorAuthPresent           | NIST SP 800-162 §3.1.3.5 (environment condition / authentication assurance); CIS AWS Foundations Benchmark v3.0.0 §1.10                 |
| Delegated creation without permission boundary | A delegated identity-creation grant (e.g., iam:CreateUser, iam:CreateRole) without an iam:PermissionsBoundary condition, allowing creation of principals that are not constrained by the delegating entity's permission boundary. | iam:PermissionsBoundary              | AWS IAM User Guide — Permissions boundaries for IAM entities [1]; NIST SP 800-162 §3.6 (delegation containment)                         |
| Resource creation without aws:RequestTag       | A resource-creation action (e.g., ec2:RunInstances, s3:CreateBucket) granted without enforcement of aws:RequestTag, allowing creation of untagged resources that bypass subsequent ABAC policies based on aws:ResourceTag.        | aws:RequestTag                       | AWS IAM Tutorial — ABAC for AWS resources [9]; NIST SP 800-162 §3.4 (attribute write controls)                                          |
| Wildcard policy without tag conditions         | An identity policy that grants wildcard actions over wildcard resource in an environment managed under Attribute-Based Access Control, but lacks aws:PrincipalTag or aws:ResourceTag conditions to constrain the grant.           | aws:PrincipalTag,<br>aws:ResourceTag | NIST SP 800-162 §3.5 (attribute-based decision); AWS IAM Condition Operators reference [4]                                              |

Each rule is implemented as a Python class exposing a single evaluate method and is accompanied by a metadata file that specifies the rule's severity, the basis for that severity, the authoritative source from which the rule is derived, the corresponding CIS Critical Security Control identifier [5], and the corresponding NIST Special Publication 800-53 Revision 5 control identifier [16].

#### **4-3-2 Family A — Identity Policy Structure**

The nineteen rules of Family A operate over the inventory of users, groups, and roles. Each rule systematically evaluates all attached managed and inline policies associated with each principal. The corresponding policy documents are retrieved from the cached inventory, with a fallback to the IAM policy collection by Amazon Resource Name when necessary. For each policy, the evaluation iterates through every Statement element and applies a set of predefined pattern-based checks.

A shared utility module provides reusable pattern primitives that encapsulate common detection logic. These primitives are used by all rules in the family to ensure consistent evaluation. They include checks for administrative-level permissions (such as wildcard actions over wildcard resources), detection of specific high-risk or privileged actions, identification of unbounded resource scopes, and recognition of well-known high-privilege managed policies. By centralizing this logic, the framework avoids duplication and ensures that all rules apply the same criteria in a uniform manner.

Each rule evaluates whether any policy statement matches a risky pattern. When a match is found, the rule generates a violation record that captures detailed context, including the principal type, principal identifier, policy type (inline or attached), policy name, the index of the statement within the policy, and a human-readable explanation of the issue. If no matches are found, the rule returns a PASS verdict. If one or more violations are detected, the rule returns a FAIL verdict along with the list of corresponding violation records.

Representative rules in this family include detection of administrator-level access granted without Multi-Factor Authentication, identification of privilege-escalation patterns based on curated sets of sensitive actions, detection of cross-service trust policies that lack `aws:SourceArn` or `aws:SourceAccount` conditions, and identification of policies that are directly attached to users instead of being assigned through groups or roles.

#### **4-3-3 Family B — Attribute-Based Access Control**

A shared utility module provides a set of helper functions that check whether specific condition keys are present inside the Condition element of a policy statement. Each helper function corresponds to one required ABAC constraint, such as verifying the presence of `iam:PassedToService`, `aws:TagKeys`, or `aws:MultiFactorAuthPresent`.

During evaluation, each rule applies these checks to determine whether a policy statement includes the necessary restrictions. If a required condition key is present, the permission is considered constrained. If the condition key is missing, the permission is treated as unsafe because it allows broader or unintended access than intended.

When such a missing condition is detected, the rule generates a structured violation record. This record includes the policy name, the index of the statement in which the issue was found, and the

specific condition key that is absent. These structured records are later used in Phase Three to correlate static policy weaknesses with CloudTrail events that trigger the same permission patterns.

#### **4-3-4 Family C — Credential and Authentication State**

The eight rules of Family C operate on credential-state data derived from the credential report and from IAM Multi-Factor Authentication device listings, both of which are collected and stored in the unified inventory. These rules evaluate the configuration and usage state of credentials for both the root account and IAM users.

The analysis includes detection of active access keys on the root account, absence of Multi-Factor Authentication for the root account in both software and hardware forms, and absence of Multi-Factor Authentication for console-enabled IAM users. It also identifies users with multiple active access keys, recent usage of the root account within a defined threshold window, and IAM users provisioned simultaneously with console and programmatic credentials.

Additional checks evaluate organization-level controls, such as the root-credentials-management feature flag exposed through AWS Organizations. Each rule produces structured violation records identifying the affected principal and the relevant credential-state attributes. For rules related to the root account, the synthetic root identifier is used. A rule returns a FAIL verdict when any violation is detected and PASS otherwise.

#### **4-3-5 Family D — Credential Lifecycle**

The four rules of Family D evaluate time-based credential hygiene by analyzing timestamp fields from the credential report and the IAM server-certificate inventory. These timestamps are parsed in ISO-8601 format and compared against the current time to determine credential age and inactivity.

The rules collectively detect access keys that exceed rotation thresholds, access keys and console credentials that have remained unused beyond defined inactivity windows, and expired Transport Layer Security server certificates retained in IAM. Missing usage timestamps are interpreted as indicating credentials that have never been used.

Threshold values are configurable through rule metadata, with default values of ninety days for key rotation and forty-five days for inactivity, consistent with the CIS AWS Foundations Benchmark version three controls.

#### **4-3-6 Family E — Account-Level Password Policy**

The six rules of Family E evaluate the account-level password policy retrieved through the IAM GetAccountPasswordPolicy operation. These rules assess both quantitative requirements, such as minimum password length and password reuse prevention, and qualitative requirements related to character composition.

The evaluation verifies enforcement of lowercase, uppercase, numeric, and symbolic character requirements, and compares policy thresholds against values prescribed by the CIS AWS Foundations Benchmark. In addition, the absence of an account-level password policy is treated as a violation, as it indicates that no password constraints have been configured for IAM users.



#### **4-3-7 Family F — Provisioning Hygiene**

The two rules of Family F evaluate the presence of dedicated security-audit and incident-support roles within the AWS account. These rules inspect the inventory of IAM roles to determine whether the AWS-managed SecurityAudit and AWSSupportAccess policies are attached to any role. A violation is reported when no such role exists.

Both rules serve as governance checks rather than detection of directly exploitable misconfigurations. Their severity is therefore calibrated at low, as the absence of these roles reflects a deficiency in audit and incident-response preparedness rather than an immediately exploitable attack path.

#### **4-3-8 Limitations of Network-Layer Controls in Substituting Phase One Rules**

Network-layer security controls cannot substitute for the rules implemented in Phase One. None of the forty-five rules can be replaced by a firewall, a Network Access Control List, an AWS Security Group, the AWS Web Application Firewall, or a network-based Intrusion Detection or Intrusion Prevention System, because each rule evaluates properties of the AWS Identity and Access Management control plane, which is not observable by devices operating at OSI layers three or four.

An AWS Application Programming Interface call issued by any AWS Software Development Kit is signed by the calling principal and transmitted over Transport Layer Security to the AWS service endpoint. A firewall observes only the five-tuple of source address, destination address, protocol, and port, together with encrypted payload that terminates at the AWS endpoint. As a result, it cannot extract the API operation name, the role being passed, the tag being modified, the policy document being attached, or the Multi-Factor Authentication state of the session. A Network Access Control List operates on the same five-tuple with even less context, as it is stateless. AWS Security Groups similarly evaluate layer three and layer four metadata at the instance boundary and do not inspect the contents of AWS Application Programming Interface requests. The AWS Web Application Firewall inspects only Hypertext Transfer Protocol traffic routed through CloudFront, an Application Load Balancer, the Application Programming Interface Gateway, or AppSync; AWS service endpoints such as iam.amazonaws.com and sts.amazonaws.com are not protected by this mechanism. Network-based Intrusion Detection and Prevention systems rely on signature matching over unencrypted or decrypted traffic; a system capable of interpreting AWS IAM policy semantics would no longer function as a conventional intrusion detection system, but rather as a specialized IAM analysis engine.

The underlying reason for this separation is that an attacker in possession of valid IAM credentials, obtained through credential exfiltration, instance metadata abuse, or other means, can invoke the AWS Application Programming Interface using correctly signed requests from any network location permitted by policy. From the perspective of a network-layer control, such requests are indistinguishable from legitimate administrative activity. Authorization decisions are made within AWS by the IAM service, which evaluates policies attached to the calling principal. Consequently, a rule that detects whether an IAM policy permits an unintended action under specific conditions cannot be implemented by any mechanism that does not have access to policy data and does not execute the IAM policy evaluation logic.

### **4-3-9 Evaluation of AWS Native IAM Security Services**

AWS services operating at the same control-plane layer, including AWS IAM Identity Center, AWS Config, AWS Security Hub, Amazon GuardDuty, AWS Organizations Service Control Policies, and IAM Permission Boundaries, provide partial overlap with the checks implemented in Phase One, but none offers complete functional equivalence to the Phase One rule catalogue.

AWS IAM Identity Center reduces the need for some Phase One rules by replacing direct authentication with federated access, but it does not address root-account, policy-structure, Attribute-Based Access Control, or credential lifecycle risks.

AWS Config covers only a limited subset of rules and does not produce structured outputs suitable for downstream analysis, while AWS Security Hub aggregates findings without extending detection coverage.

Amazon GuardDuty provides behavioral detection after events occur, whereas Phase One performs proactive analysis of credential posture; the two approaches are therefore complementary.

Service Control Policies and Permission Boundaries enforce constraints and limit the impact of misconfigurations but do not detect them; instead, they are modeled as constraints in Phase Two.

The framework therefore retains all forty-five rules. The Role-Based Access Control rules are included as a grounded and auditable baseline that produces structured findings for downstream analysis. The Attribute-Based Access Control rules capture patterns not collectively addressed by existing open-source IAM analysis tools. No peer-reviewed open-source IAM analyzer, including PMapper, Cloudsplaining, Pacu, TAC [6], and SkyEye [31], detects this combination of patterns. The integration of Phase One findings into Phase Two and Phase Three constitutes a system-level capability not provided by existing AWS-native services.

## **4-4 Phase Two — Privilege-Graph Implementation**

### **4-4-1 Graph Construction**

The graph builder iterates over the unified inventory and emits one node per principal, including IAM users, IAM roles, and IAM groups. Groups are modeled as aggregation nodes rather than independent actors, as AWS does not authorize requests directly at the group level.

Resource-policy modeling for Amazon Simple Storage Service buckets, AWS Key Management Service keys, and AWS Lambda functions is reserved for future work. In the current implementation, edges that would otherwise originate from these resources are inferred from the trust policies of the receiving roles.

Edges are constructed in two passes. The first pass enumerates all privilege-granting actions present in identity-based policies within the inventory. Each candidate edge is classified using an `EdgeType` enumeration, which includes actions such as `ASSUME_ROLE`, `PASS_ROLE`, `ATTACH_USER_POLICY`, `ATTACH_ROLE_POLICY`, `PUT_USER_POLICY`,

PUT\_ROLE\_POLICY, CREATE\_ACCESS\_KEY, CREATE\_POLICY\_VERSION, and UPDATE\_ASSUME\_ROLE\_POLICY, as well as the tag-mutation action family.

The second pass evaluates each candidate edge using a policy-evaluation core that reproduces the AWS IAM deny-overrides semantics. For each candidate, the system checks whether the source principal is explicitly denied the corresponding action by examining all applicable policies, including attached, inline, and group-inherited policies. If a matching explicit deny is found, the edge is pruned. Permission boundaries are applied as an additional constraint by intersecting the allowed permissions of each principal, further filtering the resulting set of edges.

#### 4-4-2 IAM Condition Evaluator

The Condition evaluator reads each statement's Condition block and iterates over every operator key (StringEquals, StringNotEquals, StringEqualsIfExists, StringLike, Bool, ArnLike, ArnEquals, Null, ForAllValues:StringEquals, ForAnyValue:StringEquals) for which an explicit handler is defined. The dispatch mechanism supports both the literal operator name and its case-insensitive normalized form. Operator names carrying a ForAllValues: or ForAnyValue: quantifier prefix is first stripped of the prefix, after which the remaining operator is dispatched to the corresponding single-key handler with the appropriate quantifier semantics.

The evaluator returns one of three verdicts per Condition: true, false, or unknown. The unknown verdict is produced whenever an operator without a defined handler is encountered or when the request-context value of a referenced condition key cannot be statically determined from the inventory. In such cases, the candidate edge is marked as conditional rather than admitted unconditionally, preserving soundness in the presence of unsupported operators or incomplete context.

The IfExists family of operators, together with the Null operator, encode the fail-open and fail-closed semantics defined by AWS. These semantics are preserved by explicitly checking for paired Null conditions on the same key whenever an IfExists operator is evaluated.

#### 4-4-3 Tag-Mutation Edges and Joint-State Reachability

Tag-mutation actions are modeled as a distinct edge class. When the graph engine encounters a principal that has any of the iam:TagUser, iam:UntagUser, iam:TagRole, iam:UntagRole, iam:TagPolicy, ec2:CreateTags, ec2>DeleteTags, s3:PutBucketTagging, or s3>DeleteBucketTagging permissions on a target whose tag attributes appear in a downstream *Condition*, the engine emits a STATE\_TRANSITION edge that mutates the principal's or the target's tag-state hash.

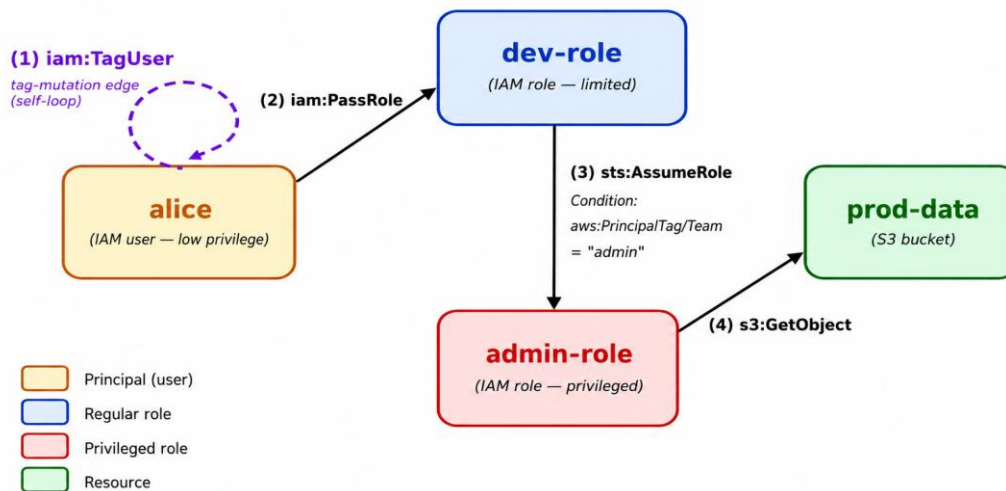
The reachability search uses a dedicated joint-state traversal strategy. Nodes are visited under a (node identity, tag-state hash) key rather than under node identity alone, allowing the same node to be visited multiple times under different attribute configurations.

The joint-state search is bounded by two parameters: the overall graph hop limit max\_hops, with a default value of five, which bounds the total number of edges traversed from any source, and the tag-mutation budget k, with a default value of three, which bounds the number of STATE\_TRANSITION edges that the search may take within those five hops. The default value of three for the tag-mutation budget is pre-registered in the project's pre-registration document and

reflects the empirical observation that every published path in the IAM Vulnerable benchmark and the Pathfinding.cloud catalog is reachable within three tag-mutation steps.

Figure 9 presents an illustrative example, showing how tag mutation can open a privilege-escalation path that would be missed by a permission-only graph. In this example, the principal *alice* can self-mutate her own tags through `iam:TagUser`; can pass *dev-role* to a downstream service through `iam:PassRole`; can then assume the privileged *admin-role* through `sts:AssumeRole`, gated by a Condition requiring the principal to carry the tag `Team = admin`; and the assumed *admin-role* can read the *prod-data* S3 bucket through `s3:GetObject`.

Without the tag-mutation edge, the `AssumeRole` condition fails and the path is closed. The joint-state breadth-first search visits *alice* once with the original tag set and a second time after the self-loop is traversed, thereby exposing the escalation that a permission-only graph would miss.



**Figure 9: Illustrative example of tag-mutation-aware privilege-escalation path discovery**

#### 4-4-4 Bounded Breadth-First Search

The path engine exposes two breadth-first traversal functions. The first performs conventional permission-only privilege-graph traversal and is used to discover paths that do not require attribute mutation. The second performs the joint-state traversal described in the previous subsection and is used for tag-mutation paths. Both traversals are bounded by `max_hops`, with a default value of five. The joint-state variant is additionally bounded by the tag-mutation budget `k`, with a default value of three.

Each traversal terminates a path when the breadth-first search reaches a node that has been flagged as `is_admin` by the graph builder. The admin-flagging routine uses two complementary detection strategies: an admin-by-document check, which scans all effective policy documents, including attached, inline, and group-inherited policies, for an Allow effect on the wildcard action over the wildcard resource; and an admin-by-policy-name shortcut, which recognizes the AWS-managed `AdministratorAccess` policy by its Amazon Resource Name, even when its document is not fetched into the inventory.

Each discovered attack path is annotated with the source principal, the sequence of traversed edges, the EdgeType of each edge, the encountered Condition verdicts, and any tag-state mutations performed along the path.

#### **4-4-5 Service Control Policy and Permission Boundary Filtering**

The Service Control Policy (SCP) filter applies organization-level guardrails to every candidate path after the breadth-first search produces it. The filter traverses the path's edge sequence and, for each edge action, evaluates the SCP hierarchy from the organization root down to the calling account using the same deny-overrides semantics as the IAM evaluator. A path is annotated as `scp_blocked` when any SCP in the hierarchy denies the action, and as `scp_evaluated_clean` when every SCP either explicitly allows or does not restrict the action.

The Permission Boundary filter operates as a per-principal intersection mask during graph construction rather than as a post-filter on complete paths. When a principal has an `iam:PermissionsBoundary` attached, the boundary policy's permitted action set is intersected with the principal's effective identity-policy permissions. Any candidate edge that depends on an action outside this intersection is pruned at construction time.

This separation between post-processing SCP evaluation and construction-time Permission Boundary enforcement reflects the difference in scope between the two mechanisms: SCPs apply at the account level, whereas Permission Boundaries apply at the individual principal level.

### **4-5 Phase Three — Correlation and Grounded Reasoning Implementation**

#### **4-5-1 CloudTrail Event Ingestion and Normalization**

In this stage, the framework processes the CloudTrail events collected from the AWS environment and converts them into a unified event format. This normalization step is necessary because CloudTrail records different AWS actions using complex and service-specific structures. By converting these records into a consistent format, the framework can later correlate them with the static findings from Phase One and the attack paths produced in Phase Two.

For each CloudTrail event, the framework first identifies the actor that performed the action. It reads the `userIdentity` field and classifies the actor into a simplified principal type, such as root user, IAM user, assumed role, AWS service, federated user, AWS account, anonymous user, or unknown identity. This simplified classification allows the correlation engine to compare CloudTrail activity with IAM principals represented in the framework.

The framework also extracts the target of each event from the `requestParameters` field. Depending on the event type, the target may be an IAM user, an IAM role, or another AWS resource. For events related to tag modification, the framework additionally extracts the tag key and tag value, since these values are important for detecting ABAC-related activity and tag-mutation behavior.

#### **4-5-2 Deterministic MITRE ATT&CK Mapping**

In this stage, the framework maps normalized CloudTrail events to the MITRE ATT&CK framework using a deterministic lookup process. Each supported CloudTrail event name is

matched against a curated mapping table that links the event to its corresponding MITRE ATT&CK tactic, technique, and sub-technique, including their official identifiers.

The mapping table is manually curated from the public MITRE ATT&CK for Cloud matrix rather than learned automatically. This means that the framework does not guess the ATT&CK category of an event. If a CloudTrail event name does not have a defined entry in the mapping table, the framework marks it as UNMAPPED instead of assigning an unsupported technique label.

Some events can be mapped more accurately when additional context from Phase Two is available. For example, an event name may be ambiguous on its own, but if Phase Two shows that the event belongs to a PASS\_ROLE attack edge, the framework can refine the mapping to the relevant ATT&CK sub-technique.

To make the mapping transparent, each mapped event includes a `mapping_basis` value. This value indicates how the ATT&CK mapping was produced:

- **HIGH:** the event was mapped directly from its CloudTrail event name
- **MEDIUM:** the event was mapped using additional Phase Two edge context
- **NONE:** no grounded ATT&CK mapping was available, so the event was marked as UNMAPPED

This deterministic approach ensures that every alert carries a verifiable ATT&CK reference when a valid mapping exists. It also prevents the framework from fabricating technique labels when the available evidence is insufficient.

#### **4-5-3 Correlation with Phase One and Phase Two Outputs**

In this stage, the framework correlates the normalized CloudTrail events with the outputs of the previous two phases. It compares each CloudTrail event with the Phase One findings and the Phase Two attack paths using the principal identity as the main matching key. In practice, the caller Amazon Resource Name recorded in the CloudTrail event is compared with the principal Amazon Resource Names stored in the Phase One findings and Phase Two paths. When the same principal appears in both CloudTrail activity and a previous-phase result, the framework treats the event as related to that previously identified risk.

After matching, each result is assigned to a correlation tier. A result is classified as **STRONGLY\_CORRELATED** when the same principal has both a high- or critical-severity Phase One finding and a Phase Two attack path. It is classified as **PARTIALLY\_CORRELATED** when only one of these two sources is present, either a Phase One finding or a Phase Two path. If the event does not match any previous-phase result but corresponds to a sensitive MITRE ATT&CK-mapped action, it is classified as **DYNAMIC\_STANDALONE**. Finally, when a Phase One finding exists for a principal but no related CloudTrail activity is observed, the result is classified as **STATIC\_STANDALONE**.

For dynamic standalone events, the framework uses a curated sensitive-action catalog to assign an `event_sensitivity` value. This value is not extracted from CloudTrail and is not a severity value defined by MITRE ATT&CK. Instead, the framework first maps selected sensitive CloudTrail event names to MITRE ATT&CK tactics using a curated catalog of procedure-linked actions. It

then assigns a framework-defined sensitivity label according to the mapped tactic family. Events associated with Privilege Escalation, Persistence, Credential Access, and Lateral Movement are labelled HIGH, whereas events associated with Defense Evasion and Discovery are labelled MEDIUM.

This `event_sensitivity` field is kept separate from the final alert severity. For DYNAMIC\_STANDALONE events, where no Phase One finding or Phase Two attack path provides additional threat context, the framework does not assign an overall severity based only on the CloudTrail event. Instead, it reports the sensitivity of the event itself. Overall alert severity is reserved for cases where sufficient threat context is established through correlation between static findings, attack paths, and observed CloudTrail activity.

#### **4-5-4 Knowledge Base and Vector Index**

The framework uses a curated knowledge base to support grounded explanations in Phase Three. This knowledge base contains authoritative passages collected from trusted sources, including MITRE ATT&CK technique pages, AWS service authorization documentation, the AWS IAM Action Reference, CIS AWS Foundations Benchmark control descriptions, and official AWS remediation guidance. These sources provide the factual background used by the framework when explaining alerts and recommended remediation steps.

To make this knowledge searchable, the framework divides the source documents into smaller text chunks and converts each chunk into a numerical embedding using a local sentence-embedding model. These embeddings are stored in a vector index, together with metadata that preserves the source of each chunk, its category, related AWS actions, and whether it comes from an authoritative source. This allows the framework to retrieve relevant supporting evidence when generating an explanation for a detected alert.

During retrieval, the framework converts the alert context into a query embedding and searches the vector index for the most semantically similar knowledge-base chunks. The retrieved passages are ranked by semantic similarity, meaning that the most relevant text is returned based on meaning rather than exact keyword matching. Source metadata is preserved with each retrieved chunk, but it does not directly control the ranking. This ensures that retrieval is driven by relevance to the alert while still allowing the system to show where the supporting information came from.

If the vector index is unavailable, the system does not fail. Instead, it continues using only the deterministic evidence already produced by the correlation engine, such as the CloudTrail event, MITRE mapping, Phase One finding, and Phase Two path. In this case, the language model receives only the structured correlation evidence, preventing it from relying on unsupported external context

#### **4-5-5 Large Language Model Adapter and Two Backends**

The framework uses a Large Language Model adapter to generate explanatory text for correlated alerts in Phase Three. The adapter is designed with a single generation interface, but it can operate with two different backends depending on the experiment configuration.

The first backend is a no-LLM baseline. In this mode, no text generation is performed. All language-model-generated fields in the alert remain empty, and the alert contains only

deterministic information, such as the correlation tier, MITRE ATT&CK mapping, evidence identifiers, and any retrieved knowledge-base passages. This backend is used as an experimental control, allowing the framework to measure what is produced without any LLM assistance.

The second backend is an OpenAI-compatible inference backend, which can communicate with any model-serving endpoint that follows the OpenAI API format. In the current implementation, this backend is configured to use a locally served Llama-3.1-8B-Instruct model through Ollama. This allows the framework to generate alert explanations while keeping the model execution local.

For reproducibility, the model temperature is fixed at zero in all evaluation runs. This reduces randomness and makes the generated outputs more stable across repeated executions. Each alert also records the selected backend and model name in its metadata, allowing direct comparison between the no-LLM baseline and the LLM-assisted configuration under the same evidence and prompt conditions.

In the current evaluation, the framework uses a general-purpose language model to test whether grounded prompting and deterministic evidence are sufficient to produce useful alert explanations.

#### **4-5-6 Hallucination Guard**

The framework includes a hallucination guard to ensure that any text generated by the Large Language Model remains grounded in the available evidence. After the correlation stage assembles the evidence for an alert, the hallucination guard checks the LLM output against that evidence before the alert is finalized.

When the framework is run in the no-LLM baseline mode, the guard does not perform hallucination checking because no language-model-generated claims exist. In this case, the output is marked as NOT\_APPLICABLE rather than ungrounded. This distinction is important because an empty LLM output in the baseline configuration should not be treated as a hallucination.

When an LLM backend is used, the guard validates four types of factual claims. First, every evidence identifier referenced by the LLM must exist in the evidence bundle. Second, every MITRE tactic mentioned in the generated explanation must match a tactic already present in the evidence. Third, every MITRE technique identifier must follow the expected ATT&CK identifier format and must also appear in the known technique list for that alert. Fourth, any Amazon Resource Name mentioned in the generated free text is checked against the evidence to ensure that the model has not introduced a resource that was not present in the input.

If any of these checks fail, the generated explanation is discarded. The framework then falls back to a deterministic alert containing only the verified evidence and the deterministic MITRE ATT&CK mapping. This design ensures that responders do not receive alerts containing unsupported LLM-generated claims.



## 4-6 Evaluation Datasets and Ground-Truth Catalogs

### 4-6-1 IAM Vulnerable Benchmark

IAM Vulnerable, published by Bishop Fox [11], is a benchmark that deploys intentionally vulnerable IAM configurations into a real AWS account using Terraform. It provides a set of in-scope privilege-escalation scenarios that can be used to evaluate whether the framework is able to detect known IAM attack paths. In this work, IAM Vulnerable is used as a primary benchmark for measuring the detection capability of Phase Two, especially for privilege-escalation paths that arise from misconfigured IAM permissions.

### 4-6-2 Pathfinding.cloud Catalog

The Pathfinding.cloud catalog, published by Datadog Security Labs [13], documents a collection of cloud privilege-escalation paths using a structured schema. Each path describes the required permissions and the expected sequence of actions that enable escalation. The catalog is used to guide the privilege-grant edge types modeled in Phase Two and to verify that the framework's privilege graph can represent known published escalation paths.

### 4-6-3 CloudGoat Scenarios

CloudGoat, published by Rhino Security Labs, is an open-source vulnerable AWS environment that includes several practical privilege-escalation scenarios. In this project, CloudGoat is used as an additional evaluation source for testing the framework against realistic AWS attack exercises. The evaluation harness includes ground-truth annotations and scoring logic for these scenarios. The full execution of CloudGoat-based experiments is reserved for the Chapter Five evaluation campaign.

### 4-6-4 Stratus Red Team Attack Catalog

Stratus Red Team, published by Datadog Security Labs, is an adversary-emulation framework that reproduces cloud attack techniques in a controlled environment. It is particularly useful for evaluating Phase Three because it can generate CloudTrail events corresponding to known attack behaviors. In this work, Stratus is used as a source of ground-truth attack activity for testing whether the correlation layer can correctly associate observed CloudTrail events with known adversarial techniques. The full Stratus-based evaluation is presented as part of the Chapter Five experimental campaign.

### 4-6-5 flaws.cloud CloudTrail Dataset

The flaws.cloud CloudTrail dataset is a public dataset containing CloudTrail activity collected from a benign AWS account over an extended period. In this project, it is used as a false-positive baseline for Phase Three. Since the dataset represents benign activity, any security alert produced against it is treated as a potential false positive. This allows the framework to evaluate whether its correlation logic and hallucination guard can suppress unsupported or weakly grounded alerts. The full false-positive evaluation using this dataset is reserved for Chapter Five.

## 4-7 Conclusion

This chapter described the practical implementation of Cloud IAM Sentinel. It first presented the programming environment and the read-only AWS permission policy that constrains every interaction with the target AWS account. It then described the forty-five rules of Phase One, organized into six detection families, including the shared detection logic and the structured violation schema used to report findings.

The chapter also explained the Phase Two privilege-graph implementation, including the edge taxonomy, deny-overrides evaluation, IAM Condition handling, tag-mutation edges, and joint-state breadth-first search. In this phase, privilege-escalation paths are explored under a maximum hop limit of five and a tag-mutation budget of three. The implementation also distinguishes between Service Control Policy filtering, which is applied after path discovery, and Permission Boundary filtering, which is applied during graph construction.

Phase Three was then described as the correlation and grounded-reasoning layer of the framework. This phase normalizes CloudTrail events, maps them deterministically to MITRE ATT&CK techniques, correlates them with Phase One findings and Phase Two attack paths, and assigns each result to one of four correlation tiers: strongly correlated, partially correlated, dynamic standalone, or static standalone. The chapter further described the knowledge-base retrieval component, the two-backend Large Language Model adapter, and the hallucination guard used to prevent unsupported model-generated claims from reaching the final alert.

Finally, the chapter introduced the open ground-truth datasets and attack catalogs used by the evaluation harness, while distinguishing between datasets exercised in the present evaluation and datasets reserved for future experimental work. The next chapter reports the evaluation results obtained from applying this implementation to the selected IAM benchmarks, attack-path catalogs, and CloudTrail-based scenarios.

## **Chapter Five: Evaluation**

## 5-1 Introduction

This chapter reports the evaluation of Cloud IAM Sentinel against externally curated benchmarks and staged end-to-end attack scenarios. The evaluation is structured around two complementary axes. The first axis evaluates each phase separately using relevant ground-truth datasets and benchmark corpora. The second axis evaluates the full pipeline by exercising the framework on staged attack scenarios deployed in a dedicated Amazon Web Services (AWS) account.

The chapter begins by presenting the methodology used across the evaluation. It then reports the Phase One results against the Bishop Fox IAM Vulnerable adversarial principal set and the Phase Two results against the Datadog Pathfinding.cloud sixty-six-path catalog and the Bishop Fox IAM Vulnerable thirty-one-path benchmark. Next, it presents the integrated end-to-end validation on four CloudGoat scenarios, showing how the framework's static analysis, privilege-graph reachability, CloudTrail correlation, and grounded reasoning operate together across distinct attack patterns. Finally, the chapter evaluates Phase Three recall using Stratus Red Team CloudTrail attack events.

## 5-2 Methodology

The evaluation is based on three principles. The first principle is the use of two layers of evidence. Each phase is evaluated both separately, using an external ground-truth corpus where available, and as part of the integrated pipeline on staged attack scenarios. The phase-level evaluation shows how each component performs on its own, while the integrated evaluation shows whether the complete framework can operate end-to-end on realistic attack activity.

The second principle is to use descriptive metrics, confidence intervals, and paired statistical comparisons where applicable, rather than relying on evaluator-defined numeric thresholds. The evaluation does not define fixed success thresholds such as a required percentage of recall or a maximum allowed false-positive rate. Instead, the results are reported using measured detection counts, recall values, confidence intervals, and comparisons against external ground truth or relevant baselines. This approach reduces the risk of selecting arbitrary thresholds and makes the evaluation more transparent.

The third principle is the use of externally authored and reproducible ground truth. Each benchmark or attack catalog used in the evaluation is derived from an open-source, version-controlled, and independently published artifact. Each ground-truth file records its source attribution and verification date against the upstream release. Statistical analysis follows established conventions: proportions are reported with Wilson score confidence intervals [20], paired binary outcomes are tested using McNemar's exact test [21], multiple comparisons are corrected using the Bonferroni adjustment with a family-wise alpha of 0.05 [22], and non-parametric confidence intervals on rate differences are computed using percentile bootstrap with 1,000 resamples and a fixed seed [34].

The fourth principle is that the comparison metric should match the design goal of the tools being compared. When two tools have the same objective, the evaluation can compare them directly using the same detection metric. For example, if the Phase Two privilege-escalation graph engine is compared with Principal Mapper, both are graph-based reachability analyzers, so the

comparison can be reported as a performance contrast on shared attack-path detection. Broader comparisons with tools that have different design goals are left for future work.

### 5-3 Phase One - Static IAM Posture Evaluation

Phase One defines a catalog of forty-five static IAM posture rules, derived from the CIS AWS Foundations Benchmark v3.0.0, the AWS Foundational Security Best Practices catalog, NIST SP 800-53 Rev. 5, and NIST SP 800-162 guidance on attribute-based access control. These rules cover IAM posture risks, policy misconfigurations, credential-related weaknesses, and ABAC-related conditions.

Phase One is evaluated through adversarial accuracy on the nine adversarial principals deployed by the Bishop Fox IAM Vulnerable free-resources Terraform module. Each principal is paired with one to three Phase One rules whose verdict depends on the specific evaluation hazard that the principal is designed to exercise. Across the nine principals, these cases define fourteen pre-committed (principal, rule) pairs recorded in the ground-truth file.

A direct comparison with existing open-source IAM analysis tools such as Prowler, Principal Mapper, Cloudsplaining, is left for future work, as discussed in the limitations section.

#### 5-3-1 Evaluating Phase One Accuracy on IAM Vulnerable False-Positive and False-Negative Principals

Bishop Fox IAM Vulnerable [11] deploys nine adversarial principals designed to test the correctness of static IAM analyzers. These principals are divided into five false-positive cases, labeled FP1 through FP5, and four false-negative cases, labeled FN1 through FN4.

The false-positive cases test whether the analyzer avoids reporting risks that are not actually exploitable. Their apparent permissions are neutralized by explicit Deny statements, resource constraints that do not match, or Conditions that cannot be satisfied. Specifically, FP1, FP2, and FP3 test deny-overrides behavior across single-policy, cross-policy, and wildcard Deny cases. FP4 tests a Resource-scope constraint, while FP5 tests a Condition expression that never evaluates to true. A correct analyzer should therefore avoid classifying these principals as exploitable.

The false-negative cases test whether the analyzer can still detect exploitable permissions even when they are hidden behind surface-level policy complexity. These cases include partial-permission cross-principal escalation (FN1), an exploitable Resource constraint that matches a deployed resource (FN2), an exploitable Condition that evaluates to true (FN3), and a NotAction-based wildcard grant (FN4). A correct analyzer should classify these principals as exploitable.

Together, these cases define a correctness matrix of fourteen pre-committed (principal, rule) verdict pairs used to evaluate Phase One adversarial accuracy. These fourteen pairs are not fourteen separate principals; rather, they are the predefined principal-rule test cases derived from the nine adversarial principals, where each principal is associated with the Phase One rule or rules relevant to its specific policy structure.

Phase One was executed against the deployed IAM Vulnerable account, and the resulting verdicts were tabulated. As shown in Table 5-1, eleven of the fourteen pairs received a strictly correct

verdict. Three pairs were classified as acceptable over-approximations, corresponding to documented static-analysis limitations involving Resource scoping and Condition expressions that cannot be fully resolved from the static inventory alone. No pair received an incorrect verdict.

The three acceptable over-approximations occur only in the FP4 and FP5 cases. These cases are not treated as framework defects, because their non-exploitability depends on dynamic Resource constraints or Condition behavior that a static analyzer cannot soundly resolve without additional runtime context.

**Table 5-1: Phase One adversarial-accuracy results (n = 14 pairs).**

| Verdict class                 | Count   | Proportion | Wilson 95 % CI   |
|-------------------------------|---------|------------|------------------|
| Strictly correct              | 11 / 14 | 78.6 %     | [52.4 %, 92.4 %] |
| Acceptable over-approximation | 3 / 14  | 21.4 %     | [7.6 %, 47.6 %]  |
| Incorrect                     | 0 / 14  | 0.0 %      | [0.0 %, 21.5 %]  |

The result satisfies the pre-registered win condition for adversarial accuracy: zero incorrect verdicts on the fourteen in-scope ground-truth pairs, with three acceptable over-approximations explicitly recognized as documented limits of static IAM analysis. Phase One correctly applies deny-overrides-allow semantics on the FP1, FP2, and FP3 principals, correctly identifies the FN4 NotAction case as a risk and treats the unresolved FP4 and FP5 cases as possible risks because they cannot be fully resolved through static analysis alone.

## 5-4 Phase Two - Privilege-Escalation Graph Evaluation

Phase Two is evaluated using two independent ground-truth corpora to reduce the risk of relying on a single benchmark. The primary corpus is the Datadog Pathfinding.cloud catalog [13], which contains sixty-six published AWS privilege-escalation paths. The catalog was evaluated at upstream commit f7170aff.

The secondary corpus is the Bishop Fox IAM Vulnerable benchmark [11]. This benchmark provides twenty-one in-scope path classes, while the full published denominator contains thirty-one paths. The thirty-one-path denominator is used to remain consistent with the baseline numbers reported by Bishop Fox in 2021.

### 5-4-1 Pathfinding.cloud Catalog as Primary Corpus

The Pathfinding.cloud catalog is used as the primary corpus for evaluating Phase Two. Comparator detection counts for PMapper, Cloudsplaining, and Pacu are taken from the catalog's published detection-tool attribution field. Therefore, the framework is compared against the catalog's published attribution data rather than against newly executed runtime outputs from those tools. A broader runtime comparison with external IAM analysis tools is left for future work, as discussed in the limitations section.

Phase Two detected 48 of 66 paths under the loose criterion and 45 of 66 paths under the strict criterion. In comparison, the catalog attribution reports 20 paths for PMapper, 19 paths for Cloudsplaining, and 19 paths for Pacu. These detection results are summarized in Table 5-2.

**Table 5-2: Phase Two detection on the Pathfinding.cloud catalog (n = 66 paths).**

| <b>Detector</b>                      | <b>Loose detected</b> | <b>Strict detected</b> | <b>Loose Wilson 95 % CI</b> |
|--------------------------------------|-----------------------|------------------------|-----------------------------|
| Cloud IAM Sentinel (this work)       | 48 / 66 (72.7 %)      | 45 / 66 (68.2 %)       | [61.0 %, 82.0 %]            |
| PMapper (catalog attribution)        | 20 / 66 (30.3 %)      | 20 / 66 (30.3 %)       | -                           |
| Cloudsplaining (catalog attribution) | 19 / 66 (28.8 %)      | 19 / 66 (28.8 %)       | -                           |
| Pacu (catalog attribution)           | 19 / 66 (28.8 %)      | 19 / 66 (28.8 %)       | -                           |

Six paired McNemar exact tests were computed against the three IAM-focused catalog-attribution comparators: three under the loose detection criterion and three under the strict detection criterion. Bonferroni correction was applied across the three comparators for each detection criterion, giving a corrected threshold of  $\alpha' = 0.05 / 3 \approx 0.0167$ , as specified in the pre-registered methodology. Because both loose and strict criteria are reported, a stricter correction across all six tests would give  $\alpha' = 0.05 / 6 \approx 0.0083$ ; all six observed p-values remain below this stricter threshold. The paired test results are shown in Table 5-3.

**Table 5-3: Paired McNemar's exact tests on the Pathfinding.cloud detection vector.**

| <b>Comparator</b> | <b>Criterion</b> | <b>p-value</b> | <b>Significant after Bonferroni</b> |
|-------------------|------------------|----------------|-------------------------------------|
| PMapper           | loose            | < 0.0001       | yes                                 |
| PMapper           | strict           | 0.0002         | yes                                 |
| Cloudsplaining    | loose            | < 0.0001       | yes                                 |
| Cloudsplaining    | strict           | < 0.0001       | yes                                 |
| Pacu              | loose            | < 0.0001       | yes                                 |
| Pacu              | strict           | < 0.0001       | yes                                 |

Phase Two satisfies the pre-registered Pathfinding.cloud comparison against the catalog-attributed PMapper baseline, detecting 45 strict-matched paths compared with 20 PMapper-attributed paths, with paired significance after Bonferroni correction. The results also show that Phase Two detects strictly more catalog paths than each of the three IAM-focused catalog-attribution comparators under both the loose and strict criteria. Comparisons with broader academic baselines such as TAC and SkyEye are not treated as behavioral head-to-head executions on the 66-path catalog in this evaluation and are therefore discussed as a methodological limitation and future-work direction.

#### **5-4-2 IAM Vulnerable Benchmark as Secondary Corpus**

The IAM Vulnerable benchmark is used as a secondary corpus for evaluating Phase Two. On the twenty-one in-scope COVERED path classes defined in the IAM Vulnerable ground truth, Phase Two detects 20 of 21 paths, corresponding to 95.2% recall with a Wilson 95% confidence interval of [77.3%, 99.2%].

When the denominator is restricted to the nineteen COVERED classes whose required principal and trust-policy topology is present in the deployed free-resources Terraform environment, Phase Two detects 19 of 19 paths, corresponding to 100% recall with a Wilson 95% confidence interval of [83.2%, 100.0%].

For comparability with the original Bishop Fox 2021 baseline, the evaluation also reports results against the full thirty-one-path denominator. Under this broader denominator, Phase Two detects 20 of 31 paths, corresponding to 64.5% recall with a Wilson 95% confidence interval of [46.9%, 78.9%]. The full-denominator comparison with the published Bishop Fox 2021 baseline results is summarized in Table 5-4.

This dual-denominator reporting distinguishes between the paths that are actually instantiated in the deployed test environment and the broader published benchmark denominator.

**Table 5-4: Phase Two detection on IAM Vulnerable (n = 31 paths).**

| Detector                          | Detected | Proportion | Wilson 95 % CI   |
|-----------------------------------|----------|------------|------------------|
| PMapper                           | 22 / 31  | 71.0 %     | -                |
| Pacu                              | 21 / 31  | 67.7 %     | -                |
| Cloud IAM Sentinel<br>(this work) | 20 / 31  | 64.5 %     | [46.9 %, 78.9 %] |
| Cloudsplaining                    | 19 / 31  | 61.3 %     | -                |
| AWSPX                             | 14 / 31  | 45.2 %     | -                |

The two corpora measure different aspects of privilege-escalation detection and therefore produce different rankings. The Bishop Fox 2021 benchmark is a legacy IAM privilege-escalation benchmark that predates the broader adoption of attribute-based access control (ABAC). It also includes several version-one non-goals, which were paths that the original benchmark did not expect first-generation tools to detect.

In contrast, the Pathfinding.cloud catalog is newer, broader, and includes ABAC-related paths. It also provides per-path detection-tool attribution, allowing Cloud IAM Sentinel to be compared against catalog-published comparator coverage using the catalog’s own ground truth.

On the Pathfinding.cloud catalog, Cloud IAM Sentinel detects substantially more paths than the catalog-attribution comparators. On the legacy IAM Vulnerable thirty-one-path denominator, the framework remains competitive, detecting 20 of 31 paths. This places it behind PMapper by two paths and behind Pacu by one path, while still exceeding the published Cloudsplaining and AWSPX baselines.

## 5-5 Stratus Red Team Phase Three Recall Evaluation

This section evaluates Phase Three against an independent attack catalog provided by Datadog Stratus Red Team [35]. Unlike the CloudGoat scenarios in Section 5-6, which execute full attack chains across all three phases of the framework, Stratus Red Team provides single-technique attack tests. Each technique performs one specific IAM or AWS Security Token Service (STS) action, such as creating a backdoor role, issuing a federation token, or modifying a user login profile, and produces the CloudTrail events associated with that action.

The goal of this evaluation is to determine whether Phase Three can detect each in-scope Stratus technique and assign the correct MITRE ATT&CK label using the framework’s deterministic mapping. The evaluation also checks whether the LLM-augmented configuration preserves this deterministic correctness while adding grounded explanation text for the headline alert.



The ground-truth catalog used in this section is the framework's locally aligned version of the Stratus IAM and STS technique set as of 2026-05-06. This alignment was performed against the upstream Datadog Stratus Red Team repository. The attacker identity used in this evaluation is a single IAM user named stratus-attacker.

Each Stratus technique was evaluated using the same process. First, any required prerequisites were prepared under the attacker's credentials. Second, the technique was executed. Third, the evaluation waited for CloudTrail propagation. Fourth, Phase Three was run using the deterministic-only configuration. Finally, Phase Three was run again on a single-event filtered CloudTrail input using the Llama-3.1-8B-Instruct backend to generate the LLM-augmented headline alert.

The deterministic and LLM-augmented outputs were then merged. For each technique, the headline event includes the LLM-generated explanation, while the remaining alerts retain only deterministic fields. After each technique was evaluated, its prerequisite resources were removed before the next technique was executed.

#### **5-5-1 Technique One: iam-create-admin-user (Persistence)**

The iam-create-admin-user technique establishes persistence by creating a new IAM user, attaching the AWS-managed AdministratorAccess policy to that user, and creating an access key for it. After the technique is executed, the attacker has a second admin-equivalent identity in the account with long-lived programmatic credentials.

Stratus classifies this technique under the Persistence tactic and the parent MITRE ATT&CK technique T1136 — Create Account. The framework maps the headline CloudTrail event, CreateUser, to Persistence / T1136.003 — Create Account: Cloud Account. This mapping is consistent with the Stratus classification because T1136.003 is a cloud-specific sub-technique under the parent technique T1136.

The detonation produces three CloudTrail events in sequence: CreateUser, AttachUserPolicy with the policy ARN pointing to AdministratorAccess, and CreateAccessKey on the newly created user. The framework emits one Phase Three alert for each event. The headline alert is the CreateUser event. Figure 10 presents the deterministic Phase Three alert for this headline event, while Figure 11 presents the LLM-augmented version of the same alert. The two follow-on alerts, AttachUserPolicy and CreateAccessKey, are included in the per-technique outputs and summarized in Table 5-5 at the end of this section.

```
{
  "alert_id": "cef1f4562c9d",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "overall_severity": "HIGH",
  "source_cloudtrail_event_names": ["CreateUser"],
  "principal_arns": ["arn:aws:iam::344809605381:user/stratus-attacker"],
  "source_phase1_check_ids": ["iam_user_administrator_access_policy", "iam_aws_attached_policy_no_administrative_privileges",
    "iam_policy_attached_only_to_group_or_roles"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Persistence",
    "technique_id": "T1136",
    "sub_technique_id": "T1136.003"
  },
  "p1_match_confidence": "ARN_EXACT"
}
```

**Figure 10: Phase Three deterministic alert for the iam-create-admin-user headline event.**

```
{
  "alert_id": "cef1f4562c9d",
  "mitre": {
    "tactic": "Persistence",
    "sub_technique_id": "T1136.003",
    "llm_tactic_confirmed": "Persistence / T1136 Create Account / T1136.003 Cloud Account",
    "llm_technique_inferred": "INSUFFICIENT EVIDENCE"
  },
  "llm_explanation":
    "The alert matters because a user with administrator access (stratus-attacker) was created via CloudTrail event 'CreateUser' by 'arn:aws:iam::344809605381:user/stratus-attacker'. This user has administrator access, as seen in Phase 1 findings (check_id=iam_user_administrator_access_policy), which is a high-risk activity.",
  "grounding_valid": true
}
```

**Figure 11: LLM-augmented Phase Three alert for the iam-create-admin-user headline event.**

The detection result for this technique is 1 of 1. The primary CloudTrail event declared for this technique is CreateUser, and the framework emitted one alert whose source\_cloudtrail\_event\_names field contains CreateUser. Therefore, the per-technique detection rate is 100%.

The LLM-augmented version of the headline alert preserves the deterministic alert identifier, correlation tier, severity, and MITRE ATT&CK label. The hallucination guard reports grounding\_valid = true, indicating that the generated explanation is supported by the evidence bundle. The verdict for this technique is Pass.

### 5-5-2 Technique Two: iam-backdoor-user (Credential Access)

The iam-backdoor-user technique creates a parallel credential channel by creating a new access key for an existing legitimate user. In the evaluation, the legitimate user is created during the warm-up step. In a real-world attack, this user would already exist in the target account and would not be part of the attacker's setup. The detonation produces one CloudTrail event: CreateAccessKey, whose request parameters identify the legitimate user as the target.

Stratus classifies this technique under the Persistence tactic and the parent MITRE ATT&CK technique T1098 — Account Manipulation. The framework maps CreateAccessKey to Credential Access / T1098.001 — Account Manipulation: Additional Cloud Credentials. The tactic differs

because Stratus views the new access key as a persistence mechanism, while the framework follows the MITRE Cloud Matrix event-level interpretation, where creating an access key is treated as credential access. The technique mapping still matches at the parent level because T1098.001 is a sub-technique of T1098.

Figure 12 presents the deterministic Phase Three alert for the iam-backdoor-user headline event. Figure 13 presents the LLM-augmented version of the same alert. In this case, the hallucination guard reports `grounding_valid = false` because the model produced a paraphrased explanation of the MITRE mapping that could not be directly anchored to a specific entity in the evidence bundle. This does not indicate that the alert or MITRE mapping is incorrect; it only means that the LLM generated explanation did not satisfy the strict grounding check.

```
{
  "alert_id": "15c7f6403a88",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "overall_severity": "HIGH",
  "source_cloudtrail_event_names": ["CreateAccessKey"],
  "principal_arns": ["arn:aws:iam::344809605381:user/stratus-attacker"],
  "source_phase1_check_ids": ["iam_user_administrator_access_policy", "iam_aws_attached_policy_no_administrative_privileges",
    "iam_policy_attached_only_to_group_or_roles"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Credential Access",
    "technique_id": "T1098",
    "sub_technique_id": "T1098.001"
  },
  "p1_match_confidence": "ARN_EXACT"
}
```

**Figure 12: Deterministic Phase Three alert for the CreateAccessKey event in iam-backdoor-user.**

```
{
  "alert_id": "15c7f6403a88",
  "mitre": {
    "tactic": "Credential Access",
    "sub_technique_id": "T1098.001",
    "llm_tactic_confirmed": "Persistence, Privilege Escalation (deterministic MITRE mapping)",
    "llm_technique_inferred": "INSUFFICIENT EVIDENCE"
  },
  "llm_explanation":
    "The alert matters because it indicates a potential privilege escalation threat due to the creation of an access key by a user with administrator privileges (check_id=iam_user_administrator_access_policy) and additional cloud credentials being added to the account (deterministic MITRE mapping).",
  "grounding_valid": false,
  "ungrounded_fields": ["mitre_tactic_confirmed"]
}
```

**Figure 13: LLM-augmented Phase Three alert for the same CreateAccessKey event.**

The detection result for this technique is 1 of 1. The primary CloudTrail event is CreateAccessKey, and the framework emitted one Phase Three alert whose `source_cloudtrail_event_names` field contains CreateAccessKey. Therefore, the per-technique detection rate is 100%.

The deterministic alert is not affected by the LLM grounding result. It is still emitted with the correct correlation tier, severity, and MITRE ATT&CK label. The verdict for this technique is Pass for detection and MITRE mapping. The failed LLM grounding result is reported transparently as an output of the hallucination guard, not as a failure of the deterministic detection pipeline.

### 5-5-3 Technique Three: iam-backdoor-role (Privilege Escalation)

The iam-backdoor-role technique creates a long-lived attacker access path by modifying the trust policy of an existing IAM role. The modified trust policy allows a foreign AWS account to assume the role. Unlike the create-backdoor-role technique discussed in Section 5-5-4, this technique does not create a new role. Instead, it changes the trust relationship of a role that already exists. The detonation produces one CloudTrail event: UpdateAssumeRolePolicy, with the new trust policy included in the event's request parameters.

Stratus classifies this technique under the Persistence tactic and the parent MITRE ATT&CK technique T1098 — Account Manipulation. The framework maps UpdateAssumeRolePolicy to Privilege Escalation / T1098.003 — Account Manipulation: Additional Cloud Roles. The tactic differs because the framework treats the trust-policy change as privilege escalation: the foreign principal can now assume the role and gain the permissions attached to it. Stratus treats the same behavior as persistence because the modified trust policy creates a long-lived access path. The technique mapping still matches at the parent level because T1098.003 is a sub-technique of T1098.

Figure 14 presents the deterministic Phase Three alert for the UpdateAssumeRolePolicy event in iam-backdoor-role. Figure 15 presents the LLM-augmented version of the same alert.

```
{
  "alert_id": "43aadb434d61",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "overall_severity": "HIGH",
  "source_cloudtrail_event_names": ["UpdateAssumeRolePolicy"],
  "principal_arns": ["arn:aws:iam::344809605381:user:stratus-attacker"],
  "source_phase1_check_ids": ["iam_user_administrator_access_policy", "iam_aws_attached_policy_no_administrative_privileges",
    "iam_policy_attached_only_to_group_or_roles"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Privilege Escalation",
    "technique_id": "T1098",
    "sub_technique_id": "T1098.003"
  },
  "p1_match_confidence": "ARN_EXACT"
}
```

**Figure 14: Deterministic Phase Three alert for the UpdateAssumeRolePolicy event in iam-backdoor-role.**

```
{
  "alert_id": "43aadb434d61",
  "mitre": {
    "tactic": "Privilege Escalation",
    "sub_technique_id": "T1098.003",
    "llm_tactic_confirmed": "Persistence, Privilege Escalation(deterministic MITRE mapping:T1098.003 Additional Cloud Roles)",
    "llm_technique_inferred": "INSUFFICIENT EVIDENCE"
  },
  "llm_explanation":
    "The attacker has updated an IAM role's assume policy, which may indicate privilege escalation or account manipulation. This is supported by the UpdateAssumeRolePolicy CloudTrail event and Phase 1 findings indicating administrator access policies attached to the attacker's user.",
  "grounding_valid": true
}
```

**Figure 15: LLM-augmented Phase Three alert for the same UpdateAssumeRolePolicy event.**

The detection result for this technique is 1 of 1. The primary CloudTrail event is UpdateAssumeRolePolicy, and the framework emitted one Phase Three alert whose source\_cloudtrail\_event\_names field contains UpdateAssumeRolePolicy. Therefore, the per-technique detection rate is 100%.

The LLM-augmented alert preserves the deterministic alert identifier, correlation tier, severity, and MITRE ATT&CK label. The hallucination guard reports grounding\_valid = true, meaning that the generated explanation is supported by the evidence bundle. The verdict for this technique is Pass, with a documented tactic-level difference between the framework mapping and the Stratus classification.

#### 5-5-4 Technique Four: iam-create-backdoor-role (Persistence)

The iam-create-backdoor-role technique establishes persistence by creating a new IAM role whose trust policy allows a foreign AWS account to assume it. The technique then attaches the AWS-managed AdministratorAccess policy to the new role. After this role is created, the foreign account can call sts:AssumeRole and obtain administrator-equivalent credentials in the victim account.

Stratus classifies this technique under the Persistence tactic and MITRE ATT&CK technique T1098.003 — Account Manipulation: Additional Cloud Roles. The framework maps the two CloudTrail events separately. The CreateRole event is mapped to Persistence / T1136.003 — Create Account: Cloud Account, while the AttachRolePolicy event is mapped to Privilege Escalation / T1098.003 — Account Manipulation: Additional Cloud Roles.

This difference occurs because Stratus provides a technique-level classification for the full attack, while the framework applies a deterministic mapping to each individual CloudTrail event. Therefore, the framework treats the role-creation event as cloud-account creation and the policy-attachment event as privilege escalation. Both mappings are defensible because they describe different parts of the same attack chain.

Figure 16 presents the deterministic Phase Three alert for the CreateRole event in iam-create-backdoor-role. Figure 17 presents the LLM-augmented version of the same alert.

```
{
  "alert_id": "2de16c9967fb",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "overall_severity": "HIGH",
  "source_cloudtrail_event_names": ["CreateRole"],
  "principal_arns": ["arn:aws:iam::344809605381:user:stratus-attacker"],
  "source_phase1_check_ids": ["iam_user_administrator_access_policy", "iam_aws_attached_policy_no_administrative_privileges",
    "iam_policy_attached_only_to_group_or_roles"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Persistence",
    "technique_id": "T1136",
    "sub_technique_id": "T1136.003"
  },
  "p1_match_confidence": "ARN_EXACT"
}
```

**Figure 16: Deterministic Phase Three alert for the CreateRole event in iam-create-backdoor-role.**

```

{
  "alert_id": "2de16c9967fb",
  "mitre": {
    "tactic": "Persistence",
    "sub_technique_id": "T1136.003",
    "llm_tactic_confirmed": "Persistence (T1136) as confirmed by deterministic MITRE mapping.",
    "llm_technique_inferred": "INSUFFICIENT EVIDENCE"
  },
  "llm_explanation":
    "The alert matters because a malicious user 'stratus-attacker' created an IAM role with administrator privileges (check_id=iam_aws_attached_policy_no_administrative_privileges), which can be used for privilege escalation.",
  "grounding_valid": true
}

```

**Figure 17: LLM-augmented Phase Three alert for the same CreateRole event.**

The detection result for this technique is 2 of 2. The technique produces two CloudTrail events, CreateRole and AttachRolePolicy, and the framework emits one Phase Three alert for each event. Therefore, the per-technique detection rate is 100%.

The LLM-augmented version of the headline CreateRole alert preserves the deterministic alert identifier, correlation tier, severity, and MITRE ATT&CK label. The hallucination guard reports `grounding_valid = true`, meaning that the generated explanation is supported by the evidence bundle. The verdict for this technique is Pass.

#### 5-5-5 Technique Five: iam-create-user-login-profile (Persistence)

The iam-create-user-login-profile technique establishes a console-access channel by adding a login profile, or console password, to an existing IAM user that was originally intended for programmatic access only. After the technique is executed, the user can sign in through the AWS Management Console using the attacker-defined password. This creates a parallel access path that may not be detected by rules focused only on access-key abuse. The detonation produces one CloudTrail event: CreateLoginProfile.

Stratus classifies this technique under the Persistence tactic and MITRE ATT&CK technique T1098.001 — Account Manipulation: Additional Cloud Credentials. The framework maps CreateLoginProfile to Persistence / T1078.004 — Valid Accounts: Cloud Accounts. The tactic matches, but the technique identifier differs at the parent-technique level.

This difference occurs because Stratus treats the added console password as an additional credential on an existing account. The framework follows the MITRE Cloud Matrix event-level interpretation, where adding a console login path is treated as a valid-account access pattern. Both interpretations are defensible because they describe different aspects of the same behavior.

Figure 18 presents the deterministic Phase Three alert for the CreateLoginProfile event in iam-create-user-login-profile. Figure 19 presents the LLM-augmented version of the same alert.

```
{
  "alert_id": "cfbf3af226ac",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "overall_severity": "HIGH",
  "source_cloudtrail_event_names": ["CreateLoginProfile"],
  "principal_arns": ["arn:aws:iam::344809605381:user/stratus-attacker"],
  "source_phase1_check_ids": ["iam_user_administrator_access_policy", "iam_aws_attached_policy_no_administrative_privileges",
    "iam_policy_attached_only_to_group_or_roles"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Persistence",
    "technique_id": "T1078",
    "sub_technique_id": "T1078.004"
  },
  "p1_match_confidence": "ARN_EXACT"
}
```

**Figure 18: Deterministic Phase Three alert for the CreateLoginProfile event in iam-create-user-login-profile.**

```
{
  "alert_id": "cfbf3af226ac",
  "mitre": {
    "tactic": "Persistence",
    "sub_technique_id": "T1078.004",
    "llm_tactic_confirmed": "Persistence / Privilege Escalation",
    "llm_technique_inferred": "INSUFFICIENT EVIDENCE"
  },
  "llm_explanation":
    "The alert matters because it indicates a potential privilege escalation threat due to the creation of a login profile by an attacker-controlled user (arn:aws:iam::344809605381:user/stratus-attacker), which has administrator access and is attached to multiple policies, including AWS managed administrator privileges (check_id=iam_aws_attached_policy_no_administrative_privileges). This could be used for persistence or privilege escalation.",
  "grounding_valid": true
}
```

**Figure 19: LLM-augmented Phase Three alert for the same CreateLoginProfile event.**

The detection result for this technique is 1 of 1. The primary CloudTrail event is CreateLoginProfile, and the framework emitted one Phase Three alert whose source\_cloudtrail\_event\_names field contains CreateLoginProfile. Therefore, the per-technique detection rate is 100%.

The LLM-augmented version reports grounding\_valid = true, meaning that the generated explanation is supported by the evidence bundle. The verdict for this technique is Pass, with a documented parent-technique difference between the Stratus classification and the framework's deterministic MITRE mapping.

### 5-5-6 Technique Six: sts-federation-token (Credential Access)

STS federation token from an existing user's access keys. These temporary credentials remain usable for their configured lifetime, even if the original long-lived access key is later disabled. The detonation produces two CloudTrail events: GetFederationToken, which issues the temporary credentials, and GetCallerIdentity, which Stratus uses to confirm that the new credentials work.

Stratus classifies this technique under the Persistence tactic and MITRE ATT&CK technique T1098.001 — Account Manipulation: Additional Cloud Credentials. The framework maps GetFederationToken to Credential Access / T1550.001 — Use Alternate Authentication Material: Application Access Token. The mapping differs because the framework follows the MITRE event-

level interpretation for STS-issued tokens, while Stratus classifies the full abuse pattern as account manipulation. This is a documented mapping difference, not a framework error.

Figure 10 presents the deterministic Phase Three alert for the GetFederationToken event in sts-federation-token. Figure 11 presents the LLM-augmented version of the same alert.

```
{
  "alert_id": "f728f5f52e08",
  "alert_type": "SUSPICIOUS_ACTIVITY",
  "correlation_tier": "DYNAMIC_STANDALONE",
  "overall_severity": "SEE_EVENT_SENSITIVITY",
  "source_cloudtrail_event_names": ["GetFederationToken"],
  "principal_arns": ["arn:aws:iam::344809605381:user:stratus-red-team-user-federation-user"],
  "source_phase1_check_ids": [],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Credential Access",
    "technique_id": "T1550",
    "sub_technique_id": "T1550.001"
  },
  "p1_match_confidence": "NONE"
}
```

**Figure 20: Deterministic Phase Three alert for the GetFederationToken event in sts-federation-token.**

```
{
  "alert_id": "f728f5f52e08",
  "mitre": {
    "tactic": "Credential Access",
    "sub_technique_id": "T1550.001",
    "llm_tactic_confirmed": "Credential Access / T1550 Use Alternate Authentication Material / T1550.001 Application Access Token",
    "llm_technique_inferred": "INSUFFICIENT EVIDENCE"
  },
  "llm_explanation":
    "The alert matters because it indicates a potential security threat related to credential access. The 'GetFederationToken' CloudTrail event by the user 'stratus-red-team-user-federation-user' maps to MITRE ATT&CK T1550.001 (Use Alternate Authentication Material), which suggests an attempt to bypass typical authentication processes and access restricted accounts or services.",
  "grounding_valid": true
}
```

**Figure 21: LLM-augmented Phase Three alert for the same GetFederationToken event.**

The detection result for this technique is 1 of 1. The primary CloudTrail event is GetFederationToken, and the framework emitted one Phase Three alert whose source\_cloudtrail\_event\_names field contains GetFederationToken. Therefore, the per-technique detection rate is 100%.

The alert is classified as DYNAMIC\_STANDALONE because the principal that generated the headline event was created during the Stratus warm-up step and was not present in the earlier Phase One scan. Therefore, no Phase One finding could be linked to that principal.

The LLM-augmented version reports grounding\_valid = true, meaning that the generated explanation is supported by the evidence bundle. The verdict for this technique is Pass, with a



documented tier downgrade for an unknown principal and a documented MITRE mapping difference between Stratus and the framework.

### 5-5-7 Technique Seven: iam-update-user-login-profile (Privilege Escalation)

The iam-update-user-login-profile technique performs a console-account takeover by changing the password of an existing console-enabled IAM user. After the technique is executed, the attacker can sign in as that user with the new password. This represents credential takeover rather than privilege escalation through a new permission grant. The detonation produces one CloudTrail event: UpdateLoginProfile. The event request parameters identify the target user by name, while the new password itself is not visible in CloudTrail.

Stratus classifies this technique under the Privilege Escalation tactic and the parent MITRE ATT&CK technique T1098 — Account Manipulation. The framework maps UpdateLoginProfile to Persistence / T1078.004 — Valid Accounts: Cloud Accounts. The mapping differs because the framework treats console-password modification as a valid-account persistence pattern, while Stratus treats the same action as a privilege-escalation technique through account takeover. Both interpretations are defensible because they describe different aspects of the same behavior.

Figure 12 presents the deterministic Phase Three alert for the UpdateLoginProfile event in iam-update-user-login-profile. Figure 13 presents the LLM-augmented version of the same alert.

```
{
  "alert_id": "ed121900f5c6",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "overall_severity": "HIGH",
  "source_cloudtrail_event_names": ["UpdateLoginProfile"],
  "principal_arns": ["arn:aws:iam:344809605381:user:stratus-attacker"],
  "source_phase1_check_ids": ["iam_user_administrator_access_policy",
    "iam_aws_attached_policy_no_administrative_privileges", "iam_policy_attached_only_to_group_or_roles"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Persistence",
    "technique_id": "T1078",
    "sub_technique_id": "T1078.004"
  },
  "p1_match_confidence": "ARN_EXACT"
}
```

**Figure 22: Deterministic Phase Three alert for the UpdateLoginProfile event in iam-update-user-login-profile.**

```

{
  "alert_id": "ed121900f5c6",
  "mitre": {
    "tactic": "Persistence",
    "sub_technique_id": "T1078.004",
    "llm_tactic_confirmed": "Persistence / Privilege Escalation (T1078.004 Cloud Accounts)",
    "llm_technique_inferred": "INSUFFICIENT EVIDENCE"
  },
  "llm_explanation":
    "The principal 'arn:aws:iam::344809605381:user:stratus-attacker' has updated their login profile using the 'UpdateLoginProfile' CloudTrail event, which matches Phase 1 findings of HIGH severity, including iam_aws_attached_policy_no_administrative_privileges and iam_user_administrator_access_policy. This indicates potential privilege escalation.",
  "grounding_valid": true
}

```

**Figure 23: LLM-augmented Phase Three alert for the same UpdateLoginProfile event.**

The detection result for this technique is 1 of 1. The primary CloudTrail event is UpdateLoginProfile, and the framework emitted one Phase Three alert whose source\_cloudtrail\_event\_names field contains UpdateLoginProfile. Therefore, the per-technique detection rate is 100%.

The LLM-augmented version reports grounding\_valid = true. The verdict for this technique is Pass, with a documented MITRE mapping difference and a documented minor LLM explanation imprecision.

## 5-6 Integrated End-to-End Validation on CloudGoat Scenarios

The CloudGoat scenario library is used to evaluate the framework end-to-end. Each scenario is deployed separately in the evaluation AWS account. The framework's three phases are then run against the deployed environment, the documented attack chain is executed using the scenario's starting credentials, and the resulting CloudTrail events are collected. Phase Three is then executed on the captured events under both the deterministic-only baseline and the LLM-augmented configuration.

This section reports four scenarios completed in the present evaluation: the negative-control scenario iam\_enum\_basics, the attribute-based access-control re-tagging scenario iam\_privesc\_by\_key\_rotation, the policy-version rollback scenario iam\_privesc\_by\_rollback, and the PassRole-via-Lambda scenario lambda\_privesc.

### 5-6-1 Scenario One: iam\_enum\_basics (Negative Control)

The iam\_enum\_basics scenario is used as a negative control. It deploys one IAM user named cg-bob, one IAM group with an unusual hierarchical path, one role that can be assumed by cg-bob, one customer-managed policy, and one inline policy attached to cg-bob.

The user cg-bob has the AWS-managed IAMReadOnlyAccess policy and an inline policy that allows ec2:DescribeInstances on all resources. However, the user has no permission to modify IAM resources and no permission to assume a role with elevated privileges. The documented attack chain therefore consists only of five read-only IAM API calls used to retrieve flags hidden in the metadata of the deployed resources.

Because the scenario does not involve IAM write actions or privilege escalation, the expected framework behavior is to produce no privilege-escalation evidence and no STRONGLY\_CORRELATED Phase Three alert.

Phase One produces one finding in this scenario: `iam_policy_attached_only_to_group_or_roles`, with MEDIUM severity. This finding is mapped to CIS AWS Foundations Benchmark v3.0.0 control 1.15. It identifies `cg-bob` as a user with a managed policy attached directly to the user account rather than through a group. This is a posture and access-management hygiene finding, not a privilege-escalation claim.

Figure 24 presents the JSON representation of the Phase One finding emitted for this scenario.

```
{
  "check_id": "iam_policy_attached_only_to_group_or_roles",
  "status": "FAIL",
  "severity": "MEDIUM",
  "evidence": {
    "details": {
      "violations": [
        {
          "resource_id": "arn:aws:iam::344809605381:user/cg-bobcgidg0dl1n7zeh",
          "name": "cg-bob-cgidg0dl1n7zeh"
        }
      ]
    }
  }
}
```

**Figure 24: Phase One finding on the `iam_enum_basics` scenario actor.**

Phase Two produced zero attack paths for this scenario, which is the expected behavior because `cg-bob` has read-only IAM permissions only.

None of Phase Three alerts reached the STRONGLY\_CORRELATED tier, and none were classified as ACTIVE\_EXPLOITATION. All alerts were classified as LATENT\_RISK at the PARTIALLY\_CORRELATED tier. This is the expected result because no Phase Two attack path exists from `cg-bob`.

The PARTIALLY\_CORRELATED alerts reflect the correlation between `cg-bob`'s read-only API activity and the Phase One CIS 1.15 posture finding. Therefore, these alerts represent access-management hygiene warnings, not privilege-escalation evidence.

Figure 25 presents the JSON representation of a Phase Three alert produced under the LLM-augmented configuration for the `GetRole` event performed by `cg-bob`.

```
{
  "alert_id": "f8d2d35838e6",
  "alert_type": "LATENT_RISK",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "source_cloudtrail_event_names": ["GetRole"],
  "principal_arns": ["arn:aws:iam::344809605381:user/cg-bob-cgidg0dl1n7zeh"],
  "source_phase1_check_ids": ["iam_policy_attached_only_to_group_or_roles"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Discovery",
    "technique_id": "T1087.004"
  },
  "llm_explanation": "The principal cg-bob-cgidg0dl1n7zeh has direct policy attachments, which may indicate a security risk (check_id=iam_policy_attached_only_to_group_or_roles).",
  "grounding_valid": true
}
```

**Figure 25: Phase Three LLM-augmented alert for the iam\_enum\_basics scenario.**

The negative-control scenario passes the strict-correlation criterion. No alerts reached the STRONGLY\_CORRELATED tier, and no alerts were classified as ACTIVE\_EXPLOITATION. The PARTIALLY\_CORRELATED alerts are expected because cg-bob has a real Phase One posture finding and also generated IAM-related CloudTrail activity. However, since there is no Phase Two attack path, these alerts are classified as LATENT\_RISK rather than active exploitation.

### **5-6-2 Scenario Two: iam\_privesc\_by\_key\_rotation (Attribute-Based Access-Control Re-Tagging)**

The iam\_privesc\_by\_key\_rotation scenario exercises an attribute-based access control (ABAC) re-tagging vulnerability. The scenario deploys a low-privileged user named manager. This user is allowed to perform iam:CreateAccessKey and iam:DeleteAccessKey only on users tagged developer = true. However, the same user also has iam:TagUser permission without a restrictive condition.

The privileged target is an admin user that can assume a role granting access to the scenario secret. The attack chain works by first modifying the tag on the admin user, then deleting an inactive access key to free an available key slot, and finally creating a new access key for the admin user. At that point, the attacker obtains credentials for the privileged user.

Phase One produces two findings for this scenario. The first is the same CIS 1.15 posture finding observed in Scenario One, where a managed policy is attached directly to the manager user. The second finding is iam\_tag\_write\_without\_tagkeys\_restriction, with HIGH severity. This finding is the framework's ABAC vulnerability detector for unsafe tag-writing permissions and represents the main privilege-escalation primitive in this scenario.

The detector identifies that the manager user has a policy statement granting iam:TagUser on Resource: \* without an aws:TagKeys restriction. This allows the user to modify tags that may later be used in ABAC authorization decisions. The finding records the affected principal, policy, and statement, providing clear attribution for the detected risk.

Figure 26 presents the JSON representation of the ABAC re-tagging finding produced for the Scenario Two actor.

```
{
  "check_id": "iam_tag_write_without_tagkeys_restriction",
  "status": "FAIL",
  "severity": "HIGH",
  "evidence": {
    "details": {
      "violations": [
        {
          "name": "manager_cg1d2ztrtw8gc5",
          "policy_type": "inline",
          "policy_name": "TagResources",
          "reason": "IAM tag write actions on Resource:* without aws:TagKeys condition - permits arbitrary tag manipulation"
        }
      ]
    }
  }
}
```

**Figure 26: Phase One ABAC re-tagging finding for iam\_privesc\_by\_key\_rotation.**

Phase Two produced zero attack paths for this scenario. This is expected because the privileged target is the cg\_secretsmanager role, which grants secretsmanager:GetSecretValue on a single secret rather than admin-equivalent access. By design, Phase Two's breadth-first search focuses on paths that reach admin-equivalent principals. Therefore, lateral access to one specific secret is outside the scope of the Phase Two reachability solver.

However, the attack primitive is still detected by Phase One through the ABAC vulnerability-class detector. This shows the complementary role of the framework's phases: Phase Two detects admin-equivalent privilege-escalation paths, while Phase One can detect ABAC misconfiguration patterns that may enable narrower but still sensitive access.

None of Phase Three alerts reached the STRONGLY\_CORRELATED tier because Phase Two did not produce an attack path. However, all alerts were classified as ACTIVE\_EXPLOITATION at the PARTIALLY\_CORRELATED tier. The framework correlates the manager user's runtime CloudTrail events, including TagUser, DeleteAccessKey, and CreateAccessKey, with the Phase One ABAC finding.

The headline CreateAccessKey event is deterministically mapped to MITRE ATT&CK sub-technique T1098.001 — Additional Cloud Credentials, which matches the credential-rotation attack pattern.

Figure 27 presents the JSON representation of the Phase Three LLM-augmented alert for the headline CreateAccessKey event performed by manager.

```

{
  "alert_id": "a8bea45b6f92",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "overall_severity": "HIGH",
  "source_cloudtrail_event_names": ["CreateAccessKey"],
  "principal_arns": ["arn:aws:iam::344809605381:user/manager_cg1d2ztrtw8gc5"],
  "source_phase1_check_ids": ["iam_policy_attached_only_to_group_or_roles", "iam_tag_write_without_tagkeys_restriction"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Credential Access",
    "sub_technique": "Additional Cloud Credentials",
    "sub_technique_id": "T1098.001"
  },
  "llm_explanation":
    "The principal manager_cg1d2ztrtw8gc5 has created an access key using the CreateAccessKey CloudTrail event, which is a potential indicator of privilege escalation via additional cloud credentials. This action is also supported by Phase 1 findings, including iam_policy_attached_only_to_group_or_roles and iam_tag_write_without_tagkeys_restriction.",
  "grounding_valid": true
}

```

**Figure 27: Phase Three LLM-augmented alert for the iam\_privesc\_by\_key\_rotation headline event.**

The LLM-generated output correctly identifies the privilege-escalation primitive, references the relevant Phase One finding identifiers, classifies the alert as unlikely to be a false positive, and provides remediation guidance that specifically names the affected principal.

The verdict for this scenario is Pass. Phase One detects the ABAC vulnerability class before the attack is executed. Phase Two produces no attack path by design because the privileged target is not admin-equivalent. Phase Three correlates the runtime activity with the static finding and produces ACTIVE\_EXPLOITATION alerts with the correct MITRE ATT&CK technique mapping.

Overall, this scenario demonstrates the framework's complementary-detector design. The Phase One vulnerability-class layer detects the static misconfiguration, while the Phase Three correlation layer detects the runtime exploitation, even when the Phase Two breadth-first search does not return an attack path.

### 5-6-3 Scenario Three: iam\_privesc\_by\_rollback (Policy-Version Rollback)

The iam\_privesc\_by\_rollback scenario exercises a privilege-escalation technique based on IAM managed policy versions. In AWS IAM, a managed policy can retain multiple versions, and the active permissions are determined by the policy version marked as the default. A principal with iam:SetDefaultPolicyVersion can change the active default version to another retained version.

The scenario deploys one IAM user named raynor and one customer-managed policy. The current default policy version grants only read-only IAM enumeration permissions and the iam:SetDefaultPolicyVersion action. However, an older retained policy version grants Action: \* on Resource: \*. The attacker discovers this older version and calls iam:SetDefaultPolicyVersion to make it the default. Once this happens, the user effectively becomes an account administrator.

Phase One produces two findings for this scenario. The first is iam\_policy\_allows\_privilege\_escalation, with HIGH severity. This finding is triggered because the deployed customer-managed policy grants the documented privilege-escalation action

iam:SetDefaultPolicyVersion. The second finding is the same CIS 1.15 posture observation reported in the previous scenarios, where a managed policy is attached directly to the raynor user.

The privilege-escalation finding is detected statically before the attack is executed, because Phase One identifies the dangerous permission directly in the policy text.

Figure 28 presents the JSON representation of the Phase One privilege-escalation finding for the iam\_privesc\_by\_rollback policy.

```
{
  "check_id": "iam_policy_allows_privilege_escalation",
  "status": "FAIL",
  "severity": "HIGH",
  "evidence": {
    "details": {
      "violations": [
        {
          "policy": "arn:aws:iam::344809605381:policy/cg-raynor-policy-cgidnqjm8w90pf",
          "allowed_privesc_actions": ["iam:setdefaultpolicyversion"]
        }
      ]
    }
  }
}
```

**Figure 28: Phase One privilege-escalation finding for the iam\_privesc\_by\_rollback policy.**

Phase Two produces one attack path for this scenario. The path is a self-escalation path: the start principal and the end principal are both raynor, and the single edge type is iam:SetDefaultPolicyVersion. The graph analysis identifies that raynor can switch the active version of an already attached policy to a more permissive retained version, thereby elevating their own privileges in one step.

Figure 29 presents the JSON representation of the Phase Two attack path.

```
{
  "path_id": "f3437147",
  "risk_level": "CRITICAL",
  "hop_count": 1,
  "start_principal": {
    "arn": "arn:aws:iam::344809605381:user/raynor-cgidnqjm8w90pf"
  },
  "end_principal": {
    "arn": "arn:aws:iam::344809605381:user/raynor-cgidnqjm8w90pf"
  },
  "steps": [
    { "edge_type": "iam:SetDefaultPolicyVersion" }
  ]
}
```

**Figure 29: Phase Two self-escalation attack path via iam:SetDefaultPolicyVersion.**

Phase Three produces seven alerts for this scenario. Three alerts reach the STRONGLY\_CORRELATED tier with CRITICAL severity, while four additional alerts are classified as PARTIALLY\_CORRELATED. All seven alerts are typed as ACTIVE\_EXPLOITATION.

The three STRONGLY\_CORRELATED alerts represent the framework's strongest detection category because each alert combines evidence from all three phases: a Phase One finding, a Phase Two attack path, and a Phase Three CloudTrail event. The headline event is the iam:SetDefaultPolicyVersion call performed by raynor. This event is deterministically mapped to MITRE ATT&CK sub-technique T1098.003 — Additional Cloud Roles, which matches the policy-version rollback attack pattern.

Figure 30 presents the JSON representation of the Phase Three STRONGLY\_CORRELATED alert for the headline iam:SetDefaultPolicyVersion event under the LLM-augmented configuration.

```
{
  "alert_id": "52bbd0d47d5e",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "STRONGLY_CORRELATED",
  "overall_severity": "CRITICAL",
  "source_cloudtrail_event_names": ["SetDefaultPolicyVersion"],
  "principal_arns": ["arn:aws:iam::344809605381:user/raynor-cgidnqjm8w90pf"],
  "source_phase1_check_ids": ["iam_policy_attached_only_to_group_or_roles", "iam_policy_allows_privilege_escalation"],
  "source_phase2_path_ids": ["f3437147"],
  "mitre": {
    "tactic": "Privilege Escalation",
    "sub_technique": "Additional Cloud Roles",
    "sub_technique_id": "T1098.003"
  },
  "llm_explanation":
    "The alert is strongly correlated due to a dynamic event on a principal with Phase 1 misconfiguration and a reachable Phase 2 escalation path. The principal has direct policy attachments (check_id=iam_policy_attached_only_to_group_or_roles) and policies allowing privilege escalation actions (check_id=iam_policy_allows_privilege_escalation). This self-escalation via iam:SetDefaultPolicyVersion is a critical risk (path_id=f3437147)",
  "grounding_valid": true
}
```

**Figure 30: Phase Three STRONGLY\_CORRELATED LLM-augmented alert for iam\_privesc\_by\_rollback.**

The LLM-generated output references the relevant Phase One check identifiers, the Phase Two path identifier, and the runtime privilege-escalation action by name. The hallucination guard reports the output as grounding-valid, meaning that every referenced entity exists in the evidence bundle.

The verdict for this scenario is Pass. The three phases align on the same privilege-escalation event: Phase One detects the dangerous permission in the policy text, Phase Two reconstructs the self-escalation path, and Phase Three correlates the runtime iam:SetDefaultPolicyVersion call with both the static finding and the attack path. This produces three CRITICAL STRONGLY\_CORRELATED alerts and provides a strong demonstration of the framework's full integrated detection chain.



#### 5-6-4 Scenario Four: lambda\_privesc (PassRole-via-Lambda)

The lambda\_privesc scenario exercises the canonical PassRole-via-Lambda privilege-escalation pattern. The scenario begins with a low-privileged user named chris, who is allowed to assume the cg-lambdaManager role and perform read-only IAM enumeration.

The cg-lambdaManager role has permissions to call iam:PassRole, lambda:CreateFunction, and lambda:InvokeFunction. A separate role, cg-debug, also exists in the account. Its trust policy allows the Lambda service to assume it, and it is attached to the AWS-managed AdministratorAccess policy.

The attack chain works as follows: the attacker first assumes the cg-lambdaManager role, then creates a Lambda function using cg-debug as the execution role, and finally invokes the function. When the function runs, it executes with the privileges of cg-debug and attaches AdministratorAccess to chris.

Phase One produces five findings for this scenario. The most important finding is iam\_passrole\_without\_pass\_condition, with HIGH severity. This finding is triggered because the cg-lambdaManager policy grants iam:PassRole on Resource: \* without conditions such as iam:PassedToService or aws:ResourceTag to restrict which roles can be passed.

The remaining findings describe the surrounding IAM posture. These include high-severity findings on the cg-debug role because it is associated with administrative access, a medium-severity finding for overly permissive identity-policy behavior without tag-based restrictions, and a medium-severity CIS 1.15 posture finding for direct policy attachment on chris.

Figure 31 presents the JSON representation of the Phase One PassRole finding for this scenario.

```
{
  "check_id": "iam_passrole_without_pass_condition",
  "status": "FAIL",
  "severity": "HIGH",
  "evidence": {
    "details": {
      "violations": [
        {
          "principal_name": "cg-lambdaManager-role-cgidg1ojmi7my7",
          "policy_arn": "arn:aws:iam::344809605381:policy/cg-lambdaManager-policy-cgidg1ojmi7my7",
          "reason": "iam:PassRole granted on Resource:* without iam:PassedToService or aws:ResourceTag conditions"
        }
      ]
    }
  }
}
```

**Figure 31: Phase One PassRole detector firing on the lambda manager policy.**

Phase Two produces two attack paths for this scenario. The first is a two-hop path with HIGH risk. It starts from chris, moves through sts:AssumeRole into cg-lambdaManager, and then reaches cg-debug through the compound edge iam:PassRole\_via\_Lambda.

The iam:PassRole\_via\_Lambda edge represents the full Lambda-based escalation condition. It requires that the source principal can pass a role, create a Lambda function, and invoke that

function, while the destination role's trust policy allows the Lambda service to assume it. In this scenario, these conditions allow the attacker to create and invoke a Lambda function that runs under the privileged cg-debug role.

The second retained path is a one-hop path with CRITICAL risk, representing the same escalation primitive starting from cg-lambdaManager directly. Together, the two paths reconstruct the runtime attack chain observed in the scenario.

Figure 32 presents the JSON representation of the two-hop Phase Two attack path.

```
{
  "path_id": "1bbb794b",
  "risk_level": "HIGH",
  "hop_count": 2,
  "start_principal": {
    "arn": "arn:aws:iam::344809605381:user/chris-cgidg1ojmi7my7"
  },
  "end_principal": {
    "arn": "arn:aws:iam::344809605381:role/cg-debug-role-cgidg1ojmi7my7"
  },
  "steps": [
    {
      "edge_type": "sts:AssumeRole",
      "target_name": "cg-lambdaManager-role-cgidg1ojmi7my7"
    },
    {
      "edge_type": "iam:PassRole_via_Lambda",
      "target_name": "cg-debug-role-cgidg1ojmi7my7"
    }
  ]
}
```

**Figure 32: Phase Two two-hop attack path via sts:AssumeRole and iam:PassRole\_via\_Lambda.**

Phase Three produces eight alerts for this scenario. Two alerts reach the STRONGLY\_CORRELATED tier, while the remaining six are classified as PARTIALLY\_CORRELATED. All eight alerts are typed as ACTIVE\_EXPLOITATION.

The CRITICAL STRONGLY\_CORRELATED alert is linked to the lambdaManager-session assumed-role principal, which is created when chris calls sts:AssumeRole. The second STRONGLY\_CORRELATED alert corresponds to the AssumeRole event itself and is attributed directly to chris.

The headline privilege-escalation event is iam:AttachUserPolicy, where the AWS-managed AdministratorAccess policy is attached to chris. This alert is classified as PARTIALLY\_CORRELATED rather than STRONGLY\_CORRELATED because the CloudTrail identity for the AttachUserPolicy event is not chris directly. Instead, the event is recorded under the cg-debug assumed-role session created by the Lambda runtime.

This behavior is expected. In AWS CloudTrail, actions performed by a Lambda function are attributed to the execution role used by the function. Therefore, the framework correctly attributes the runtime action to cg-debug, the role under which the Lambda function executed. This explains why the static attack path begins with chris, while the final runtime privilege-escalation event appears under the Lambda execution role. The deterministic MITRE ATT&CK mapping for this behavior is T1098.003 — Additional Cloud Roles [15].

Figure 33 presents the JSON representation of the Phase Three LLM-augmented alert for the headline iam:AttachUserPolicy event.

```
{
  "alert_id": "d6ba3e761a25",
  "alert_type": "ACTIVE_EXPLOITATION",
  "correlation_tier": "PARTIALLY_CORRELATED",
  "overall_severity": "HIGH",
  "source_cloudtrail_event_names": ["AttachUserPolicy"],
  "principal_arns": ["arn:aws:sts::344809605381:assumed-role/cg-debug-rolecgidg1ojmi7my7/admin_function_cg1dg1ojmi7my7"],
  "affected_policies": ["arn:aws:iam::aws:policy/AdministratorAccess"],
  "source_phase1_check_ids": ["iam_aws_attached_policy_no_administrative_privileges", "iam_role_administratoraccess_policy"],
  "source_phase2_path_ids": [],
  "mitre": {
    "tactic": "Privilege Escalation",
    "sub_technique": "Additional Cloud Roles",
    "sub_technique_id": "T1098.003"
  },
  "llm_explanation":
    "The principal has attached a policy to a user without administrative
    privileges (check_id=iam_aws_attached_policy_no_administrative_privileges),
    and the role has administrator access (check_id=iam_role_administratoraccess_policy).
    This indicates potential privilege escalation.",
  "grounding_valid": true
}
```

**Figure 33: Phase Three LLM-augmented alert on the lambda\_privesc headline event.**

The verdict for this scenario is Pass. Phase One statically identifies the PassRole privilege-escalation primitive. Phase Two reconstructs the multi-hop attack chain using the iam:PassRole\_via\_Lambda compound edge type, which captures the Lambda-specific escalation pattern. Phase Three produces eight ACTIVE\_EXPLOITATION alerts, including two STRONGLY\_CORRELATED alerts on the role-assumption step and one HIGH PARTIALLY\_CORRELATED alert on the actual privilege-escalation event.

This scenario demonstrates that the framework can correctly attribute a runtime privilege-escalation effect to the privileged role passed to a Lambda function, even when the original attacker identity does not appear directly on the final privilege-escalation event.

#### 5-6-5 Cross-Technique Aggregate Scoring

This subsection summarizes the aggregate detection and grounding outcomes across the CloudGoat and Stratus evaluations. For the Stratus Red Team corpus, the detection rate is computed as the number of techniques whose primary CloudTrail event produced a Phase Three alert, divided by the total number of in-scope Stratus techniques. All seven Stratus techniques produced an alert for their declared primary event, resulting in a detection rate of 7/7, or **100%**. The Wilson 95% confidence interval is **[64.6%, 100.0%]**, reflecting the small denominator.

The LLM grounding rate is computed using one LLM-augmented headline alert per evaluated scenario or technique. Across the four CloudGoat scenarios and seven Stratus techniques, eleven headline alerts were evaluated. Ten of these alerts reported grounding\_valid = true, while one reported grounding\_valid = false. The resulting grounding rate is 10/11, or 90.9%, with a Wilson 95% confidence interval of [62.3%, 98.4%]. The single false grounding outcome does not affect deterministic detection; it indicates only that the hallucination guard flagged the LLM-generated explanation as insufficiently grounded.

**Table 5-5: Aggregate detection and LLM grounding outcomes across CloudGoat and Stratus scenarios.**

| Technique / scenario                        | Primary event           | Detection (k/n) | Grounding |
|---------------------------------------------|-------------------------|-----------------|-----------|
| CloudGoat 1:<br>iam_enum_basics             | (none - read-only)      | n/a             | True      |
| CloudGoat 2:<br>iam_privesc_by_key_rotation | CreateAccessKey         | 1/1             | True      |
| CloudGoat 3:<br>iam_privesc_by_rollback     | SetDefaultPolicyVersion | 1/1             | True      |
| CloudGoat 4:<br>lambda_privesc              | AttachUserPolicy        | 1/1             | True      |
| Stratus 1: iam-create-admin-user            | CreateUser              | 1/1             | True      |
| Stratus 2: iam-backdoor-user                | CreateAccessKey         | 1/1             | False     |
| Stratus 3: iam-backdoor-role                | UpdateAssumeRolePolicy  | 1/1             | True      |
| Stratus 4: iam-create-backdoor-role         | CreateRole              | 1/1             | True      |
| Stratus 5: iam-create-user-login-profile    | CreateLoginProfile      | 1/1             | True      |
| Stratus 6: sts-federation-token             | GetFederationToken      | 1/1             | True      |
| Stratus 7: iam-update-user-login-profile    | UpdateLoginProfile      | 1/1             | True      |

## 5-7 The Large-Language-Model Augmentation

This section summarizes the role of the Large Language Model (LLM) augmentation in the evaluation. The LLM layer is used as an explanation layer on top of the deterministic Phase Three correlator. It does not decide which alerts are emitted, does not change the correlation tier, does not change the severity, and does not modify the deterministic MITRE ATT&CK mapping.

For each LLM-augmented alert, the deterministic alert identifier, correlation tier, severity, MITRE label, and evidence references are preserved. The model adds a natural-language explanation, a priority justification, and an assessment of whether the activity appears benign or suspicious based on the supplied evidence bundle. Therefore, the LLM output is advisory and explanatory rather than part of the detection logic.

The hallucination guard validates each LLM-augmented alert against the evidence bundle supplied to the model. Any referenced IAM principal, Amazon Resource Name, Phase One check identifier, Phase Two path identifier, or CloudTrail event identifier must exist in the evidence bundle. If the model references unsupported information, the output is marked as ungrounded.

As reported in Section 5-6-5, ten of the eleven LLM-augmented headline alerts were marked as `grounding_valid = true`, while one was marked as `grounding_valid = false`. The false grounding outcome occurred in the Stratus iam-backdoor-user technique, where the model produced a paraphrased explanation that could not be directly anchored to a specific entity in the evidence bundle. This did not affect deterministic detection, tier assignment, severity, or MITRE mapping.

The LLM used throughout the evaluation is Llama-3.1-8B-Instruct, served locally through an OpenAI-compatible interface. The evaluation therefore measures whether a locally served general-purpose model can provide grounded explanations when constrained by deterministic evidence and a hallucination guard.

## 5-8 Threats to Validity

The evaluation has several limitations. First, the datasets and scenarios used in this chapter are controlled test environments rather than production AWS accounts. IAM Vulnerable, CloudGoat, Stratus Red Team, and the Pathfinding.cloud catalog provide useful ground truth for controlled evaluation, but they do not fully reflect the size, policy complexity, inherited permissions, and organizational structure of real production environments. Evaluation on production-scale AWS accounts is therefore left for future work.

Second, the evaluation does not include a full runtime comparison with all external IAM analysis tools discussed in Chapter Two. Such a comparison would require running each tool under the same AWS account, permission policy, and evaluation schema. Due to time and setup constraints, this direct tool-to-tool comparison is left for future work. The present evaluation therefore focuses on externally authored ground-truth benchmarks, catalog attribution where available, and staged attack scenarios.

Third, the Phase Two breadth-first search is designed to identify paths that reach admin-equivalent principals. As a result, it does not surface every form of lateral access to a sensitive but non-admin resource. For example, access to a single secret or service-specific resource may be detected through Phase One vulnerability-class detectors or Phase Three runtime correlation rather than through Phase Two graph reachability. This is a design boundary of the current graph model.

Fourth, the hallucination guard validates whether the LLM output references entities that exist in the evidence bundle, such as IAM principals, Amazon Resource Names, CloudTrail events, Phase One findings, and Phase Two paths. However, it does not fully verify whether the model always assigns the correct role to each entity, such as distinguishing the actor from the target of an action in every explanation. This means that a generated explanation may pass grounding checks while still containing a minor actor-target wording imprecision. Extending the guard to validate actor-target role assignment is left for future work.

Finally, the LLM used in Phase Three is a locally served quantized eight-billion-parameter general-purpose model. This setup supports reproducibility and local execution, but the output quality may be lower than that of larger or cybersecurity-specialized models. The evaluation therefore treats the LLM as an explanatory layer rather than a detection authority. The deterministic correlator remains responsible for alert generation, tier assignment, severity, and MITRE ATT&CK mapping. A future user study is needed to assess whether the generated explanations improve analyst understanding, triage speed, or remediation quality.

## 5-9 Conclusion

This chapter evaluated Cloud IAM Sentinel across its three phases using independent ground-truth corpora, staged CloudGoat attack scenarios, and Stratus Red Team single-technique tests. Phase

One was evaluated for adversarial accuracy on the nine adversarial principals of the Bishop Fox IAM Vulnerable free-resources module. Across the fourteen pre-committed (principal, rule) pairs, the framework produced zero incorrect verdicts, with three results recorded as acceptable over-approximations that reflect documented limits of static IAM analysis.

Phase Two was evaluated on two independent corpora. On the sixty-six-path Datadog Pathfinding.cloud catalog, it detected forty-eight paths under the loose criterion and forty-five under the strict criterion, exceeding the catalog-attributed counts for Principal Mapper and Pacu, with paired McNemar exact tests reaching Bonferroni-corrected significance. On the Bishop Fox IAM Vulnerable benchmark, Phase Two achieved 100% recall on the nineteen testable path classes present in the deployed subset and 64.5% recall on the thirty-one-path full denominator.

The integrated validation exercised the framework on four CloudGoat scenarios and seven Stratus Red Team techniques. The CloudGoat scenarios confirmed the framework’s complementary-detector design. The negative-control scenario produced zero `STRONGLY_CORRELATED` alerts and zero `ACTIVE_EXPLOITATION` alerts, while the three positive scenarios were detected through the appropriate combination of Phase One vulnerability-class detection, Phase Two reachability analysis, and Phase Three runtime correlation, depending on the attack pattern. The seven Stratus Red Team techniques were all detected at their declared primary CloudTrail event, resulting in a Phase Three detection rate of 7/7, or 100%, with a Wilson 95% confidence interval of [64.6%, 100.0%].

The LLM augmentation was evaluated separately from the deterministic correlator. Across the eleven headline alerts in the integrated corpus, ten were validated as grounded by the hallucination guard and one was flagged as ungrounded. This corresponds to a grounding rate of 10/11, or 90.9%, with a Wilson 95% confidence interval of [62.3%, 98.4%]. The single ungrounded outcome did not affect deterministic detection; it shows that the hallucination guard can reject insufficiently grounded model output rather than passing every LLM-generated explanation unconditionally.

Taken together, the results show that the framework’s three phases detect the attack patterns they are designed to detect across independent corpora and staged attack activity. They also show that the LLM augmentation layer can add grounded natural-language explanations without changing the deterministic alert decision, correlation tier, severity, or MITRE ATT&CK mapping.

# **Conclusion**

## 6-1 Summary of Contributions

This work presented Cloud IAM Sentinel, a hybrid AWS Identity and Access Management (IAM) security analysis framework that combines static posture analysis, privilege-escalation graph reachability, and dynamic CloudTrail correlation with grounded large language model (LLM) reasoning.

The framework makes four principal contributions. First, it provides a static IAM analysis layer that detects policy misconfigurations, credential risks, and attribute-based access control (ABAC) weaknesses. In particular, it evaluates IAM Condition logic and identifies ABAC vulnerability classes related to tag-based access control, as documented by the National Institute of Standards and Technology (NIST) and Amazon Web Services (AWS).

Second, the framework introduces an ABAC-aware privilege-escalation graph model that represents tag-write actions as state-transition edges. This allows the reachability analysis to detect escalation paths whose feasibility depends on the attacker first modifying a tag or attribute and then satisfying a Condition that was previously false.

Third, the framework provides a deterministic MITRE ATT&CK Cloud mapping for IAM-relevant CloudTrail events. Instead of asking the LLM to infer the ATT&CK technique, the framework applies a curated static catalog during correlation. This ensures that technique assignments are reproducible and verifiable.

Fourth, the framework introduces a grounding and hallucination guard for LLM-generated alert explanations. This guard checks whether the model’s factual claims are supported by the available evidence bundle and rejects unsupported output. As a result, the framework’s LLM-assisted reasoning remains tied to verifiable evidence rather than free-form model generation.

## 6-2 Summary of Results

The evaluation reported in Chapter Five establishes the framework’s performance on independent corpora, four end-to-end CloudGoat scenarios, and seven Stratus Red Team single-technique tests. Phase One achieved zero incorrect verdicts on the fourteen pre-committed (principal, rule) pairs of the Bishop Fox IAM Vulnerable adversarial principal set, with three acceptable over-approximations explicitly recognized as documented limits of static IAM analysis.

Phase Two detected 48 of 66 paths under the loose criterion and 45 of 66 paths under the strict criterion on the Datadog Pathfinding.cloud catalog at upstream commit f7170aff. These results exceeded the catalog-attribution counts for PMapper and Pacu, with all four paired McNemar exact tests reaching Bonferroni-corrected significance. Phase Two also achieved 100% recall on the nineteen testable IAM Vulnerable path classes present in the deployed free-resources subset, and 64.5% recall on the thirty-one-path Bishop Fox 2021 full denominator.

The integrated end-to-end validation on four CloudGoat scenarios exercised the framework across four distinct attack patterns. The negative-control scenario produced zero STRONGLY\_CORRELATED alerts and zero ACTIVE\_EXPLOITATION alerts, satisfying the strict-correlation negative-control criterion. The ABAC re-tagging scenario was detected through



the Phase One vulnerability-class detector and produced eight ACTIVE\_EXPLOITATION alerts with the correct deterministic MITRE ATT&CK sub-technique label at Phase Three, even though Phase Two returned no path because the privileged target was not admin-equivalent.

The policy-version rollback scenario was detected across all three layers: Phase One identified the privilege-escalation primitive in the policy text, Phase Two enumerated the self-escalation path, and Phase Three correlated the runtime call with both the static finding and the attack path, producing three CRITICAL STRONGLY\_CORRELATED alerts. The PassRole-via-Lambda scenario was also detected across all three layers: Phase One identified the iam:PassRole primitive, Phase Two reconstructed the multi-hop chain with the iam:PassRole\_via\_Lambda compound edge type, and Phase Three produced eight ACTIVE\_EXPLOITATION alerts, including two STRONGLY\_CORRELATED alerts on the role-assumption step and one HIGH PARTIALLY\_CORRELATED alert on the actual privilege-escalation event.

The seven Stratus Red Team techniques were each detected at their declared primary CloudTrail event, giving an aggregate Phase Three detection rate of 7/7, or 100%, across the in-scope techniques. Across the eleven headline alerts from the combined CloudGoat and Stratus corpus, the LLM augmentation layer achieved a grounding rate of 10/11, or 90.9%. The hallucination guard correctly flagged the single ungrounded explanation without affecting the deterministic detection outcome.

Together, these results demonstrate the framework's complementary-detector design: Phase One identifies static vulnerability patterns, Phase Two reconstructs reachable privilege-escalation paths, and Phase Three correlates observed CloudTrail activity with deterministic MITRE ATT&CK mapping and grounded alert explanation.

### **6-3 Limitations**

The evaluation also identified several limitations beyond those discussed in Section 5-8. First, the Phase Two breadth-first search is designed to stop when it reaches an admin-equivalent principal. Therefore, it may return no path when the attacker reaches a sensitive resource that is not equivalent to an admin role. In these cases, detection depends on the Phase One vulnerability detectors or on Phase Three CloudTrail correlation, rather than on graph reachability alone.

Second, the static analyzer cannot fully evaluate IAM Conditions that depend on runtime information, such as request time, source IP address, or session-specific attributes. When these values cannot be known from the inventory, the framework treats the condition as potentially satisfiable and marks the result with a confidence indicator. This avoids missing possible risks, but it may also produce some over-approximation.

Third, the Large Language Model used for grounded reasoning is a quantized eight-billion-parameter general-purpose model. Under strict formatting and grounding requirements, it may sometimes produce output that the system cannot read correctly or that is not fully supported by the evidence. The hallucination guard rejects such output, but this means that some alerts may not include populated LLM-generated explanation fields.

Finally, the framework was evaluated on controlled test environments and benchmark AWS scenarios rather than large production AWS accounts. These environments are useful because they provide known ground truth, but they do not fully represent the size, complexity, and policy density of real production accounts. Further testing on production-like environments is needed to evaluate the framework's scalability and operational effectiveness.

## **6-4 Future Work**

Several directions are identified for future work. The first direction is to deploy Phase Three in streaming mode. In the current implementation, CloudTrail events are processed from collected files. A production deployment could replace this with an event-driven design, where CloudTrail events are sent through Amazon EventBridge and processed by a Lambda-based adapter. IAM write events could trigger incremental updates to Phase One and Phase Two outputs, while other CloudTrail events could be sent directly to Phase Three for correlation. Since the correlation layer already accepts CloudTrail-shaped input, the framework is suitable for this type of streaming deployment.

The second direction is to evaluate the framework with a cybersecurity-specialized Large Language Model. The current evaluation uses a general-purpose model. Future work could compare this baseline with open-source models fine-tuned on cybersecurity data. To ensure a fair comparison, both models should be tested using the same evidence bundles, prompts, and hallucination guard. The results could then be compared using the same statistical testing approach used elsewhere in the evaluation.

The third direction is to validate the ABAC vulnerability-class detectors on larger and more realistic IAM policy datasets. In the present work, these detectors are validated using controlled test cases derived from documented attack patterns. This supports construct validity, but it does not fully show how the detectors behave on real production-scale policies. Future work should evaluate them against externally curated datasets containing real or production-like ABAC policy weaknesses.

The fourth direction is to conduct a user study of the LLM-generated explanations. The hallucination guard checks whether the model's output is supported by the evidence, but it does not measure whether the explanations are useful for human analysts. A future study with security practitioners could compare alerts with grounded LLM explanations against alerts without LLM support, measuring triage speed, explanation clarity, and remediation correctness.

The fifth direction is to extend the privilege-escalation graph beyond IAM principals to include long-lived service-managed objects. In the current framework, graph nodes are limited to IAM users, roles, and groups. As a result, some attack paths involving objects such as CloudFormation stacks, Systems Manager documents, or Elastic Container Service task definitions are outside the graph model, even when those objects carry or use service roles. Extending the graph to include these service-managed objects would allow the framework to represent additional attack paths that are currently out of scope. This extension is separate from the ABAC-focused contribution of the present work and is left as a future research direction.

# References

- [1] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," NIST Special Publication 800-145, National Institute of Standards and Technology, 2011.
- [2] Amazon Web Services, "AWS Identity and Access Management User Guide," AWS Documentation, 2024. [Online]. Available: <https://docs.aws.amazon.com/IAM/latest/UserGuide/>. [Accessed 2026].
- [3] Amazon Web Services, "AWS Shared Responsibility Model," AWS, 2024. [Online]. Available: <https://aws.amazon.com/compliance/shared-responsibility-model/>. [Accessed 2026].
- [4] Y. Wang and others, "Edge-enabled IAM for IoTs with edge-based access management and context-driven sync service," Journal of Systems Architecture, Elsevier, 2024.
- [5] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark, v3.0.0," CIS, 2024.
- [6] Y. Hu, S. Khurshid, and M. Tiwari, "Efficient IAM Greybox Penetration Testing," arXiv preprint arXiv:2304.14540, version 7, 2026.
- [7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.
- [8] V. C. Hu, D. Ferraiolo, R. Kuhn, A. R. Friedman, A. J. Lang, M. M. Cogdell, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," NIST Special Publication 800-162, National Institute of Standards and Technology, 2014.
- [9] V. C. Hu, D. R. Kuhn, and D. Yaga, "Attribute Considerations for Access Control Systems," NIST Special Publication 800-205, National Institute of Standards and Technology, 2019.
- [10] Amazon Web Services, "IAM JSON Policy Elements: Condition Operators," AWS Documentation, 2024. [Online]. Available: [https://docs.aws.amazon.com/IAM/latest/UserGuide/reference\\_policies\\_elements\\_condition\\_operators.html](https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_condition_operators.html). [Accessed 2026].
- [11] BishopFox, "IAM Vulnerable: A Benchmark of AWS IAM Privilege-Escalation Paths," BishopFox, 2024. [Online]. Available: <https://github.com/BishopFox/iam-vulnerable>. [Accessed 2026].
- [12] Amazon Web Services, "IAM Tutorial: Define Permissions to Access AWS Resources Based on Tags," AWS Documentation, 2024. [Online]. Available: [https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial\\_attribute-based-access-control.html](https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_attribute-based-access-control.html). [Accessed 2026].
- [13] Datadog Security Labs, "Pathfinding.cloud: An Open Catalogue of Cloud Privilege-Escalation Paths," Datadog, 2025. [Online]. Available: <https://github.com/DataDog/pathfinding.cloud>. [Accessed 2026].

- [14] Amazon Web Services, "AWS CloudTrail User Guide," AWS Documentation, 2024. [Online]. Available: <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/>. [Accessed 2026].
- [15] The MITRE Corporation, "MITRE ATT&CK for Cloud Matrix (IaaS)," MITRE ATT&CK Knowledge Base, 2024. [Online]. Available: <https://attack.mitre.org/matrices/enterprise/cloud/iaas/>. [Accessed 2026].
- [16] Joint Task Force, "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53 Revision 5, National Institute of Standards and Technology, 2020.
- [17] G. Adediran, K. Awuson-David, and Y. Ahmed, "Retrieval-Augmented Large Language Model for AWS Cloud Threat Detection and Modelling: CloudTrail MITRE ATT&CK Mapping," *Computers, Materials & Continua*, Tech Science Press, 2026, doi: 10.32604/cmc.2026.077606.
- [18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [19] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [20] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927. [Online]. Available: <https://www.jstor.org/stable/2276774>.
- [21] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. doi: 10.1007/BF02295996.
- [22] C. E. Bonferroni, "Teoria statistica delle classi e calcolo delle probabilità," *Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze*, vol. 8, pp. 3–62, 1936.
- [23] Y. Hu, W. Wang, P. Khurana, M. Lentz, and others, "Detecting Privilege Escalation in IAM Policies via Combined Symbolic and Learning-Based Analysis," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023.
- [24] R. Marbel, Y. Cohen, R. Dubin, A. Dvir, and C. Hajaj, "Cloudy with a Chance of Anomalies: Dynamic Graph Neural Network for Early Detection of Cloud Services' User Anomalies," 2024.
- [25] A. K. Kaleru, "Identity and access management: foundations, principles, and best practices," *World Journal of Advanced Engineering Technology and Sciences*, vol. 15, no. 1, pp. 1894–1904, 2025. DOI: 10.30574/wjaets.2025.15.1.0393.
- [26] P. Jain, "Identity and Access Management in the Cloud," *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 11, no. 2, pp. 1528–1535, 2025. DOI: 10.32628/CSEIT25112523.

- [27] S. Bello and others, "Scalable IAM Audit Framework for Distributed Cloud Security," ResearchGate publication 390928990, April 2025.
- [28] S. Bello and others, "A Zero Trust IAM Framework for Unified Auditing in Hybrid Distributed Systems," ResearchGate publication 390929151, April 2025.
- [29] V. T. Madireddy, "Graph Neural Network-Based Adaptive Threat Detection for Cloud Identity and Access Management Logs," arXiv preprint arXiv:2512.10280, 2025.
- [30] Dikshant and G. Verma, "Alert or Noise? Reducing False Positives with Active Behavioral Analysis for Cloud Security," 2025.
- [31] M. H. Nguyen, A. M. Ho, and B. S. To, "SkyEye: When Your Vision Reaches Beyond IAM Boundary Scope in AWS Cloud," arXiv preprint arXiv:2507.21094, 2025.
- [32] Cloud Security Alliance, "Top Threats to Cloud Computing 2025," CSA, 2025. [Online]. Available: <https://cloudsecurityalliance.org/research/topics/top-threats>. [Accessed 2026].
- [33] Microsoft Security, "2024 State of Multicloud Security Report," Microsoft, 2024. [Online]. Available: <https://www.microsoft.com/en-us/security/business/security-insider/>. [Accessed 2026].
- [34] B. Efron, "Bootstrap methods: Another look at the jackknife," *Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. doi: 10.1214/aos/1176344552.
- [35] Datadog Security Labs, "Stratus Red Team: Open-source adversary emulation framework for AWS, Azure, and Kubernetes," Datadog, 2025. [Online]. Available: <https://github.com/DataDog/stratus-red-team>. [Accessed 2026].